HyperionDev

Curriculum title	Python Programmer
Curriculum code	900221-000-00-00
Module code	900221-000-00-KM-02, Python Data Types and Structures
NQF level	4
Credit(s)	6
Quality assurance functionary	QCTO - Quality Council for Trades and Occupations
Originator	MICT SETA
Qualification type	Skills Programme

Python Data Types and Structures

Learner Guide

Name	
Contact Address	
Telephone (H)	
Telephone (W)	
Cellular	
Email	

Table of contents

Table of contents **2**
Introduction **5**
Purpose of the Knowledge Module **5**
Getting Started **6**
Knowledge Topic KM-02-KT01: **7**
1.1 Concept, definition and functions (KT0101) (IAC0101) 8
1.2 Syntax and characteristics (KT0102) (IAC0101) 9
1.3 String operators (KT0103) (IAC0101) 10
1.4 Replace, Join, Split, Reverse, Uppercase and Lowercase (KT0104) (IAC0101) 12
1.5 Python String strip () Function (KT0105) (IAC0101) 15
1.6 Python String count () Method (KT0106) (IAC0101) 17
1.7 Python String format () Function (KT0107) (IAC0101) 18
1.8 Python String length len() Method (KT0108) (IAC0101) 19
1.9 Python String find () Method (KT0109) (IAC0101) 20
1.10 Use of Backslash in Strings (KT0110) (IAC0101) 22
1.11 Convert Integer into String (KT0111) (IAC0101) 23
1.12 Python string length (KT0112) (IAC0101) 25
1.13 Escape sequences (KT0113) (IAC0101) 26
Formative Assessment Activity [1] 28
Knowledge Topic KM-02-KT02 **29**
2.1 Concept, definition and functions (KT0201) (IAC0201) 30
2.2 Syntax and characteristics (KT0202) (IAC0201) 33
2.3 Use of numbers in Python (KT0203) (IAC0201) 35
2.4 Three numeric types in Python: integers, floating-point numbers and complex numbers (KT0204) (IAC0201) 38
2.5 Numeric and decimal datatypes in Python (KT0205) (IAC0201) 41
2.6 Type conversion (KT0206) (IAC0201) 44
2.7 Arithmetic operations in python (KT0207) (IAC0201) 47
Formative Assessment Activity [2] 50
Knowledge Topic KM-02-KT03: **51**
3.1 Concept, definition and functions (KT0301) (IAC0301) 52
3.2 Syntax and characteristics (KT0302) (IAC0301) 54
3.3 Decimal values (KT0303) (IAC0301) 57
3.4 Use of float in Python (KT0304) (IAC0301) 59
3.5 Python float () method (KT0305) (IAC0301) 62
3.6 Floating-point numbers (KT0306) (IAC0301) 64
Formative Assessment Activity [3] 67
Knowledge Topic KM-02-KT04: **68**
4.1 Concept, definition and functions (KT0401) (IAC0401) 69

Copyright © 2025 HyperionDev. All rights reserved.

4.2 Syntax and characteristics (KT0402) (IAC0401) 71
4.3 Use of double data types in Python (KT0403) (IAC0401) 74
Formative Assessment Activity [4] 77
Knowledge Topic KM-02-KT05: 78
5.1 Concept, definition and functions (KT0501) (IAC0501) 79
5.2 Syntax and characteristics (KT0502) (IAC0501) 82
5.3 Use of strings in Python (KT0503) (IAC0501) 85
5.4 Two values: denoted true and false (KT0504) (IAC0501) 87
5.5 Problem solving and optimising code (KT0505) (IAC0501) 88
5.6 Uppercase (KT0506) (IAC0501) 92
Formative Assessment Activity [5] 93
Knowledge Topic KM-02-KT06: 94
6.1 Concept, definition and functions (KT0601) (IAC0601) 95
6.2 Syntax (KT0602) (IAC0601) 98
6.3 Use of Range Function in Python (KT0603) (IAC0601) 101
6.4 List in Python (KT0604) (IAC0601) 103
6.5 Mixed List in Python (KT0605) (IAC0601) 106
6.6 List Length Method (KT0606) (IAC0601) 109
6.7 List Remove and Del Method (KT0607) (IAC0601) 113
6.8 Python Append Method (KT0608) (IAC0601) 115
6.9 Python Sort Method (KT0609) (IAC0601) 118
6.10 Python List count() method (KT0610) (IAC0601) 119
6.11 Python List index() method (KT0611) (IAC0601) 120
6.12 List Comprehension and Reverse (KT0612) (IAC0601) 123
6.13 Nested lists (KT0613) (IAC0601) 126
Formative Assessment Activity [6] 129
Knowledge Topic KM-02-KT07: 130
7.1 Concept, definition and functions (KT0701) (IAC0701) 131
7.2 Syntax (KT0702) (IAC0701) 134
7.3 Tuple assignment (KT0703) (IAC0701) 136
7.4 Pack, Unpack, Compare, Slicing, Delete, Keys in dictionaries (KT0704) (IAC0701) 138
7.5 Built-in functions with tuples (KT0705) (IAC0701) 141
7.6 Advantages of tuples over lists (KT0706) (IAC0701) 145
Formative Assessment Activity [7] 147
Knowledge Topic KM-02-KT08: 148
8.1 Concept, definition and functions (KT0801) (IAC0801) 149
8.2 Syntax (KT0802) (IAC0801) 151
8.3 Properties of dictionary keys (KT0803) (IAC0801) 154
8.4 Python dictionary (Dict) (KT0804) (IAC0801) 157
8.5 Inbuilt methods (KT0805) (IAC0801) 160
8.6 Length of Dictionary (KT0806) (IAC0801) 165

Copyright © 2025 HyperionDev. All rights reserved.

8.7 Dictionary Clear Method (KT0807) (IAC0801) 166
8.8 Update, Cmp, Len, Sort, Copy, Items, str (KT0808) (IAC0801) 168
8.9 Merging dictionaries (KT0809) (IAC0801) 171
Formative Assessment Activity [8] 174
Knowledge Topic KM-02-KT09: 175
9.1 Concept, definition and functions (KT0901) (IAC0901) 176
9.2 Syntax (KT0902) (IAC0901) 179
9.3 Parameters (KT0903) (IAC0901) 183
9.4 Methods (KT0904) (IAC0901) 187
9.5 Operations performed on sets in Python (KT0905) (IAC0901) 191
Formative Assessment Activity [9] 194
Knowledge Topic KM-02-KT10: 195
10.1 Concept, definition and functions (KT1001) (IAC1001) 196
10.2 Syntax to create an array (KT1002) (IAC1001) 199
10.3 Python module (KT1003) (IAC1001) 201
10.4 Methods (KT1004) (IAC1001) 205
10.5 Array representation (KT1005) (IAC1001) 210
10.6 Array operations (KT1006) (IAC1001) 213
Formative Assessment Activity [10] 218
References 218

Copyright © 2025 HyperionDev. All rights reserved.

Introduction

Welcome to this skills programme!

This guide will help you understand the content we will cover. You will also complete several class activities as part of the formative assessment process. These activities give you a safe space to practise and explore new skills.

Make the most of this opportunity to gather information for your self-study and practical learning. In some cases, you may need to research and complete tasks in your own time.

Take notes as you go, and share your insights with your classmates. Sharing knowledge helps you and others deepen your understanding and apply what you've learned.

Purpose of the Knowledge Module

The main focus of the learning in this knowledge module is to build an understanding of the functionalities of Python data type and structures and when/how to use them

This learner guide will enable you to gain an understanding of the following topics. (The relative weightings within the module are also shown in the following table.)

Code	Topic	Weight
KM-02-KT01	Python strings	15%
KM-02-KT02	Numbers in Python	10%
KM-02-KT03	Float in Python	10%
KM-02-KT04	Double data types in Python	5%
KM-02-KT05	Boolean in Python	10%
KM-02-KT06	Python List	15%
KM-02-KT07	Python tuple	10%
KM-02-KT08	Dictionary in Python	10%
KM-02-KT09	Python sets	5%
KM-02-KT10	Arrays in Python	10%

Copyright © 2025 HyperionDev. All rights reserved.

Getting Started

To begin this module, click on the link to the **Learner Workbook** or download the PDF to easily access the content and obtain your workbook. This will prompt you to make a copy of the document, where you will complete all formative and summative assessments throughout the module. Ensure you save your copy in a secure location, as you will need to access it frequently.

Once you have made a copy, you can start working through the module materials. **Complete the assessments after reading and mastering the content covered in this module**. For each knowledge topic within the workbook, you may request a review of your work for assessment. Be sure to upload this document to your GitHub folder so your facilitator can review and mark it.



Take note

This module has a total of 400 marks available. To meet the passing requirements, you will need to achieve a minimum of 300 marks, which represents 75% of the total marks. Achieving this threshold will ensure that you have met the necessary standards for this module.

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT01:

Topic Code	KM-02-KT01:
Topic	Python strings
Weight	15%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0101)
- Syntax and characteristics (KT0102)
- String operators (KT0103)
- Replace, Join, Split, Reverse, Uppercase and Lowercase (KT0104)
- Python String strip() Function (KT0105)
- Python String count() Method (KT0106)
- Python String format() Function (KT0107)
- Python String length len() Method (KT0108)
- Python String find() Method (KT0109)
- Use of Backslash in Strings (KT0110)
- Convert Integer into String (KT0111)
- Python string length (KT0112)
- Escape sequences (KT0113)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0101 Definitions, functions and features of Python strings are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

1.1 Concept, definition and functions (KT0101) (IAC0101)

Python **strings** are immutable sequences of characters with various functions and features.

1. **Creating Strings**: Enclosed in single, double, or triple quotes. Strings are **immutable**.
2. **Concatenation and Repetition**: Use + to concatenate, * to repeat strings.
3. **Indexing and Slicing**: Access individual characters using indexing (s[0]), and extract substrings with slicing (s[1:4]).
4. **Common String Methods**:
    - **Case Changes**: lower(), upper(), capitalize().
    - **Trimming Whitespace**: strip(), lstrip(), rstrip().
    - **Find and Replace**: find(), replace().
    - **Splitting and Joining**: split(), join().
5. **String Formatting**: Use %, str.format(), or **f-strings** (f"My name is {name}") to format strings.
6. **Escape Sequences**: Use \ for special characters like newlines (\n) and quotes.
7. **Membership and Comparisons**: Use in to check substrings and compare strings lexicographically.
8. **Multiline Strings**: Triple quotes (''' or """) allow multiline strings.
9. **String Length and Iteration**: Use len() to get length and loop through characters.

These features make Python strings flexible and easy to manipulate.

Copyright © 2025 HyperionDev. All rights reserved.

1.2 Syntax and characteristics (KT0102) (IAC0101)

In Python, **strings** are a fundamental data type used to represent sequences of characters.

1. String Syntax

- **Creating Strings**: Strings can be created by enclosing characters in single quotes ('), double quotes ("), or triple quotes (''' or """).

```
single_quote_string = 'Hello'
double_quote_string = "World"
triple_quote_string = '''This is a
multiline string'''
```

- **Escape Characters**: Special characters like newlines (\n), tabs (\t), and quotes (\", \') can be included in strings using the backslash (\).

```
string_with_newline = "Hello\nWorld"
escaped_quote = "She said, \"Hello!\""
```

2. Characteristics of Strings

- **Immutability**: Once a string is created, it cannot be changed. Any operation that modifies a string returns a new string.

```
s = "Python"
s[0] = "J"  # This will raise an error since strings are immutable.
```

- **Indexing and Slicing**: Strings support **indexing** (accessing individual characters) and **slicing** (extracting substrings). Indexing starts at 0 for the first character and -1 for the last character.

```
s = "Python"
first_char = s[0]  # 'P'
slice_part = s[1:4]  # 'yth'
```


Copyright © 2025 HyperionDev. All rights reserved.

- **Concatenation and Repetition**: Strings can be combined using the + operator (concatenation) and repeated using the * operator.

```
greeting = "Hello" + " " + "World"  # 'Hello World'
repeated = "Hi!" * 3  # 'Hi!Hi!Hi!'
```

- **Length**: You can find the number of characters in a string using the len() function.

```
length = len("Python")  # 6
```

- **String Methods**: Strings come with many built-in methods for common tasks like changing case (lower(), upper()), trimming spaces (strip()), searching (find()), and replacing (replace()).

```
s = " hello "
s = s.strip()  # 'hello' (removes leading and trailing spaces)
```

Python strings are immutable sequences of characters, created with quotes, and they support indexing, slicing, concatenation, and a variety of built-in methods. The immutability and rich set of string methods make them versatile for text processing in Python.

1.3 String Operators (KT0103) (IAC0101)

In Python, strings have several operators that allow you to perform various operations such as concatenation, repetition, membership checking, comparison, and formatting.

1. Concatenation Operator (+)

The + operator is used to join two or more strings into a single string. This is commonly used to build complete messages or combine dynamic values into readable output.

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
first_name = "John"
last_name = "Doe"
full_name = first_name + " " + last_name
print(full_name)  # Output: John Doe
```

2. Repetition Operator (*)

The * operator repeats a string a specified number of times. It is useful when generating repeated patterns or default values.

Example:

```
separator = "-" * 10
print(separator)  # Output: ----------
```

3. Membership Operators (in, not in)

The `in` operator checks if a substring exists within another string.

The `not in` operator checks if a substring is **absent** from the string.

Example:

```
message = "Python is fun"
print("Python" in message)      # Output: True
print("Java" not in message)    # Output: True
```

4. Comparison Operators (==, !=, <, >, <=, >=)

These operators compare strings **lexicographically**, i.e., based on the Unicode values of the characters. This means "abc" < "bcd" because "a" comes before "b".

Examples:

Copyright © 2025 HyperionDev. All rights reserved.

```
print("apple" == "apple")    # Output: True
print("apple" != "banana")   # Output: True
print("apple" < "banana")    # Output: True
print("grape" > "grapefruit")  # Output: False
```

5. String Formatting Operators (%)

The `%` operator is used for **old-style** string formatting. It uses format specifiers like `%s` for strings, `%d` for integers, etc.

Example:

```
name = "Alice"
age = 25
info = "Name: %s, Age: %d" % (name, age)
print(info)  # Output: Name: Alice, Age: 25
```

Summary:

- `+`: Concatenates strings.
- `*`: Repeats strings.
- `in` / `not in`: Checks for substring presence.
- Comparison operators: Compare strings lexicographically.
- `%`: Formats strings (old-style).

1.4 String Methods – Replace, Join, Split, Reverse, Uppercase and Lowercase (KT0104) (IAC0101)

Python provides built-in string methods that allow you to manipulate strings efficiently.

Copyright © 2025 HyperionDev. All rights reserved.

1. `replace()`

Purpose: Replaces occurrences of a substring with another substring.

Syntax:

```
string.replace(old, new, count)
```

Example:

```
text = "I like Java"
new_text = text.replace("Java", "Python")
print(new_text)  # Output: I like Python
```

2. `join()`

Purpose: Joins elements of an iterable (like a list or tuple) into a single string using a specified separator.

Syntax:

```
separator.join(iterable)
```

Example:

```
words = ['one', 'two', 'three']
joined = ", ".join(words)
print(joined)  # Output: one, two, three
```

3. `split()`

Purpose: Splits a string into a list of substrings based on a delimiter.

Syntax:

```
string.split(separator, maxsplit)
```


Copyright © 2025 HyperionDev. All rights reserved.

Example:

```
sentence = "apple,banana,grape"
fruits = sentence.split(",")
print(fruits)  # Output: ['apple', 'banana', 'grape']
```

4. Reverse (using slicing)

Purpose: Reverses a string using slicing notation. Python does not have a built-in `reverse()` for strings.

Syntax:

`string[::-1]`

Example:

```
text = "Python"
print(text[::-1])  # Output: nohtyP
```

5. `upper()`

Purpose: Converts all characters in a string to uppercase.

Example:

```
name = "alice"
print(name.upper())  # Output: ALICE
```

6. `lower()`

Purpose: Converts all characters in a string to lowercase.

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
name = "ALICE"
print(name.lower())  # Output: alice
```

Summary of Methods:

- `replace()`: Replaces one substring with another.
- `join()`: Joins iterable elements into a string.
- `split()`: Splits a string into parts.
- `[::-1]`: Reverses the string.
- `upper()`: Converts to uppercase.
- `lower()`: Converts to lowercase.

1.5 Python `strip()` Function (KT0105) (IAC0101)

The `strip()` function is used to remove characters from the beginning and end of a string. By default, it removes whitespace.

Syntax:

`string.strip([chars])`

- `chars` (optional): A string specifying the characters to remove. If omitted, whitespace is removed.

Examples:

Removing whitespace:

Copyright © 2025 HyperionDev. All rights reserved.

```
text = "   Hello World   "
print(text.strip())  # Output: Hello World
```

Removing specific characters:

```
text = "###Header###"
print(text.strip("#"))  # Output: Header
```

Related Methods:

- `lstrip()`: Removes characters only from the beginning of the string.
- `rstrip()`: Removes characters only from the end of the string.

Examples:

```
text = "  spaced  "
print(text.lstrip())  # Output: "spaced  "
print(text.rstrip())  # Output: "  spaced"
```

Features of `strip()`:

- Removes leading and trailing whitespace by default.
- Can remove specified characters from both ends.
- Does **not** affect characters in the middle of the string.

Copyright © 2025 HyperionDev. All rights reserved.

1.6 Python String count() Method (KT0106) (IAC0101)

The `count()` method in Python is used to determine how many times a specific substring appears within a string. It allows you to count occurrences over the entire string or within a specified portion of the string.

Function of `count()`

Syntax:

```
string.count(substring, [start, end])
```

- `substring`: The part of the string you want to count.
- `start` (optional): The position in the string where the search starts. If omitted, it starts from the beginning.
- `end` (optional): The position in the string where the search ends. If omitted, it searches until the end of the string.

Example:

```
text = "apple banana apple grape apple"
print(text.count("apple"))           # Output: 3
print(text.count("apple", 10, 30))   # Output: 1
```

Features of `count()`:

- **Counts Substring Occurrences**: It returns the number of times the substring appears within the string.
- **Optional Start and End Parameters**: You can define where to start and end the search in the string by providing the start and end indices.

Copyright © 2025 HyperionDev. All rights reserved.

- **Case-Sensitive**: The method is case-sensitive, meaning "Hello" and "hello" are treated as different substrings.

```
text = "Hello hello HELLO"
print(text.count("Hello"))  # Output: 1
```

The `count()` method in Python counts how many times a specific substring occurs in a string. It is case-sensitive, and you can optionally specify the range to search within by using start and end indices. This method is useful for analysing the frequency of substrings in text.

1.7 Python String `format()` Function (KT0107) (IAC0101)

The `format()` method in Python is used to format strings by inserting values into placeholders within the string. It provides a flexible way to create dynamic strings by replacing placeholders with actual data.

Function of `format()`

Syntax:

```
string.format(value1, value2, ...)
```

- **Placeholders**: {} are used as placeholders within the string. The values passed into `format()` are inserted into these placeholders.

- You can use numbered or named placeholders to control where each value goes.

Example:

```
name = "Alice"
age = 25
print("Name: {}, Age: {}".format(name, age))  # Output: Name: Alice, Age: 25
```


Copyright © 2025 HyperionDev. All rights reserved.

Features of `format()`:

- **Positional and Named Placeholders**: You can insert values by position or by using named placeholders.
  Positional Example:

```
print("Name: {}, Age: {}".format("Bob", 30))  # Output: Name: Bob, Age: 30
```

Named Example:

```
print("Name: {n}, Age: {a}".format(n="Carol", a=28))  # Output: Name: Carol, Age: 28
```

- **Reordering Placeholders**: You can specify the order of values using index numbers inside the placeholders.

```
print("{1} comes after {0}".format("one", "two"))  # Output: two comes after one
```

- **Formatting Data**: You can also format numbers, dates, or specify alignment within the placeholders.

```
value = 12.34567
print("Rounded: {:.2f}".format(value))  # Output: Rounded: 12.35
```

The `format()` method is a flexible way to insert values into strings using placeholders {}. You can pass values either by position or by name, and you can control the format and

Copyright © 2025 HyperionDev. All rights reserved.

order of those values. This method is widely used for creating dynamic, readable, and well-formatted strings in Python.

1.8 Python String Length `len()` Method (KT0108) (IAC0101)

The `len()` function in Python is used to determine the length of a string (or any iterable such as lists or tuples). When applied to a string, it returns the number of characters in the string, including spaces, punctuation, and special characters.

Function of `len()`

Syntax:

`len(string)`

- **Parameter**: The string whose length you want to measure.
- **Returns**: An integer representing the number of characters in the string.

Example:

```
text = "Hello, World!"
print(len(text))  # Output: 13
```

Features of `len()`:

- **Counts All Characters**: The `len()` function counts all characters, including whitespace, punctuation, and special characters like `\n` (newlines) or `\t` (tabs).

```
text = "Line\nBreak"
print(len(text))  # Output: 10
```


Copyright © 2025 HyperionDev. All rights reserved.

- **Works with Empty Strings**: If the string is empty, `len()` returns 0.

```
empty = ""
print(len(empty))  # Output: 0
```

- **Fast and Efficient**: The `len()` function is optimized for performance and works quickly, even with large strings.

The `len()` function is used to find the number of characters in a string. It counts all characters, including spaces and punctuation, and returns an integer representing the string's length. This function is essential for tasks that require the analysis or manipulation of string lengths, such as input validation or slicing operations.

1.9 Python String find() Method (KT0109) (IAC0101)

The find() method in Python is used to locate the first occurrence of a specified substring within a string. It returns the index position of where the substring starts, or -1 if the substring is not found.

Syntax:

```
string.find(substring, start, end)
```

- substring: The substring you want to search for in the main string.
- start (optional): The index where the search begins (default is the start of the string).
- end (optional): The index where the search ends (default is the end of the string).
- Returns: The index of the first occurrence of the substring, or -1 if it is not found.

Copyright © 2025 HyperionDev. All rights reserved.

Example:

```
text = "Welcome to Python programming"
print(text.find("Python"))        # Output: 11
print(text.find("Java"))          # Output: -1
print(text.find("Python", 0, 20)) # Output: 11
```

Features of find():

- Searches for a Substring: It finds the first occurrence of the substring and returns its position in the string.

- Returns -1 if Not Found: If the substring does not exist in the string, find() returns -1.

- Optional Start and End Parameters: You can specify the range within which to search by using the optional start and end arguments.

- Case-Sensitive: The find() method is case-sensitive, meaning it distinguishes between uppercase and lowercase characters.

The find() method is used to locate the index of the first occurrence of a substring in a string. It returns the index if found or -1 if the substring is not present. The method is case-sensitive and can search within specific parts of a string using optional start and end parameters. It is useful for finding specific patterns or text within a string.

1.10 Use of Backslash in Strings (KT0110) (IAC0101)

The backslash (\) is used as an escape character within strings. It signals that the character following it has a special meaning and should not be interpreted literally. This allows you to include special characters, like newlines or tabs, or to escape quotes within strings.

Function of Backslashes

The backslash is used to insert special characters or escape certain characters within a string.

Syntax:
\ followed by a specific character or code.

Copyright © 2025 HyperionDev. All rights reserved.

Features of Using Backslashes:

- Escape Special Characters: You can use backslashes to escape quotes and other special characters in strings, preventing syntax errors.

Example:

```
quote = "She said, \"Hello!\""
print(quote)  # Output: She said, "Hello!"
```

- Newlines, Tabs, and Other Special Characters: Backslashes are used to insert special characters like:
    - Newline (\n): Moves the cursor to a new line.
    - Tab (\t): Inserts a tab space.

Example:

```
print("Line 1\nLine 2")   # Output: Line 1
                          #         Line 2
print("Name:\tJohn")      # Output: Name:    John
```

- Raw Strings: If you want to prevent backslashes from being treated as escape characters, you can use a raw string by prefixing the string with r.

Example:

```
path = r"C:\Users\John\Documents"
print(path)  # Output: C:\Users\John\Documents
```

- Unicode Characters: You can use backslashes to include Unicode characters in strings using `\u` followed by a Unicode code.

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
print("\u263A")  # Output: ☺ (Unicode smiley face)
```

The backslash (\) in Python strings is an escape character used to insert special characters (like newlines or tabs), escape quotes, and represent Unicode characters. It helps prevent certain characters from being interpreted literally, and you can use raw strings (`r"string"`) to ignore escape sequences when needed.

1.11 Convert Integer into String (KT0111) (IAC0101)

You often need to convert integers into strings to display or manipulate them alongside other text. Converting integers to strings allows you to treat numbers as text and perform operations like concatenation or formatting.

Function of Converting Integers to Strings

The most common way to convert an integer into a string is by using the `str()` function.

Syntax:

```
str(integer)
```

- `integer`: The number you want to convert to a string.

Example:

```
number = 123
string_number = str(number)
print(string_number)           # Output: '123'
print(type(string_number))     # Output: <class 'str'>
```

Features of Converting Integers to Strings:

- Allows String Concatenation: Once an integer is converted to a string, it can be easily concatenated with other strings.

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
age = 25
message = "I am " + str(age) + " years old."
print(message)  # Output: I am 25 years old.
```

- Works with String Formatting: Converting integers to strings allows for easy use in string formatting methods like `format()` or f-strings.

Example:

```
age = 30
print("Age: {}".format(str(age)))   # Output: Age: 30
print(f"Age: {str(age)}")           # Output: Age: 30
```

- Used for Displaying Numbers as Text: If you need to display an integer in the context of a message (e.g., in user interfaces, web pages), converting the integer to a string is necessary.

Example:

```
total = 1000
print("Your total is: " + str(total))  # Output: Your total is: 1000
```

To convert an integer to a string in Python, use the `str()` function. This conversion is essential for tasks like concatenating numbers with strings, displaying numbers as text, or including integers in formatted text**.**

1.12 Python String Length (KT0112) (IAC0101)

The length of a string refers to the total number of characters it contains, including letters, numbers, spaces, punctuation, and special characters. To find the length of a string, you use the built-in `len()` function.

Function of `len()` for String Length

The `len()` function is used to determine the length of a string. It returns an integer that represents the total number of characters in the string.

Syntax:

Copyright © 2025 HyperionDev. All rights reserved.

```
len(string)
```

- **`string`: The string whose length you want to measure.**

Example:

```
message = "Hello, Python!"
print(len(message))  # Output: 14
```

Features of Python String Length:

- Counts All Characters: The `len()` function counts all characters, including spaces, punctuation, and special characters like newlines (`\n`) or tabs (`\t`).

Example:

```
text = "Line1\nLine2"
print(len(text))  # Output: 11 (includes newline character)
```

- Works with Empty Strings: If the string is empty, `len()` will return 0, as there are no characters in the string.

Example:

```
empty = ""
print(len(empty))  # Output: 0
```

- Used for String Manipulation: Knowing the length of a string is useful for various operations like slicing, validating input, or determining loop boundaries.

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
text = "Welcome"
if len(text) > 5:
    print("This is a long string.")  # Output: This is a long string.
```

The `len()` function in Python is used to determine the length of a string, which includes all characters such as spaces and punctuation. It returns the total number of characters in the string as an integer and is helpful in tasks like string manipulation, validation, and text processing.

1.13 Escape sequences (KT0113) (IAC0101)

Escape sequences are special combinations of characters that start with a **backslash (\)** and are used to represent characters that would otherwise be difficult to include directly in a string, such as newlines, tabs, or quotes. Escape sequences allow you to insert these characters into strings without breaking the string syntax.

Function of Escape Sequences

Escape sequences allow for the inclusion of special characters in a string that are not normally allowed or would be difficult to type directly. For example, you can insert a newline (\n) or a tab (\t), or escape quotation marks inside a string.

- **Syntax**: \ followed by a specific character to represent a special character or action.

Example:

```
text = "Hello\nWorld"
print(text)
# Output:
# Hello
# World
```

Features of Escape Sequences:

Inserting Special Characters:

Copyright © 2025 HyperionDev. All rights reserved.

Escape sequences let you insert characters that have special meaning or formatting within strings.

- Newline (`\n`): Moves the text cursor to the next line.
- Tab (`\t`): Inserts a horizontal tab space.

Example:

```
print("Line1\nLine2")
# Output:
# Line1
# Line2

print("Column1\tColumn2")
# Output:
# Column1    Column2
```

Escaping Quotes:

Escape sequences can be used to include quote characters inside strings without ending the string early.

Example:

```
quote = 'She said, \"Hello!\"'
print(quote)
# Output: She said, "Hello!"
```

You can also use single quotes inside single-quoted strings:

```
text = 'It\'s a sunny day.'
print(text)
# Output: It's a sunny day.
```


Copyright © 2025 HyperionDev. All rights reserved.

Special Characters:

Escape sequences allow you to include non-printable or special characters.

- Backslash (\\): Inserts a literal backslash.

- Single quote (\'): Inserts a single quote in a string enclosed by single quotes.

Example:

```
print("File path: C:\\Users\\Name")
# Output: File path: C:\Users\Name

print('It\'s Friday.')
# Output: It's Friday.
```

Unicode Characters:

Escape sequences are used to insert Unicode characters using \u followed by the Unicode code.

Example:

```
print("\u2764")  # Unicode for a heart symbol
# Output: ❤
```

Escape sequences in Python are used to insert special characters into strings, such as newlines (\n), tabs (\t), quotes (\", \'), and Unicode characters. They are essential for handling text formatting and for including characters that are otherwise difficult to type or interpret within strings.



Formative Assessment Activity [1]

Python Strings

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT02

Topic Code	KM-02-KT02:
Topic	Numbers in Python
Weight	10%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0201)
- Syntax and characteristics (KT0202)
- Use of numbers in Python (KT0203)
- Three numeric types in Python: integers, floating-point numbers and complex numbers (KT0204)
- Numeric and decimal datatypes in Python (KT0205)
- Type conversion (KT0206)
- Arithmetic operations in python (KT0207)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0201 Definitions, functions and features of numbers in Python are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

2.1 Concept, definition and functions (KT0201) (IAC0201)

Numbers are a fundamental data type used to represent various kinds of numerical values. Python supports multiple types of numbers, including integers, floating-point numbers, and complex numbers, and offers numerous built-in operations and functions to work with them.

Types of Numbers in Python

1. **Integers (int)**:
    - Whole numbers, positive or negative, without a fractional component.
    - Examples: -10, 0, 42
    - No limit on the size of integers in Python.

Example:

Floating-Point Numbers (`float`):

```
Numbers with decimal points (or scientific notation).
 Examples: 3.14, -2.5, 1.5e3 (which equals 1500.0)
```

Example:

```
a = 3.14
b = -2.5
c = 1.5e3  # Scientific notation: 1.5 × 10^3 = 1500.0

print(a)  # Output: 3.14
print(b)  # Output: -2.5
print(c)  # Output: 1500.0
```

Complex Numbers (`complex`):

```
Numbers with a real and imaginary part, represented as a + bj, where a is the
real part and b is the imaginary part.
 Examples: 3+4j, 5j
```

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
x = 3 + 4j
y = 5j

print(x)        # Output: (3+4j)
print(x.real)   # Output: 3.0
print(x.imag)   # Output: 4.0
print(y)        # Output: 5j
```

Functions and Features of Numbers in Python

Basic Arithmetic Operations:

Python supports all standard mathematical operations, such as addition (+), subtraction (-), multiplication (*), division (/), and more.

Example:

```
a = 10
b = 3

print(a + b)  # Output: 13
print(a - b)  # Output: 7
print(a * b)  # Output: 30
print(a / b)  # Output: 3.333...
```

Integer Division and Modulus:

```
Integer Division (//): Returns the quotient of the division, discarding the
decimal part.
Modulus (%): Returns the remainder of the division.
```

Example:

```
print(10 // 3)  # Output: 3
print(10 % 3)   # Output: 1
```


Copyright © 2025 HyperionDev. All rights reserved.

Exponentiation:

Use ** to raise a number to the power of another.

Example:

```
print(2 ** 3)   # Output: 8
print(5 ** 2)   # Output: 25
```

Mathematical Functions:

Python's `math` module provides more advanced mathematical functions, such as square roots, trigonometric functions, logarithms, etc.

Example:

```
import math

print(math.sqrt(16))     # Output: 4.0
print(math.sin(math.pi)) # Output: 0.0
print(math.log10(100))   # Output: 2.0
```

Type Conversion:

Python supports converting between different number types:

- `int()`: Converts a value to an integer.
- `float()`: Converts a value to a float.
- `complex()`: Converts a value to a complex number.

Example:

```
a = int(3.99)        # Output: 3
b = float(5)         # Output: 5.0
```


Copyright © 2025 HyperionDev. All rights reserved.

```
c = complex(2, 3)    # Output: (2+3j)

print(a)
print(b)
print(c)
```

Automatic Type Promotion:

When performing operations between integers and floats, Python automatically promotes integers to floats to ensure the result is accurate.

Example:

```
x = 5       # int
y = 2.0     # float

result = x + y
print(result)        # Output: 7.0
print(type(result))  # Output: <class 'float'>
```

Python handles multiple types of numbers including **integers**, **floats**, and **complex numbers**. It provides a wide range of **basic arithmetic operations**, advanced mathematical functions through the math module, and supports **type conversion** between different numerical types. Python's flexible number handling and automatic type promotion make working with numbers simple and intuitive.

2.2 Syntax and characteristics (KT0202) (IAC0201)

Syntax and **characteristics** refer to the rules and features that define how code is written and how it behaves in a programming language.

1. Syntax

Syntax is the set of rules that defines the structure of a programming language. It determines how symbols, keywords, and punctuation should be used to create valid

Copyright © 2025 HyperionDev. All rights reserved.

instructions for the computer to execute. Just like grammar in a spoken language, syntax helps ensure that the code is understood by both the programmer and the interpreter/compiler.

- **Key Functions of Syntax**:
    - **Defines How Code is Written**: Syntax specifies how statements, variables, functions, and operators are used in a language.
    - **Ensures Correctness**: Proper syntax ensures that the code can be executed without errors by the interpreter or compiler.
    - **Provides Structure**: Syntax organizes code into understandable components, such as loops, conditionals, and functions.
- **Example (Python)**:

```python
# Syntax of a basic Python if statement
age = 18

if age >= 18:
    print("You are an adult.")
```

Common Syntax Features:

- **Variables**: Rules for declaring and using variables.
    - Example: x = 5
- **Control Structures**: Keywords and symbols for loops (for, while) and conditionals (if, else).
- **Functions**: Syntax for defining and calling functions.
    - Example: def my_function():

2. Characteristics

Copyright © 2025 HyperionDev. All rights reserved.

Characteristics of a programming language refer to the specific behaviours, features, and attributes that define how the language operates. These include concepts like data types, memory management, typing systems, and execution models.

- **Key Features of Characteristics**:
    - **Data Types**: Defines what types of data a language can handle (e.g., integers, strings, floats, etc.).
        - Example (Python): int, float, str
    - **Type System**: Refers to whether a language is **statically** or **dynamically typed**, meaning whether types are checked at compile-time or run-time.
        - Example: Python is dynamically typed, meaning variables do not need type declarations.
    - **Memory Management**: How a language handles memory allocation, such as automatic garbage collection.
    - **Execution Model**: Refers to whether the language is **compiled** or **interpreted**.
        - Example: Python is an interpreted language.
    - **Error Handling**: How the language handles errors (e.g., try-except blocks in Python).
- **Example (Python characteristics)**:
    - **Dynamic Typing**:

```
x = 10       # x is an integer
x = "Hello"  # x can be changed to a string without a type declaration
```

    - **Automatic Memory Management**: Python has built-in garbage collection, meaning it automatically handles memory cleanup for unused variables.
    - **Interpreted Language**: Python code is executed line by line by the Python interpreter, rather than being compiled into machine code.

Copyright © 2025 HyperionDev. All rights reserved.

Syntax defines the rules and structure for writing code, ensuring that it is correct and executable. **Characteristics** refer to the features of the language, such as its data types, typing system, memory management, and execution model. Together, syntax and characteristics help define how a language behaves and how programmers interact with it to create software.

2.3 Use of numbers in Python (KT0203) (IAC0201)

Numbers are a core data type used to perform mathematical operations and represent various kinds of numerical values. Python supports several types of numbers such as integers, floats, and complex numbers, each with its own specific functions and features. Python also offers a rich set of operations and functions to manipulate and work with numbers.

Functions of Numbers in Python

1. **Basic Arithmetic Operations**: Python supports standard arithmetic operations, including:
    - **Addition (+)**: Adds two numbers.
    - **Subtraction (-)**: Subtracts one number from another.
    - **Multiplication (*)**: Multiplies two numbers.
    - **Division (/)**: Divides one number by another (always returns a float).

Example:

```
a = 10
b = 5

sum_result = a + b          # 15
division_result = a / b     # 2.0
```

2. **Floor Division and Modulus**:

Copyright © 2025 HyperionDev. All rights reserved.

- **Floor Division (//)**: Divides two numbers and returns the integer result, discarding the decimal part.
- **Modulus (%)**: Returns the remainder when dividing two numbers.

Example:

```
result = 7 // 2             # 3 (integer division)
remainder = 7 % 2           # 1 (remainder of the division)
```

3. **Exponentiation**:
   - **Exponentiation (**)**: Raises a number to the power of another.

Example:

```
result = 2 ** 3             # 8 (2 raised to the power of 3)
```

4. **Type Conversion**: You can convert between different types of numbers using the built-in functions:
   - **int()**: Converts a value to an integer.
   - **float()**: Converts a value to a float.
   - **complex()**: Converts a value to a complex number.

Example:

```
a = 3.5
b = int(a)     # 3 (float to integer)
c = float(5)   # 5.0 (integer to float)
```

Features of Numbers in Python

Copyright © 2025 HyperionDev. All rights reserved.

1. **Dynamic Typing**: Python is dynamically typed, meaning you don't need to declare the type of a variable. The interpreter automatically assigns the appropriate type based on the value.
    - **Example**:

```
x = 10          # Integer
x = 3.14        # Now a float
```

2. **Support for Large Integers**: Python allows arbitrarily large integers, meaning you can work with numbers as large as your memory allows without worrying about overflow.
    - **Example**:

```
large_number = 99999999999999999999999999999999
```

3. **Floating-Point Precision**: Floating-point numbers (decimals) in Python have a limited precision due to how they are represented in memory. Python follows the IEEE 754 standard for floating-point arithmetic.
    - **Example**:

```
pi = 3.14159
result = pi * 2  # 6.28318
```

4. **Complex Numbers**: Python natively supports complex numbers, which have a real and imaginary part. Complex numbers are represented as a + bj, where a is the real part and b is the imaginary part.
    - **Example**:

```
z = 3 + 4j
```


Copyright © 2025 HyperionDev. All rights reserved.

```
result = z.real  # 3.0 (real part)
result = z.imag  # 4.0 (imaginary part)
```

5. **Math Module**: Python provides the **math** module, which includes advanced mathematical functions such as square roots, trigonometric functions, logarithms, and more.
    - **Example**:

```
import math

result = math.sqrt(16)  # 4.0
```

- Python supports different types of numbers: **integers**, **floats**, and **complex numbers**.
- It provides standard arithmetic operations like addition, subtraction, multiplication, division, and more.
- Python offers dynamic typing, meaning variables automatically take on the correct type, and supports arbitrarily large integers.
- The math module provides additional functionality for more advanced mathematical calculations.

2.4 Three numeric types in Python: integers, floating-point numbers and complex numbers (KT0204) (IAC0201)

There are three main types of numbers used to represent numerical data: **integers**, **floating-point numbers**, and **complex numbers**. Each type has unique characteristics and functions, allowing for versatile mathematical computations.

1. Integers (int)

Definition: Integers are whole numbers that can be positive, negative, or zero, and they do not have a decimal point.

Copyright © 2025 HyperionDev. All rights reserved.

Features:

- **No Size Limit**: Python's integers are of arbitrary precision, meaning you can work with very large or very small integers without worrying about overflow.
- **Operations**: Integers support all basic arithmetic operations such as addition, subtraction, multiplication, division (though division results in a float), and more.

Example:

```
x = 10   # Integer value
y = -5   # Negative integer
result = x + y   # 5 (basic arithmetic)
```

Common Operations:

- **Arithmetic**: +, -, *, // (integer division), % (modulus).
- **Example**:

```
a = 7
b = 3
quotient = a // b   # 2 (integer division)
remainder = a % b   # 1 (modulus)
```

2. Floating-Point Numbers (float)

Definition: Floating-point numbers (or floats) are numbers that contain a decimal point or are written in scientific notation (e.g., 1.5e3 for 1500.0).

Features:

- **Limited Precision**: Floats have limited precision and are stored in memory according to the IEEE 754 standard. This can lead to small rounding errors in some calculations.

Copyright © 2025 HyperionDev. All rights reserved.

- **Support for Decimal and Scientific Notation**: Floats can represent very large or very small numbers using scientific notation.

Example:

```
pi = 3.14159   # Float value
large_float = 1.2e6   # 1200000.0 (scientific notation)
```

Common Operations:

- Floats support the same arithmetic operations as integers, but division always returns a float.

Example:

```
result = 10 / 4   # 2.5 (division returns float)
```

3. Complex Numbers (complex)

Definition: Complex numbers consist of two parts: a real part and an imaginary part, represented as a + bj, where a is the real part and b is the imaginary part (with j denoting the imaginary unit).

Features:

- **Real and Imaginary Parts**: You can access the real and imaginary components of a complex number using .real and .imag attributes.
- **Mathematical Operations**: Complex numbers support addition, subtraction, multiplication, and division, and they follow the rules of complex number arithmetic.

Example:

```
z = 2 + 3j   # Complex number with real part 2 and imaginary part 3
real_part = z.real   # 2.0
imaginary_part = z.imag   # 3.0
```


Copyright © 2025 HyperionDev. All rights reserved.

Common Operations:

- **Addition/Subtraction**:

```
z1 = 1 + 2j
z2 = 3 + 4j
result = z1 + z2   # (4+6j)
```

- **Multiplication**:

```
z1 = 2 + 3j
z2 = 1 + 4j
result = z1 * z2   # (-10+11j)
```

- **Integers (int)**: Whole numbers without a decimal point, supporting unlimited size.
- **Floating-Point Numbers (float)**: Numbers with a decimal point or in scientific notation, used for more precise calculations but with limited precision.
- **Complex Numbers (complex)**: Numbers with real and imaginary parts, useful in scientific and engineering computations.

These three numeric types allow Python to handle a wide range of mathematical tasks, from basic arithmetic to complex number operations.

2.5 Numeric and decimal datatypes in Python (KT0205) (IAC0201)

Numerical data types allow you to work with various forms of numbers, and the **decimal data type** provides a more precise way to handle floating-point arithmetic. Python has built-in support for common numeric types like integers, floats, and

Copyright © 2025 HyperionDev. All rights reserved.

complex numbers, but also offers the decimal module for more accurate calculations when working with decimals.

1. Numerical Data Types in Python

Python provides three main numerical types:

a) Integers (int)

- **Definition**: Whole numbers, positive or negative, without a decimal point.
- **Key Features**:
    - **No size limit**: Python supports arbitrarily large integers.
    - **Operations**: Integers can be used in arithmetic operations like addition, subtraction, multiplication, and division.
    - **Example**:

```
x = 42      # Integer
y = -15     # Negative integer
result = x + y   # 27
```

b) Floating-Point Numbers (float)

- **Definition**: Numbers with decimal points or in scientific notation.
- **Key Features**:
    - **Limited precision**: Floats have a limited precision due to the way they are stored in memory (based on IEEE 754 standard). This can result in small rounding errors during calculations.
    - **Operations**: Like integers, floats support arithmetic operations.
    - **Example**:

```
pi = 3.14159   # Float
result = pi * 2   # 6.28318
```


Copyright © 2025 HyperionDev. All rights reserved.

c) Complex Numbers (complex)

- **Definition**: Numbers that consist of a real part and an imaginary part, written as a + bj, where a is the real part and b is the imaginary part.

- **Key Features**:
    - **Real and Imaginary Parts**: You can access the real and imaginary components of a complex number with .real and .imag.
    - **Useful in scientific computations**: Commonly used in fields like engineering and physics.
    - **Example**:

```
z = 2 + 3j

real_part = z.real       # 2.0
imaginary_part = z.imag  # 3.0
```

2. Decimal Data Type

Python's decimal module provides a **Decimal** data type for precise arithmetic, especially useful in financial and scientific applications where floating-point precision might lead to errors.

Key Features of Decimal:

- **Arbitrary Precision**: The Decimal type provides arbitrary precision, meaning it can handle very large or very small numbers with greater accuracy than floats.
- **Avoids Floating-Point Errors**: Decimal avoids common issues with floating-point representation by storing numbers as exact decimal values rather than binary fractions.
- **Customizable Precision**: You can set the precision level for Decimal operations.

Copyright © 2025 HyperionDev. All rights reserved.

- **Operations**: Supports the same arithmetic operations as integers and floats but with more reliable results in terms of precision.

How to Use Decimal:

- You must import the decimal module to use the Decimal type.
- **Example**:

```
from decimal import Decimal

x = Decimal('3.14159')  # More accurate representation
y = Decimal('2.5')
result = x * y  # Accurate multiplication without floating-point errors
print(result)   # 7.853975
```

Why Use Decimal Instead of float:

- **Precision Control**: If you need exact decimal representation, especially in financial calculations (e.g., currency), Decimal ensures that you don't run into the rounding errors common with float.
- **Rounding**: Decimal can be configured to round in various ways, providing more control over precision and rounding behaviour.

Numerical Data Types in Python include:

- **Integers (int)**: Whole numbers with no size limit.
- **Floats (float)**: Numbers with decimals, but with limited precision.
- **Complex numbers (complex)**: Numbers with real and imaginary parts for advanced mathematical computations.

Decimal Data Type:

Copyright © 2025 HyperionDev. All rights reserved.

- The **Decimal** type (from the decimal module) provides high precision for arithmetic, avoiding common floating-point errors, especially useful for financial and scientific applications.

2.6 Type conversion (KT0206) (IAC0201)

Type conversion refers to the process of converting one data type into another. This is useful when you need to perform operations that require specific data types, such as converting a string input to a number for mathematical operations or converting a number to a string for display purposes.

Python provides two types of type conversion:

1. **Implicit Type Conversion (Automatic)**: Python automatically converts one data type to another whenever necessary without explicit intervention by the programmer.
2. **Explicit Type Conversion (Casting)**: The programmer manually converts data from one type to another using specific functions.

1. Implicit Type Conversion (Automatic)

In **implicit type conversion**, Python automatically converts data types when it makes sense to do so without losing information or precision. This typically occurs when mixing different numerical types in an expression.

- **Example**:

```
x = 5        # Integer
y = 2.5      # Float

result = x + y  # Implicitly converts 'x' to a float before addition
print(result)   # 7.5 (float)
```

Key Features:

Copyright © 2025 HyperionDev. All rights reserved.

- **Automatic Promotion**: Python promotes integers to floats when necessary to avoid loss of precision.
- **Happens Without User Intervention**: Python handles the conversion behind the scenes.

2. Explicit Type Conversion (Casting)

In **explicit type conversion**, also known as **casting**, the programmer manually converts data from one type to another using built-in functions. This is necessary when Python cannot automatically convert types, or when you want to ensure data is in a specific format.

- **Functions for Type Conversion**:
    - **int()**: Converts data to an integer.
    - **float()**: Converts data to a floating-point number.
    - **str()**: Converts data to a string.
    - **list()**: Converts data to a list.
    - **tuple()**: Converts data to a tuple.
- **Example**:

```
# Converting string to integer
s = "10"
num = int(s)  # 'num' becomes an integer 10
print(num + 5)  # 15

# Converting float to string
f = 3.14
str_f = str(f)  # 'str_f' becomes the string "3.14"
print("Pi is approximately " + str_f)  # Concatenates string
```

Key Features:

- **Control Over Conversion**: Explicit conversion ensures that the data is converted exactly how you need it.

Copyright © 2025 HyperionDev. All rights reserved.

- **Casting Functions**: Several built-in functions are used to cast between types, such as int(), float(), str(), and list().

Common Use Cases for Type Conversion

1. **Converting User Input**: Since user input in Python is always a string, converting the input to an integer or float is necessary for numerical operations.
    - **Example**:

```
age = int(input("Enter your age: "))
print("You will be", age + 1, "next year.")
```

2. **Handling Mathematical Operations**: When performing arithmetic, you may need to convert between floats and integers to ensure accurate results.
    - **Example**:

```
num = 10
result = float(num) / 3  # Convert integer to float for accurate division
print(result)  # 3.3333...
```

3. **Formatting for Output**: Converting numbers to strings when concatenating with text.
    - **Example**:

```
price = 19.99
print("The price is $" + str(price))
```

- **Implicit Conversion**: Python automatically converts types, when necessary, usually promoting integers to floats in mixed operations.

Copyright © 2025 HyperionDev. All rights reserved.

- **Explicit Conversion (Casting)**: Programmers use functions like int(), float(), and str() to manually convert between types for specific purposes.

Type conversion is essential in Python when working with different data types, ensuring that your code operates correctly, and data is in the correct format for the task at hand.

2.7 Arithmetic operations in python (KT0207) (IAC0201)

Arithmetic operations in Python allow you to perform basic mathematical calculations such as addition, subtraction, multiplication, division, and more. These operations can be performed on various numerical types such as integers, floats, and complex numbers. Python follows standard mathematical rules for these operations and provides additional features to handle more complex calculations.

Functions of Arithmetic Operations

1. **Addition (+)**:
    - Adds two values together.
    - Works with integers, floats, and complex numbers.
    - **Example**:

```
Result = 10 + 5  # 15
```

2. **Subtraction (-)**:
    - Subtracts one value from another.
    - **Example**:

```
a = 10
b = 4
print(a - b)  # Output: 6
```

3. **Multiplication (*)**:
    - Multiplies two values.
    - **Example**:

```
x = 5
y = 3
print(x * y)  # Output: 15
```


Copyright © 2025 HyperionDev. All rights reserved.

4. **Division (/)**:
    - Divides one value by another and always returns a float, even if both operands are integers.
    - **Example**:

```
result = 8 / 2
print(result)        # Output: 4.0
print(type(result))  # Output: <class 'float'>
```

5. **Floor Division (//)**:
    - Divides one value by another and returns the integer (floor) result by discarding the decimal part.
    - **Example**:

```
print(7 // 2)  # Output: 3
```

6. **Modulus (%)**:
    - Returns the remainder of a division.
    - **Example**:

```
print(10 % 3)  # Output: 1
```

7. **Exponentiation (**)**:
    - Raises a number to the power of another number.
    - **Example**:

```
print(2 ** 4)  # Output: 16
```

Features of Arithmetic Operations in Python

Copyright © 2025 HyperionDev. All rights reserved.

1. **Supports Mixed Types**:
    - Python can perform arithmetic operations between different types (e.g., integers and floats) and will automatically promote the result to the more precise type.
    - **Example**:

```
a = 5     # int
b = 2.0   # float
print(a + b)        # Output: 7.0
print(type(a + b))  # Output: <class 'float'>
```

2. **Automatic Type Conversion**:
    - Python automatically converts the result of arithmetic operations based on the types of the operands (e.g., integer division always returns a float).
    - **Example**:

```
x = 9
y = 2
print(x / y)        # Output: 4.5
print(type(x / y))  # Output: <class 'float'>
```

3. **Handling of Complex Numbers**:
    - Python allows arithmetic operations on complex numbers, where the real and imaginary parts are handled separately.
    - **Example**:

```
a = 2 + 3j
b = 1 + 1j
print(a + b)  # Output: (3+4j)
print(a * b)  # Output: (-1+5j)
```

4. **Chained Arithmetic Operations**:
    - You can chain multiple arithmetic operations together, and Python will follow the order of operations (PEMDAS: Parentheses, Exponents, Multiplication/Division, Addition/Subtraction).

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
result = 2 + 3 * 4 ** 2
print(result)  # Output: 50
# Because 4 ** 2 = 16, then 3 * 16 = 48, then 2 + 48 = 50
```

5. **Negative Numbers**:
    - Python supports operations on negative numbers just as it does for positive ones.
    - **Example**:

```
a = -5
b = 3
print(a + b)  # Output: -2
print(a * b)  # Output: -15
```

- Python supports a wide range of **arithmetic operations**: addition, subtraction, multiplication, division, floor division, modulus, and exponentiation.
- These operations work with different number types (integers, floats, complex numbers) and automatically handle type conversions.
- Python follows standard mathematical rules (PEMDAS) for the order of operations and allows chaining multiple operations in a single expression.



Formative Assessment Activity [2]

Numbers in Python

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT03:

Topic Code	KM-02-KT03:
Topic	Float in Python
Weight	10%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0301)
- Syntax and characteristics (KT0302)
- Decimal values (KT0303)
- Use of float in Python (KT0304)
- Python float () method (KT0305)
- Floating-point numbers (KT0306)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0301 Definitions, functions and features of Python float are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

3.1 Concept, definition and functions (KT0301) (IAC0301)

The **float** type represents real numbers that have a **decimal point**. These are used to store numbers that require more precision, such as fractions or values that result from division.

Functions of Python float

1. **Represents Decimal Numbers**:
    - **float** handles numbers with decimal points, such as 3.14, -0.5, or numbers in scientific notation like 1.2e3 (which equals 1200.0).
    - **Example**:

```
a = 3.14
b = -0.5
c = 1.2e3

print(a)  # Output: 3.14
print(b)  # Output: -0.5
print(c)  # Output: 1200.0
```

2. **Supports Arithmetic Operations**:
    - Floats can be used in arithmetic operations such as addition (+), subtraction (-), multiplication (*), division (/), and more.
    - **Example**:

```
x = 2.5
y = 1.2

print(x + y)  # Output: 3.7
print(x * y)  # Output: 3.0
```

3. **Type Conversion**:

Copyright © 2025 HyperionDev. All rights reserved.

- Python provides the float() function to convert other types (e.g., integers or strings) into floats.
- **Example**:

```
a = float(5)         # Converts integer to float
b = float("3.14")    # Converts string to float

print(a)  # Output: 5.0
print(b)  # Output: 3.14
```

4. **Scientific Notation**:
   - Floats can represent very large or small numbers using scientific notation, such as 1.2e6 (which equals 1200000.0).
   - **Example**:

```
num = 1.2e6
print(num)         # Output: 1200000.0
print(type(num))   # Output: <class 'float'>
```

Features of Python float

1. **Limited Precision**:
   - **Floats are not exact**: They are stored using binary floating-point representation (IEEE 754), which can lead to small rounding errors in some calculations.
   - **Example**:

```
print(0.1 + 0.2)  # Output: 0.30000000000000004 (not exactly 0.3)
```

2. **Automatic Type Conversion**:

Copyright © 2025 HyperionDev. All rights reserved.

- Python automatically promotes integers to floats in mixed operations (e.g., integer + float).
- **Example**:

```
result = 3 + 2.5
print(result)        # Output: 5.5
print(type(result))  # Output: <class 'float'>
```

3. **Infinite and NaN Values**:
   - Python can represent special float values like **infinity (inf)** and **Not a Number (NaN)**.
   - **Example**:

```
import math

print(math.inf)       # Output: inf
print(-math.inf)      # Output: -inf
print(math.nan)       # Output: nan
print(math.sqrt(-1))  # Output: nan (via math domain error with error
handling)
```

To safely create NaN or infinity:

```
a = float('inf')
b = float('-inf')
c = float('nan')

print(a, b, c)  # Output: inf -inf nan
```

4. **Math Operations**:
   - Floats can be used with Python's math module, which provides functions like math.sqrt(), math.sin(), math.exp(), etc., for more complex operations.
   - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
import math

x = 16.0
print(math.sqrt(x))     # Output: 4.0
print(math.exp(2))      # Output: 7.38905609893065
print(math.sin(math.pi / 2))  # Output: 1.0
```

- The **float** type in Python is used to represent real numbers with decimal points.
- It supports standard arithmetic operations and allows conversions from other types.
- **Features** include automatic promotion in mixed operations, the ability to represent large numbers in scientific notation, and handling special values like infinity and NaN.
- Due to limited precision, floats may have small rounding errors, which should be considered in calculations requiring high accuracy.

3.2 Syntax and characteristics (KT0302) (IAC0301)

Float is a data type used to represent **real numbers** that have a **decimal point**. Floats are essential when dealing with numbers that are not whole, such as fractions, measurements, or numbers that result from division.

Syntax of Python float

1. **Basic Syntax**:
    - A float can be created by writing a number with a decimal point or by using scientific notation for very large or small numbers.
    - **Examples**:

```
x = 3.14        # standard float
y = -0.25       # negative float
z = 1.5e4       # scientific notation (1.5 × 10^4 = 15000.0)

print(x)  # Output: 3.14
print(y)  # Output: -0.25
```


Copyright © 2025 HyperionDev. All rights reserved.

```
print(z)  # Output: 15000.0
```

2. **Type Conversion**:
    - You can convert other data types (like integers or strings) to float using the float() function.
    - **Example**:

```
a = float(10)        # integer to float
b = float("5.75")    # string to float

print(a)  # Output: 10.0
print(b)  # Output: 5.75
```

Characteristics of Python float

1. **Limited Precision**:
    - Floats are stored using the **IEEE 754 standard**, which means they have limited precision. This can lead to small **rounding errors** when performing arithmetic operations with floats.
    - **Example**:

```
print(0.1 + 0.2)  # Output: 0.30000000000000004 (not exactly 0.3)
```

2. **Automatic Type Conversion**:
    - When performing arithmetic between an integer and a float, Python **automatically promotes** the integer to a float to ensure precision in the result.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
result = 5 + 2.5
print(result)        # Output: 7.5
print(type(result))  # Output: <class 'float'>
```

3. **Scientific Notation Support**:
    - Floats can be represented in **scientific notation**, using the letter e or E to indicate powers of 10. This is useful for working with very large or very small numbers.
    - **Example**:

```
big_number = 3e8      # 3 × 10^8
small_number = 4.5e-5 # 4.5 × 10^-5

print(big_number)    # Output: 300000000.0
print(small_number)  # Output: 4.5e-05
```

4. **Infinity and NaN (Not a Number)**:
    - Python supports special float values like **positive infinity (inf)**, **negative infinity (-inf)**, and **Not a Number (NaN)**. These values are useful in calculations that go beyond the normal range of numbers.
    - **Example**:

```
import math

print(float('inf'))     # Output: inf
print(float('-inf'))    # Output: -inf
print(float('nan'))     # Output: nan
print(math.isinf(math.exp(1000)))  # Output: True (may raise OverflowError in
some environments)
```


Copyright © 2025 HyperionDev. All rights reserved.

5. **Memory Efficiency**:
    - Floats use **64 bits** of memory, providing sufficient precision for many real-world applications but are less precise than **Decimal** (from the decimal module) when exact decimal representation is required, such as in financial calculations.
    - **Example**

```
from decimal import Decimal

a = 0.1 + 0.2
b = Decimal('0.1') + Decimal('0.2')

print(a)  # Output: 0.30000000000000004 (float)
print(b)  # Output: 0.3 (Decimal)
```

6. **Arithmetic Operations**:
    - Floats support all standard arithmetic operations: addition, subtraction, multiplication, division, and exponentiation.
    - **Example**:

```
a = 5.0
b = 2.0

print(a + b)  # Output: 7.0
print(a - b)  # Output: 3.0
print(a * b)  # Output: 10.0
print(a / b)  # Output: 2.5
print(a ** b) # Output: 25.0
```

- **Syntax**: A Python float is created by writing a number with a decimal point or in scientific notation. You can also convert other data types to floats using float().

Copyright © 2025 HyperionDev. All rights reserved.

- **Characteristics**:
    - Floats have **limited precision**, which can lead to rounding errors.
    - Python automatically converts integers to floats in arithmetic operations to ensure precision.
    - Floats support **scientific notation** and special values like **infinity** and **NaN**.
    - Floats use **64 bits** of memory and support all standard arithmetic operations.

3.3 Decimal values (KT0303) (IAC0301)

Float is a data type used to represent **real numbers** with **decimal points**. These numbers can include both positive and negative values, as well as very large or very small numbers written in scientific notation. However, due to the way floats are represented internally in memory, there are important characteristics about their precision and behaviour with decimal values.

Key Characteristics of Decimal Values in Python float

1. **Floating-Point Representation**:
    - Python's float type uses **floating-point arithmetic**, following the **IEEE 754 standard**, which represents numbers as binary fractions. This allows Python to handle a wide range of decimal numbers efficiently but introduces **limitations in precision**.
    - **Example**:

```
a = 0.5       # exactly representable
b = 0.1       # not exactly representable in binary
print(a)      # Output: 0.5
print(b)      # Output: 0.1 (approximate)
```

2. **Limited Precision**:

Copyright © 2025 HyperionDev. All rights reserved.

- Floats can only store a limited number of **significant digits**. This limitation can lead to **rounding errors** when performing arithmetic on decimal values because not all decimal numbers can be represented exactly in binary.
- **Example**:

```
x = 0.1 + 0.2
print(x)            # Output: 0.30000000000000004
print(x == 0.3)     # Output: False
```

3. **Decimal Approximation**:
   - Certain decimal values, such as 0.1, cannot be exactly represented in binary, so they are stored as **approximations**. When performing calculations with these values, small errors may accumulate.
   - **Example**:

```
value = 0.1
total = value * 3
print(total)        # Output: 0.30000000000000004
```

4. **Scientific Notation**:
   - Floats can represent very large or small decimal values using **scientific notation**. This is done using the letter e or E to represent powers of 10.
   - **Example**:

```
big = 1.2e6      # 1.2 × 10^6 = 1200000.0
small = 4.5e-4   # 4.5 × 10^-4 = 0.00045

print(big)       # Output: 1200000.0
print(small)     # Output: 0.00045
```


Copyright © 2025 HyperionDev. All rights reserved.

5. **Automatic Type Promotion**:
    - When you mix float and int values in arithmetic operations, Python automatically promotes the result to float to preserve decimal precision.
    - **Example**:

```
a = 5      # int
b = 2.5    # float
result = a + b

print(result)        # Output: 7.5
print(type(result))  # Output: <class 'float'>
```

6. **Handling Large and Small Decimal Values**:
    - Floats can handle very large or very small numbers using **scientific notation**, but as the size of the number increases, precision decreases due to rounding.
    - **Example**:

```
large = 1.7976931348623157e+308  # Largest representable float
small = 5e-324                   # Smallest positive float (subnormal)

print(large)  # Output: 1.7976931348623157e+308
print(small)  # Output: 5e-324

# Going beyond limits
import math
print(math.isinf(large * 10))  # Output: True (infinite)
```

- **Python float** represents real numbers with **decimal values** using the **IEEE 754 floating-point standard**, which provides wide-range decimal representation but with limited precision.

Copyright © 2025 HyperionDev. All rights reserved.

- Floats can handle both very large and very small numbers through **scientific notation**.
- Due to the **binary representation** of floats, certain decimal numbers are stored as approximations, leading to **rounding errors** in calculations.
- For applications requiring exact decimal precision, such as financial calculations, Python's float may not be ideal, and you might want to consider using the **Decimal** type from the decimal module instead.

Floats are ideal for general-purpose calculations involving decimal numbers but should be used carefully when exact precision is required.

3.4 Use of float in Python (KT0304) (IAC0301)

The **float** data type is used to represent **real numbers** that contain **decimal points**. Floats are essential when dealing with numbers that are not whole, such as fractions, measurements, and numbers that result from division. This data type is widely used for mathematical computations requiring precision with decimal values.

Key Uses of float in Python

1. **Representing Decimal Numbers**:
    - Floats are used to represent numbers with decimal points, such as 3.14, 0.5, or -2.75.
    - **Example**:

```
pi = 3.14
half = 0.5
negative = -2.75

print(pi, half, negative)  # Output: 3.14 0.5 -2.75
```


Copyright © 2025 HyperionDev. All rights reserved.

2. **Performing Arithmetic with Decimals**:
    - Python's float supports all standard arithmetic operations, such as addition, subtraction, multiplication, and division, while maintaining precision with decimal numbers.
    - **Example**:

```
a = 1.5
b = 2.25

print(a + b)  # Output: 3.75
print(a * b)  # Output: 3.375
```

3. **Handling Division**:
    - When performing division in Python using /, even if both operands are integers, the result is returned as a float to preserve precision.
    - **Example**:

```
print(7 / 2)        # Output: 3.5
print(type(7 / 2))  # Output: <class 'float'>
```

4. **Working with Large or Small Numbers (Scientific Notation)**:
    - Floats can handle very large or very small numbers using **scientific notation**. This is useful when working with values like scientific constants or extremely large datasets.
    - **Example**:

```
speed_of_light = 3e8      # 3 × 10^8
mass_of_electron = 9.1e-31

print(speed_of_light)     # Output: 300000000.0
print(mass_of_electron)   # Output: 9.1e-31
```


Copyright © 2025 HyperionDev. All rights reserved.

5. **Converting Data Types to Float**:
    - You can use the float() function to convert other types (e.g., integers, strings) into floats when necessary, such as when processing user input or performing precise calculations.
    - **Example**:

```
user_input = "42.5"
converted = float(user_input)

print(converted)         # Output: 42.5
print(type(converted))   # Output: <class 'float'>
```

6. **Handling Precision and Rounding Errors**:
    - Because floats in Python follow the IEEE 754 standard, some decimal numbers cannot be represented exactly, leading to **rounding errors** in certain calculations. For example, 0.1 + 0.2 may not exactly equal 0.3 due to how floats are stored in memory.
    - **Example**:

```
result = 0.1 + 0.2
print(result)       # Output: 0.30000000000000004
print(result == 0.3)  # Output: False
```

7. **Automatic Type Promotion**:
    - Python automatically converts integers to floats when mixing data types in arithmetic operations to ensure precision.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
a = 5     # int
b = 2.0   # float

print(a + b)        # Output: 7.0
print(type(a + b))  # Output: <class 'float'>
```

Common Use Cases for float

- **Financial Calculations**: Handling prices, interest rates, or other decimal-based values.

```
price = 19.99
tax_rate = 0.15
total = price + (price * tax_rate)
print(total)  # Output: 22.9885
```

- **Scientific Computations**: Managing measurements, constants, or data in scientific notation.

```
avogadro = 6.022e23
print(avogadro)  # Output: 6.022e+23
```

- **Divisions**: Calculating ratios or averages where the result is not necessarily an integer.

```
total_points = 85
max_points = 100
average = total_points / max_points
print(average)  # Output: 0.85
```


Copyright © 2025 HyperionDev. All rights reserved.

- **Precision Operations**: Performing calculations that require precise decimal values.

Float is a data type used in Python to represent **decimal numbers**.

It supports **arithmetic operations**, can handle large and small values using **scientific notation**, and automatically promotes integers to floats in mixed operations.

Floats are commonly used in scientific, financial, and mathematical calculations that require decimal precision.

It's important to be aware of potential **rounding errors** due to how floats are stored in memory. For applications that require exact decimal precision (e.g., currency calculations), consider using Python's **Decimal** type from the decimal module.

3.5 Python float () method (KT0305) (IAC0301)

The **float()** method in Python is used to **convert a value** (such as an integer, string, or other numeric type) into a **floating-point number** (a number with a decimal point). This method is useful when you need to ensure that a value is represented as a **float** for calculations or further operations that require decimal precision.

Syntax of float()

`float(value)`

- **value**: The input that you want to convert to a float. It can be an integer, string, or another type that represents a number.

Common Uses of float()

1. **Converting an Integer to a Float**:
    - You can use float() to convert an integer into a float, adding a decimal point.
    - **Example**:

```
x = 10
y = float(x)
```


Copyright © 2025 HyperionDev. All rights reserved.

```
print(y)         # Output: 10.0
print(type(y))   # Output: <class 'float'>
```

2. **Converting a String to a Float**:
    - If a string contains a numeric value, float() can convert it to a float, provided the string is formatted correctly.
    - **Example**:

```
s = "3.14159"
f = float(s)
print(f)         # Output: 3.14159
print(type(f))   # Output: <class 'float'>
```

3. **Handling Special Numeric Values**:
    - The float() method can also handle special cases like **infinity (inf)** and **Not a Number (NaN)**.
    - **Example**:

```
print(float('inf'))   # Output: inf
print(float('-inf'))  # Output: -inf
print(float('nan'))   # Output: nan
```

4. **Converting from Other Data Types**:
    - The float() method can convert values from other types, like boolean values, where True becomes 1.0 and False becomes 0.0.
    - **Example**:

```
print(float(True))   # Output: 1.0
print(float(False))  # Output: 0.0
```


Copyright © 2025 HyperionDev. All rights reserved.

Important Considerations

- **Invalid String Input**: If the string passed to float() cannot be converted to a valid float, Python raises a ValueError.
    - **Example**:

```
value = "abc"
# print(float(value))  # Uncommenting this will raise: ValueError: could not convert string to float: 'abc'
```

- **Precision**: The result from the float() function may lose precision due to how floats are represented in Python (based on the IEEE 754 floating-point standard).

The **float()** method is used to convert various data types (like integers and strings) into floating-point numbers.

It handles numeric strings, integers, special values like infinity and NaN, and even booleans.

The method is essential for ensuring that values can be used in calculations where **decimal precision** is needed.

This function is especially useful when you need to perform arithmetic operations or work with values that require decimal representation.

3.6 Floating-point numbers (KT0306) (IAC0301)

Floating-point numbers are a data type used to represent **real numbers** that include **fractional parts** or **decimals**. They are essential when you need to work with numbers that are not whole, such as measurements, fractions, or numbers resulting from division.

Copyright © 2025 HyperionDev. All rights reserved.

In Python, floating-point numbers are represented by the **float** data type. These numbers are stored in memory using a format called **IEEE 754**, which allows Python to handle a wide range of values, including very large or very small numbers, but with certain limitations in precision.

Key Characteristics of Floating-Point Numbers

1. **Decimal Representation**:
    - Floating-point numbers include a decimal point, which allows them to represent fractions or parts of a whole number.
    - **Examples**:

```
a = 3.14
b = 0.5
c = -2.75
print(a, b, c)  # Output: 3.14 0.5 -2.75
```

2. **Scientific Notation**:
    - Floating-point numbers can also be represented using **scientific notation**, which is useful for expressing very large or very small numbers. In Python, this is done using e or E to represent powers of 10.
    - **Example**:

```
mass = 9.1e-31  # 9.1 × 10^-31
speed = 3e8     # 3 × 10^8
print(mass)     # Output: 9.1e-31
print(speed)    # Output: 300000000.0
```

3. **Limited Precision**:
    - Floating-point numbers have **limited precision** because they are stored as binary fractions. This can lead to **rounding errors** or imprecise results in calculations, especially when dealing with very small or very large numbers.
    - **Example**:

```
x = 0.1 + 0.2
print(x)        # Output: 0.30000000000000004
```


Copyright © 2025 HyperionDev. All rights reserved.

```
print(x == 0.3) # Output: False
```

4. **Range of Values**:
    - Floats can represent numbers as large as 1.8 × 10^308 and as small as 5.0 × 10^−324, making them suitable for scientific calculations that deal with extreme values.
    - **Example**

```
import sys
print(sys.float_info.max)  # Largest float: ~1.7976931348623157e+308
print(sys.float_info.min)  # Smallest positive float:
~2.2250738585072014e-308
```

5. **Automatic Type Conversion**:
    - When performing operations between integers and floating-point numbers, Python automatically promotes the result to a float to maintain precision.

    - **Example**:

```
a = 7        # int
b = 2.5      # float
result = a + b
print(result)        # Output: 9.5
print(type(result))  # Output: <class 'float'>
```

Use Cases for Floating-Point Numbers

- **Mathematical Calculations**: Floats are used in calculations that require precision with decimal numbers, such as financial applications, scientific computations, and measurements.

Copyright © 2025 HyperionDev. All rights reserved.

- **Divisions**: When performing division, even between two integers, the result is always a float in Python.
    - **Example**:

```
print(5 / 2)        # Output: 2.5
print(type(5 / 2))  # Output: <class 'float'>
```

Limitations of Floating-Point Numbers

- **Rounding Errors**: Due to the way floats are stored (in binary), some decimal values cannot be represented exactly, leading to rounding errors in certain calculations.
- **Precision**: While floating-point numbers offer a wide range, they are not ideal for calculations that require **exact precision**, such as in financial calculations. For such cases, the **Decimal** type (from Python's decimal module) is more appropriate.

Floating-point numbers are used to represent real numbers with decimal points or fractions in Python.
They are useful for calculations involving non-integer values, scientific notation, and large or small numbers.
Due to how they are stored, floating-point numbers have **limited precision** and can produce **rounding errors** in some cases.
In general, floating-point numbers are great for most real-world applications but should be used carefully when **exact precision** is critical.



Formative Assessment Activity [3]

Float in Python

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT04:

Topic Code	KM-02-KT04:
Topic	Double data types in Python
Weight	5%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0401)
- Syntax and characteristics (KT0402)
- Use of double data types in Python (KT0403)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0401 Definitions, functions and features of Python double data types are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

4.1 Concept, definition and functions (KT0401) (IAC0401)

In Python, there is no specific **double** data type like in some other programming languages (e.g., C, Java). Instead, Python uses the **float** data type to represent **floating-point numbers**, which internally provides **double precision** (64-bit). This means Python's float behaves similarly to a double in other languages, offering high precision for decimal numbers.

Concept of Python float (Double Precision)

- Python's **float** data type is used to represent **real numbers** with **decimal points**.
- The underlying implementation of float follows the **IEEE 754 double-precision floating-point** standard, providing 64-bit precision.
- This allows Python to represent numbers with significant digits and a wide range, from very small to very large values.

Functions of Python float (Double)

1. **Representation of Decimal Numbers**:
    - Python float can handle numbers with decimal points, such as 3.14, -0.001, or numbers in scientific notation like 1.5e3 (1500.0).
    - **Example**:

```
a = 3.14
b = -0.001
c = 1.5e3
print(a, b, c)  # Output: 3.14 -0.001 1500.0
```

2. **Arithmetic Operations**:
    - You can use float values in arithmetic operations, such as addition, subtraction, multiplication, and division.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
x = 2.5
y = 1.2
print(x + y)  # Output: 3.7
print(x * y)  # Output: 3.0
```

3. **Type Conversion**:
    - Use the float() function to convert integers or strings into floating-point numbers.
    - **Example**:

```
print(float(5))       # Output: 5.0
print(float("4.5"))   # Output: 4.5
```

Features of Python float (Double)

1. **Double Precision (64-bit)**:
    - Python's float uses **64-bit double precision**, meaning it provides up to **15-17 significant decimal digits of precision**.
    - **Wide Range**: It can represent numbers as large as approximately 1.8 × 10^308 and as small as 5.0 × 10^−324.
2. **Handling Scientific Notation**:
    - float can represent very large or very small numbers using **scientific notation** (e.g., 1.5e6 represents 1500000.0).
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
large = 6.02e23
small = 4.2e-10
print(large)  # Output: 6.02e+23
print(small)  # Output: 4.2e-10
```

3. **Automatic Type Promotion**:
    - When performing operations between integers and floats, Python automatically promotes integers to floats to maintain precision.

    - **Example**:

```
result = 5 + 2.5
print(result)        # Output: 7.5
print(type(result))  # Output: <class 'float'>
```

4. **Precision Limitations**:
    - Due to the way floating-point numbers are stored (in binary), not all decimal numbers can be represented exactly, which can lead to **rounding errors**.
    - **Example**:

```
print(0.1 + 0.2)  # Output: 0.30000000000000004
```

5. **Special Values**:
    - float can represent **infinity** (float('inf')), **negative infinity** (float('-inf')), and **NaN (Not a Number)** (float('nan')).

```
print(float('inf'))   # Output: inf
```


Copyright © 2025 HyperionDev. All rights reserved.

```
print(float('-inf'))  # Output: -inf
print(float('nan'))   # Output: nan
```

Python does not have a specific **double** data type, but the **float** type provides **double-precision (64-bit)** floating-point arithmetic.

float can represent decimal numbers, perform arithmetic operations, and handle scientific notation.

It provides up to **15-17 significant digits** of precision but has limitations due to rounding errors and precision constraints.

Float is versatile, supporting special values like infinity and NaN.

4.2 Syntax and characteristics (KT0402) (IAC0401)

There is no specific **double** data type. Instead, Python uses the **float** type, which internally provides **double precision (64-bit)** floating-point representation. This makes Python's float equivalent to the **double** type in other programming languages like C or Java.

Syntax of Python float (Double Precision)

1. **Basic Syntax**:
    - To declare a floating-point number (equivalent to double), simply assign a value with a decimal point or in scientific notation.
    - **Example**:

```
x = 3.14159
y = 2.5e3
print(x, y)  # Output: 3.14159 2500.0
```

2. **Type Conversion**:

Copyright © 2025 HyperionDev. All rights reserved.

- You can convert other data types (like integers or strings) to floats using the float() function.
- **Example**:

```
print(float(100))       # Output: 100.0
print(float("3.14"))    # Output: 3.14
```

Characteristics of Python float (Double Precision)

1. **Double Precision (64-bit)**:
   - Python's float uses **64-bit double precision**, meaning it can represent numbers with up to **15-17 significant decimal digits**. This ensures higher accuracy when working with real numbers.
   - **Range**: Floats can represent very large numbers (up to approximately 1.8 × 10^308) and very small numbers (as low as 5.0 × 10^−324).
2. **Scientific Notation**:
   - Floats can be written using **scientific notation**, which is helpful for representing very large or very small numbers.
   - **Example**:

```
speed = 3e8
mass = 1.67e-27
print(speed, mass)  # Output: 300000000.0 1.67e-27
```

3. **Precision Limitations**:
   - While floats provide high precision, certain decimal numbers cannot be represented exactly in binary (as floats are stored in binary format), which can lead to **rounding errors**.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
print(0.1 + 0.2)  # Output: 0.30000000000000004
```

4. **Automatic Type Promotion**:
    - When performing arithmetic between an integer and a float, Python automatically **promotes integers to floats** to ensure precision in calculations.
    - **Example**:

```
print(10 + 2.5)        # Output: 12.5
print(type(10 + 2.5))  # Output: <class 'float'>
```

5. **Special Values**:
    - Python float can represent **infinity** (float('inf')), **negative infinity** (float('-inf')), and **NaN** (Not a Number, float('nan')), which are useful for handling exceptional cases in numerical computations.
    - **Example**:

```
print(float('inf'), float('-inf'), float('nan'))
# Output: inf -inf nan
```

6. **Arithmetic Operations**:
    - Python float supports all standard arithmetic operations (addition, subtraction, multiplication, division) and can handle complex mathematical calculations.
    - **Example**:

```
a = 5.5
b = 2.0
print(a + b, a * b, a / b)  # Output: 7.5 11.0 2.75
```


Copyright © 2025 HyperionDev. All rights reserved.

- Python's **float** type serves as the equivalent of **double** data types in other languages, providing **64-bit double precision** for decimal numbers.
- Floats can represent very large and small numbers using **scientific notation** and support up to **15-17 significant digits** of precision.
- Although floats provide high precision, they are subject to **rounding errors** due to their binary representation, especially for certain decimal values.
- Floats can also handle special values like **infinity** and **NaN**, and Python automatically promotes integers to floats in mixed arithmetic operations.

4.3 Use of double data types in Python (KT0403) (IAC0401)

There is no specific **double** data type. Instead, Python uses the **float** type, which internally provides **double-precision (64-bit)** floating-point representation. This makes Python's float equivalent to the **double** data type found in languages like C or Java.

Uses of Double-Precision (float) in Python

1. **Representing Decimal Numbers**:
    - Python's float is used to represent **real numbers** that include a decimal point, such as measurements, scientific values, or numbers resulting from division.

    - **Example**:

```
length = 2.75
area = 10.5
print(length, area)  # Output: 2.75 10.5
```

2. **Handling Large or Small Numbers (Scientific Notation)**:

Copyright © 2025 HyperionDev. All rights reserved.

- Python float supports **scientific notation** for representing very large or very small numbers. This is particularly useful in scientific and engineering calculations.
- **Example**:

```
avogadro = 6.022e23
planck = 6.626e-34
print(avogadro, planck)  # Output: 6.022e+23 6.626e-34
```

3. **Performing Arithmetic Operations**:
   - Python's float type is used for **arithmetic operations** involving decimal numbers. The float type supports all basic operations like addition, subtraction, multiplication, and division.
   - **Example**:

```
x = 7.5
y = 2.0
print(x + y, x * y, x / y)  # Output: 9.5 15.0 3.75
```

4. **Precise Calculations in Science and Engineering**:
   - Double-precision (float) values are commonly used in fields like physics, engineering, and finance where precise decimal calculations are required.
   - **Example**:

```
gravity = 9.81
mass = 75.0
force = gravity * mass
print(force)  # Output: 735.75
```

5. **Converting Values to Float**:
   - You can use the float() function to convert integers, strings, or other numeric data types to float for decimal precision.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
print(float(9))         # Output: 9.0
print(float("3.5"))     # Output: 3.5
```

6. **Handling Special Cases**:
   - Python's float type also supports special values like **infinity** (float('inf')), **negative infinity** (float('-inf')), and **NaN (Not a Number)** (float('nan')), which are useful for error handling or representing undefined results in mathematical operations.
   - **Example**:

```
print(float('inf'))    # Output: inf
print(float('-inf'))   # Output: -inf
print(float('nan'))    # Output: nan
```

7. **Automatic Type Conversion**:
   - Python automatically promotes integers to float in mixed operations involving both integers and floats, ensuring that the precision of the result is maintained.
   - **Example**:

```
result = 8 + 1.25
print(result)        # Output: 9.25
print(type(result))  # Output: <class 'float'>
```

- In Python, **float** is used as the equivalent of **double-precision (64-bit)** floating-point numbers, representing **real numbers with decimals**.
- Floats are useful for performing arithmetic operations, handling large and small numbers via **scientific notation**, and ensuring precision in calculations.
- Python float supports special cases like **infinity** and **NaN**, and the float() function allows for easy conversion of other data types into floating-point numbers.

Copyright © 2025 HyperionDev. All rights reserved.

- Python automatically handles **type promotion**, converting integers to floats in mixed operations.

The use of Python's float (which functions like a **double** in other languages) is essential for tasks that require **decimal precision**, making it widely applicable in fields like finance, science, and engineering.



Formative Assessment Activity [4]

Double data types in Python

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT05:

Topic Code	KM-02-KT05:
Topic	Boolean in Python
Weight	10%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0501)
- Syntax and characteristics (KT0502)
- Use of strings in Python (KT0503)
- Two values: denoted true and false (KT0504)
- Problem solving and optimising code (KT0505)
- Uppercase (KT0506)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0501 Definitions, functions and features of Python Boolean are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

5.1 Concept, definition and functions (KT0501) (IAC0501)

A **Boolean** in Python is a data type that can hold one of two possible values: **True** or **False**. It is based on the Boolean logic system, which is fundamental to computer science and programming. In Python, Booleans are used to represent the outcome of logical operations, conditions, and comparisons.

Functions of Python Boolean

1. **Representing Truth Values**:
    - Boolean values (True or False) are used to represent the truthfulness or falsity of an expression or condition.
    - **Example**:

```
is_raining = True
print(is_raining)  # Output: True
```

2. **Conditional Statements**:
    - Booleans are primarily used in control flow statements like if, elif, and else to decide which block of code to execute based on a condition.
    - **Example**:

```
if True:
    print("This block will execute.")
```

3. **Logical Operations**:
    - Booleans are involved in **logical operations** using and, or, and not. These operations return Boolean values based on the combination of conditions.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
print(True and False)  # Output: False
print(True or False)   # Output: True
print(not True)        # Output: False
```

4. **Comparisons**:
    - Comparison operators like ==, !=, >, <, >=, and <= return Boolean values based on the relationship between two values.
    - **Example**:

```
print(5 > 3)  # Output: True
print(5 == 2) # Output: False
```

5. **Type Conversion to Boolean**:
    - Python has a built-in bool() function to convert values to Booleans. Values like 0, None, [] (empty lists), and "" (empty strings) are considered False, while all other values are True.
    - **Example**:

```
print(bool(0))       # Output: False
print(bool("Hello")) # Output: True
```

Features of Python Boolean

1. **Binary Nature (Two Values)**:
    - Booleans only have two possible values: **True** and **False**.

Copyright © 2025 HyperionDev. All rights reserved.

- In Python, **True** is treated as 1 and **False** is treated as 0 when used in arithmetic operations.
- **Example**:

```
print(True + True)   # Output: 2
print(False * 10)    # Output: 0
```

2. **Used in Logical and Comparison Operations**:
   - Booleans are essential in evaluating conditions and determining the flow of a program using logical (and, or, not) and comparison operations (==, >, <, etc.).
   - **Example**:

```
x = 5
y = 10
print(x < y and y > 5)  # Output: True
```

3. **Implicit Type Conversion**:
   - In conditional statements, Python automatically converts values to their Boolean equivalent. Any non-zero number, non-empty string, list, or object evaluates to True, while 0, None, [], and "" evaluate to False.
   - **Example**:

```
if 0:  # This condition evaluates to False, so the block will not execute
    print("This won't print")
```

- **Booleans** in Python are a data type representing **True** or **False**, often used in conditions, comparisons, and logical operations.

Copyright © 2025 HyperionDev. All rights reserved.

- **Functions**: They are used to control flow (if statements), perform logical operations (and, or, not), and evaluate comparisons.
- **Features**: Booleans have a binary nature (two values), can be converted from other types using bool(), and integrate smoothly with Python's control flow and comparison systems.

Python Booleans are essential for decision-making and logical evaluations in code, making them a foundational part of the language's functionality.

5.2 Syntax and characteristics (KT0502) (IAC0501)

Booleans are a fundamental data type used to represent **truth values**, which can either be **True** or **False**. Booleans are essential in decision-making, conditional statements, and logical operations in Python programming.

Syntax of Python Boolean

1. **Boolean Values**:
    - In Python, the Boolean values are represented by the keywords **True** and **False** (with uppercase 'T' and 'F').
    - **Example**:

```
is_active = True
is_logged_in = False
```

2. **Logical Operators**:
    - Booleans are often used with logical operators such as **and**, **or**, and **not** to evaluate conditions.
    - **Examples**:

Copyright © 2025 HyperionDev. All rights reserved.

```
x = True
y = False
result = x and y  # False
result = x or y  # True
result = not x  # False
```

3. **Comparison Operators**:
    - Python Booleans are frequently the result of comparison operators like **==**, **!=**, **>, <, >=, <=**, which compare values and return True or False.
    - **Examples**:

```
print(10 > 5)   # Output: True
print(5 != 5)   # Output: False
```

4. **Type Conversion (bool() Function)**:
    - You can use the bool() function to convert other data types to their Boolean equivalent. Non-zero numbers, non-empty strings, and non-empty data structures are evaluated as True, while 0, None, empty strings, and empty data structures evaluate as False.
    - **Example**:

```
print(bool(1))       # Output: True
print(bool(0))       # Output: False
print(bool(""))      # Output: False
print(bool("text"))  # Output: True
```

Characteristics of Python Boolean

1. **Two Possible Values**:
    - A Boolean can only hold one of two values: **True** or **False**.

Copyright © 2025 HyperionDev. All rights reserved.

- These values are treated as special instances of the integers 1 and 0, respectively.
- **Example**:

```
print(True == 1)    # Output: True
print(False == 0)   # Output: True
```

2. **Used in Conditional Statements**:
   - Booleans are essential in **if, elif, and else** statements, allowing decisions to be made based on conditions.
   - **Example**:

```
is_raining = False
if is_raining:
    print("Take an umbrella.")
else:
    print("No umbrella needed.")
```

3. **Boolean Context**:
   - In Python, many objects and expressions can be evaluated in a **Boolean context**, such as in if statements or loops. Values like None, 0, empty strings (""), and empty data structures (e.g., [], {}) are treated as False, while non-zero numbers, non-empty strings, and data structures are treated as True.
   - **Example**:

```
if "hello":
    print("String is non-empty.")  # Output: String is non-empty.
```

4. **Implicit Type Conversion**:

Copyright © 2025 HyperionDev. All rights reserved.

- Python will automatically convert values to Booleans when needed in conditional statements or logical operations. This is known as **implicit type conversion**, where non-Boolean values are treated as True or False based on their content.
- **Example**:

```
print(bool([]))      # Output: False
print(bool([1, 2]))  # Output: True
```

- **Syntax**: Python Boolean values are represented by **True** and **False**, and they can be combined with logical operators (and, or, not) and comparison operators (==, >, <, etc.).
- **Characteristics**:
  - Booleans have two possible values: **True** and **False**.
  - They are frequently used in **conditional statements** and **logical expressions**.
  - Python automatically converts other data types to their Boolean equivalent in contexts like if statements, where non-empty or non-zero values evaluate to True and empty or zero values evaluate to False.

Booleans are crucial for decision-making and control flow in Python programs, helping determine which actions to take based on certain conditions.

5.3 Use of strings in Python (KT0503) (IAC0501)

Strings can be evaluated as **Boolean values** in various contexts, such as conditional statements and logical operations. Python automatically converts (or **coerces**) strings to Booleans when needed, following specific rules to determine if the string evaluates to **True** or **False**.

Key Rules for Strings in Boolean Context

1. **Non-Empty Strings Evaluate to True**:

Copyright © 2025 HyperionDev. All rights reserved.

- Any string that contains one or more characters, even a single space, is considered **True** when evaluated in a Boolean context.
- **Example**:

```
if "hello":
    print("True")  # Output: True
```

2. **Empty Strings Evaluate to False**:
   - An **empty string** (i.e., "") is considered **False** when evaluated in a Boolean context.
   - **Example**:

```
if "":
    print("This will not print.")
```

Common Use Cases for Strings in Python Boolean

1. **Conditional Statements**:
   - Strings are often evaluated in conditional statements like if, elif, and else. If the string is non-empty, the condition is considered **True**, and if it is empty, the condition is **False**.
   - **Example**:

```
user_input = "yes"
if user_input:
    print("Input received.")
```

2. **Logical Operations**:
   - Strings can be used with logical operators such as and, or, and not to determine the outcome of Boolean expressions. Non-empty strings evaluate to True, and empty strings evaluate to False.
   - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
print("hello" and "world")  # Output: world
print("" or "default")      # Output: default
```

3. **Checking Input**:
    - When checking for user input or values from data, you can directly use strings in Boolean contexts to check if input was provided or if it is empty.
    - **Example**:

```
input_text = input("Enter something: ")
if input_text:
    print("You entered:", input_text)
```

- In Python, **non-empty strings** are evaluated as **True**, and **empty strings** are evaluated as **False** in Boolean contexts.
- This behaviour is useful in **conditional statements** and **logical operations** where you want to check if a string has content.

Using strings in Boolean contexts simplifies checks for empty or non-empty values in Python code.

5.4 Two values: denoted true and false (KT0504) (IAC0501)

The **Boolean** data type consists of two values: **True** and **False**. These two values are used to represent the **truth values** in logical operations, comparisons, and control flow structures like if, elif, and else statements.

1. True in Python Boolean

- **Definition**: In Python, **True** represents a Boolean value that signifies something is logically correct, valid, or has a positive outcome.
- **Numeric Representation**: Internally, Python treats **True** as the integer **1**.
- **Use Cases**:

Copyright © 2025 HyperionDev. All rights reserved.

- **Conditional Statements**: True can be used in conditions where a certain action should be executed.
- **Logical Operations**: True is returned when a condition or comparison evaluates positively.

```
if True:
    print("This will execute.")

print(True == 1)  # Output: True
```

Example:

```
is_sunny = True

if is_sunny:
    print("It's a sunny day!")  # This will print because the condition is
True.
```

2. False in Python Boolean

- **Definition**: **False** represents a Boolean value that signifies something is logically incorrect, invalid, or has a negative outcome.
- **Numeric Representation**: Python treats **False** as the integer **0**.
- **Use Cases**:
  - **Conditional Statements**: False is used when a condition should prevent the execution of a certain block of code.
  - **Logical Operations**: False is returned when a condition or comparison evaluates negatively.

```
if False:
    print("This will NOT execute.")
```


Copyright © 2025 HyperionDev. All rights reserved.

```
print(False == 0)  # Output: True
```

Example:

```
is_raining = False

if is_raining:
    print("Take an umbrella.")
else:
    print("No umbrella needed.")  # This will print because the condition is False.
```

- **True** and **False** are the two values of Python's Boolean data type.
    - **True** represents a logical or correct condition and is treated as **1**.
    - **False** represents an incorrect condition and is treated as **0**.
- These Boolean values are essential in controlling the flow of a program using conditional statements and are widely used in comparisons and logical operations.

5.5 Problem solving and optimising code (KT0505) (IAC0501)

Problem-solving with Python Boolean involves using **Boolean logic** to design solutions that evaluate conditions and make decisions in your code. This typically involves leveraging **True** and **False** values to control the flow of execution using conditional statements and logical operators.

Optimizing code with Booleans means writing code that is both **efficient** and **readable** by simplifying Boolean expressions and minimizing unnecessary computations.

Copyright © 2025 HyperionDev. All rights reserved.

Problem Solving Using Python Boolean

1. **Conditional Logic**:
    - Booleans are essential in creating conditional statements like if, elif, and else, which allow your program to take different actions based on whether certain conditions are True or False.
    - **Example**:

```
age = 20
if age >= 18:
    print("You are an adult.")
```

    - Here, the condition age >= 18 evaluates to a Boolean value (True or False) that determines which block of code runs.
2. **Using Logical Operators**:
    - Python Boolean logic uses operators such as **and**, **or**, and **not** to combine multiple conditions, allowing for more complex decision-making.
    - **Example**:

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("Access granted.")
```

3. **Problem Solving with Short-Circuiting**:
    - Logical operators in Python perform **short-circuiting**, meaning that they stop evaluating as soon as the result is determined. This behavior can be leveraged to improve efficiency in problem solving by avoiding unnecessary calculations.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
def check():
    print("Function called")
    return True

if True or check():  # check() won't be called due to short-circuit
    print("Short-circuited")
```

Optimizing Code with Python Boolean

1. **Simplifying Boolean Expressions**:
    - Avoid redundant conditions by simplifying Boolean expressions. For example, instead of checking if condition == True, you can simply write if condition.
    - **Example**:

```
# Instead of:
if is_valid == True:
    print("Valid")

# Use:
if is_valid:
    print("Valid")
```

2. **Eliminating Unnecessary Comparisons**:
    - You don't need to compare Booleans to True or False directly. Just use the Boolean variable itself or use the not operator for negation.
    - **Example**:

```
# Instead of:
if condition == False:
```


Copyright © 2025 HyperionDev. All rights reserved.

```
    print("False")

# Use:
if not condition:
    print("False")
```

3. **Using Short-Circuiting for Efficiency**:
    - **Short-circuit evaluation** in Python can optimize code by reducing the number of operations. For example, in an or condition, Python stops evaluating as soon as one condition is True. Similarly, for and, it stops when one condition is False.
    - **Example**:

```
x = None
if x is not None and x > 5:
    print("x is greater than 5")
```

4. **Minimizing Nested Conditions**:
    - Too many nested if statements can make code harder to read and maintain. Consider combining conditions or using **early exits** to simplify logic.
    - **Example**:

```
# Instead of:
if x > 0:
    if x < 10:
        print("x is between 1 and 9")

# Use:
if 0 < x < 10:
    print("x is between 1 and 9")
```


Copyright © 2025 HyperionDev. All rights reserved.

- **Problem Solving**: Use Boolean logic (True, False) to control program flow, evaluate conditions, and make decisions using logical operators and conditional statements.
- **Optimizing Code**: Simplify Boolean expressions, leverage short-circuiting to avoid unnecessary evaluations, eliminate redundant comparisons, and minimize nested conditions for better readability and efficiency.

Effective use of Python Booleans in problem-solving ensures that your code is **clear, efficient, and maintainable**.

5.6 Uppercase (KT0506) (IAC0501)

Uppercase does not have a specific function in Boolean logic itself. However, uppercase strings can be evaluated in **Boolean contexts**. Here's how Booleans interact with strings that are in uppercase:

1. Strings and Booleans

- In Python, **non-empty strings** are treated as **True** in Boolean contexts, while **empty strings** are treated as **False**, regardless of whether the string is in uppercase or lowercase.

Example:

```
message = "HELLO"
if message:
    print("String is non-empty.")  # Output: String is non-empty.
```

In this case, the string "HELLO" is in uppercase, but it still evaluates to True simply because it is non-empty.

2. Case Sensitivity of Strings

- Python strings are **case-sensitive**, meaning that "TRUE" (in uppercase) is **not the same** as the Boolean value **True**. Therefore, a string like "TRUE" will evaluate as True only because it is non-empty, but it is still a string and not the Boolean True.

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
val = "TRUE"

print(val == True)   # Output: False
print(bool(val))     # Output: True (non-empty string)
```

- **Non-empty strings**, whether uppercase or lowercase, evaluate as **True** in Boolean contexts.
- **Empty strings** (including empty uppercase strings like "") evaluate as **False**.
- Strings like "TRUE" (uppercase) are **not** the same as the Boolean True; they are treated as non-empty strings, which evaluate as True in Boolean logic but are still strings.

In Python Booleans, the concept of **uppercase** is relevant when dealing with string comparisons, but it does not directly influence the Boolean values themselves.



Formative Assessment Activity [5]

Boolean in Python

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT06:

Topic Code	KM-02-KT06:
Topic	Python List
Weight	50%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0601)
- Syntax (KT0602)
- Use of Range Function in Python (KT0603)
- List in Python (KT0604)
- Mixed List in Python (KT0605)
- List Length Method (KT0606)
- List Remove and Del Method (KT0607)
- Python Append Method (KT0608)
- Python Sort Method (KT0609)
- Python List count() method (KT0610)
- Python List index() method (KT0611)
- List Comprehension and Reverse (KT0612)
- Nested Lists (KT0613)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0601 Definitions, functions and features of Python list are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

6.1 Concept, definition and functions (KT0601) (IAC0601)

A **list** in Python is a **mutable**, **ordered collection** of elements that can hold **multiple data types** (e.g., integers, strings, floats, other lists).

Lists are defined using **square brackets [ ]**, with elements separated by commas.

- **Example**:

```
my_list = [1, "apple", 3.14, True]
```

2. Functions of Python List

1. **Storing Multiple Items**:
    - A list can store a sequence of elements, allowing you to group related data together.
    - **Example**:

```
fruits = ["apple", "banana", "cherry"]
```

2. **Accessing Elements by Index**:
    - Lists are **indexed** starting from 0. You can access individual elements or slices of the list by referencing their index.
    - **Example**:

```
fruits = ["apple", "banana", "cherry"]
print(fruits[1])  # Outputs: "banana"
```

3. **Modifying List Elements**:
    - Lists are **mutable**, meaning you can change elements after the list is created.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
fruits[1] = "orange"  # Now fruits is ["apple", "orange", "cherry"]
```

4. **Adding and Removing Elements**:
    - Lists allow adding or removing elements dynamically using methods like append(), insert(), and remove().
    - **Example**:

```
fruits.append("grape")  # Adds "grape" to the end of the list
fruits.remove("apple")  # Removes "apple" from the list
```

5. **Iterating Over a List**:
    - You can loop through a list's elements using a for loop.
    - **Example**:

```
for fruit in fruits:
    print(fruit)
```

3. Features of Python List

1. **Heterogeneous Data**:
    - Lists can store elements of **different types** (e.g., integers, strings, floats, Booleans) in the same list.
    - **Example**:

```
mixed_list = [1, "hello", 3.14, False]
```

2. **Dynamic Size**:
    - Python lists are **dynamic**, meaning they can grow or shrink in size as elements are added or removed.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
numbers = [1, 2, 3]
numbers.append(4)  # List now has 4 elements
```

3. **Negative Indexing and Slicing**:
    - Lists support **negative indexing**, where -1 refers to the last element, -2 refers to the second last, and so on.
    - **Slicing** allows you to access a subset of the list.
    - **Example**:

```
my_list = [10, 20, 30, 40, 50]
print(my_list[-1])  # Outputs: 50 (last element)
print(my_list[1:4])  # Outputs: [20, 30, 40] (slice of the list)
```

4. **List Methods**:
    - Lists come with built-in methods for various operations, such as:
        - **append()**: Add an item to the end.
        - **remove()**: Remove the first occurrence of an item.
        - **sort()**: Sort the list.
        - **reverse()**: Reverse the order of the list.
        - **len()**: Get the number of elements in the list.
    - **Example**:

```
my_list = [3, 1, 2]
my_list.sort()  # [1, 2, 3]
```


Copyright © 2025 HyperionDev. All rights reserved.

- **Concept**: A Python list is a **mutable, ordered collection** that can hold elements of different data types.
- **Functions**: Lists are used for **storing**, **accessing**, **modifying**, and **iterating** over data.
- **Features**: Lists support **heterogeneous data**, **dynamic sizing**, **indexing**, **slicing**, and various built-in methods like append(), remove(), and sort().

Lists are versatile and widely used in Python for handling collections of items in a flexible and efficient manner.

6.2 Syntax (KT0602) (IAC0601)

A **list** in Python is a **data structure** used to store multiple items in a single variable. The syntax for creating and working with lists is simple and intuitive. Here's a breakdown of how Python lists work syntactically:

1. Defining a List

- Lists are created using **square brackets [ ]**, with the elements separated by commas.
- **Syntax**:

```
my_list = [element1, element2, element3]
```

- **Example**:

```
fruits = ["apple", "banana", "cherry"]
```

2. Accessing Elements

- List elements are **indexed** starting from 0. You can access an element by referring to its index in square brackets.
- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_list[index]
```

- **Example**:

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])  # Outputs: apple
print(fruits[2])  # Outputs: cherry
```

3. Modifying Elements

- Lists are **mutable**, meaning you can change their elements. To modify an element, access it using its index and assign a new value.
- **Syntax**:

```
my_list[index] = new_value
```

- **Example**:

```
fruits = ["apple", "banana", "cherry"]
fruits[1] = "orange"  # Changes "banana" to "orange"
```

4. Adding Elements

- You can add elements to a list using methods like **append()** or **insert()**.
- **Syntax**:

```
my_list.append(new_value)  # Adds to the end
my_list.insert(index, new_value)  # Adds at specific index
```

- **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
fruits = ["apple", "banana"]
fruits.append("cherry")  # Adds "cherry" at the end
fruits.insert(1, "orange")  # Inserts "orange" at index 1
```

5. Removing Elements

- You can remove elements using methods like **remove()**, **pop()**, or **del**.
- **Syntax**:

```
my_list.remove(value)  # Removes the first occurrence of the value
my_list.pop(index)  # Removes the element at the specified index
del my_list[index]  # Deletes the element at the specified index
```

- **Example**:

```
fruits = ["apple", "banana", "cherry"]
fruits.remove("banana")  # Removes "banana"
fruits.pop(0)  # Removes the first element ("apple")
del fruits[1]  # Deletes the element at index 1
```

6. Slicing a List

- You can **slice** a list to get a subset of its elements.
- **Syntax**:

```
my_list[start:stop]  # Returns a new list from index 'start' to 'stop' (excluding 'stop'
my_list[start:stop:step]  # Includes a step to specify how many elements to skip
```

- **Example**:

```
numbers = [1, 2, 3, 4, 5, 6]
print(numbers[1:4])  # Outputs: [2, 3, 4]
print(numbers[::2])  # Outputs: [1, 3, 5] (step of 2)
```


Copyright © 2025 HyperionDev. All rights reserved.

7. List Length

- You can find the number of elements in a list using the **len()** function.
- **Syntax**:

```
len(my_list)
```

- **Example**:

```
fruits = ["apple", "banana", "cherry"]
print(len(fruits))  # Outputs: 3
```

- **Lists** are created using square brackets ([ ]) with elements separated by commas.
- Lists allow **indexing**, **modification**, **adding**, and **removing** of elements.
- They support **slicing** to retrieve subsets of elements and have a variety of methods like append(), insert(), remove(), and pop() for manipulating the data.

Python's list syntax is highly flexible and allows for powerful operations on collections of data.

6.3 Use of Range Function in Python (KT0603) (IAC0601)

The **range()** function in Python generates a sequence of numbers and is often used in combination with lists, particularly when you want to **iterate** over a list or create a list of numbers in a specific range.

1. Using range() to Iterate Over a List

The range() function is commonly used in loops, especially **for loops**, to iterate over list indices.

- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
range(start, stop, step)
```

- **start**: The starting number (inclusive). Default is 0.
- **stop**: The number where the sequence ends (exclusive).
- **step**: The difference between each number in the sequence. Default is 1.

- **Example: Iterating over a list**:

```
fruits = ["apple", "banana", "cherry"]
for i in range(len(fruits)):
    print(f"Index {i}: {fruits[i]}")
```

Output:

```
Index 0: apple
Index 1: banana
Index 2: cherry
```

In this example, range(len(fruits)) generates a sequence of indices [0, 1, 2], which is used to access and print each element of the list by its index.

2. Using range() to Create a List of Numbers

You can use the range() function along with **list()** to generate a list of numbers.

- **Example: Creating a list of numbers**:

```
numbers = list(range(5))  # [0, 1, 2, 3, 4]
print(numbers)
```

Output:

```
[0, 1, 2, 3, 4]
```

- **Example: Using range() with a step**:

Copyright © 2025 HyperionDev. All rights reserved.

```
even_numbers = list(range(0, 10, 2))  # [0, 2, 4, 6, 8]
print(even_numbers)
```

In this example, the range(0, 10, 2) generates even numbers from 0 to 9.

3. Benefits of Using range() with Lists

- **Memory Efficient**: range() does not create a list in memory but instead generates the numbers on demand. This makes it efficient, especially when dealing with large ranges.
- **Control over Indices**: Using range() gives you precise control over the **start**, **stop**, and **step** values when iterating over lists or creating numeric sequences.

Range() is used to generate sequences of numbers and is often combined with lists for **iteration** and **list creation**.

It allows for efficient looping over list indices and creating lists of numbers with defined **start**, **stop**, and **step** values.

Example uses include iterating through a list using its indices and creating lists of numbers directly from the range() function.

6.4 List in Python (KT0604) (IAC0601)

A **list** in Python is a **mutable, ordered collection** of elements, which means that the elements are stored in a specific sequence and can be changed after the list is created. Lists are one of the most versatile data structures in Python and can store **multiple data types**, such as integers, strings, floats, or even other lists.

Key Characteristics of a Python List

1. **Ordered**:
    - Elements in a list have a defined order. You can access items by their **index** (starting from 0 for the first item).
2. **Mutable**:

Copyright © 2025 HyperionDev. All rights reserved.

- Lists can be modified after creation. You can add, remove, or change elements within the list.

3. **Heterogeneous**:
   - Lists can store elements of **different data types** within the same list (e.g., integers, strings, floats).
   - **Example**:

```
my_list = [1, "apple", 3.14]
```

4. **Dynamic**:
   - Lists are **dynamic** in size, meaning you can add or remove elements without needing to declare the size in advance.

Syntax of a Python List

- Lists are created using **square brackets [ ]** with elements separated by commas.
- **Example**:

```
fruits = ["apple", "banana", "cherry"]
```

Basic Operations on Lists

1. **Accessing Elements**:
   - You can access elements in a list using **indexing**. The first element has an index of 0.
   - **Example**:

```
print(fruits[1])  # Outputs: "banana"
```

2. **Modifying Elements**:
   - Lists allow you to change elements by assigning new values to specific indices.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
fruits[1] = "orange"  # Changes "banana" to "orange"
```

3. **Adding Elements**:
    - Use **append()** to add an element to the end of the list, or **insert()** to add an element at a specific position.
    - **Example**:

```
fruits.append("grape")  # Adds "grape" to the end
fruits.insert(1, "orange")  # Inserts "orange" at index 1
```

4. **Removing Elements**:
    - Use **remove()** to delete a specific element or **pop()** to remove an element by its index.
    - **Example**:

```
fruits.remove("apple")  # Removes "apple" from the list
fruits.pop(0)  # Removes the first element ("apple")
```

5. **Slicing**:
    - You can **slice** a list to get a subset of elements.
    - **Example**:

```
numbers = [1, 2, 3, 4, 5]
print(numbers[1:3])  # Outputs: [2, 3]
```

- A **list** in Python is a **mutable, ordered collection** of elements, which can hold multiple data types.
- Lists are created using square brackets [ ] and can be modified after creation.

Copyright © 2025 HyperionDev. All rights reserved.

- They support various operations like **accessing, modifying, adding, removing**, and **slicing** elements, making them highly flexible for managing collections of data.

Lists are one of the most used and powerful data structures in Python due to their versatility and ease of use.

6.5 Mixed List in Python (KT0605) (IAC0601)

A **mixed list** in Python is a list that contains elements of **different data types**. This could include integers, strings, floats, Booleans, and even other lists. Python's flexibility allows you to store heterogeneous data in a single list, making it a powerful tool for handling various types of information in one collection.

Functions of a Mixed List

1. **Storing Multiple Data Types**:
    - A mixed list can store elements of different types, such as numbers, strings, Booleans, and even other lists, in the same collection.
    - **Example**:

```
mixed_list = [42, "apple", 3.14, True]
```

2. **Accessing Elements by Index**:
    - You can access any element in a mixed list by its **index**. Lists are zero-indexed, so the first element has an index of 0.
    - **Example**:

```
print(mixed_list[1])  # Outputs: "apple"
```

3. **Modifying Elements**:

Copyright © 2025 HyperionDev. All rights reserved.

- Since lists are **mutable**, elements in a mixed list can be changed by accessing them through their index and assigning a new value.
- **Example**:

```
mixed_list[2] = 7.89  # Changes 3.14 to 7.89
```

4. **Adding and Removing Elements**:
   - You can use **append()** to add new elements to the list or **remove()** and **pop()** to remove elements. The data type of the new or removed element can differ from the rest of the elements.
   - **Example**:

```
mixed_list.append(False)  # Adds a Boolean value
mixed_list.pop(1)  # Removes the element at index 1 ("apple")
```

5. **Iterating Over a Mixed List**:
   - You can iterate over all elements in a mixed list using a for loop, regardless of their data types.
   - **Example**:

```
for item in mixed_list:
    print(item)
```

6. **Handling Nested Lists**:
   - Mixed lists can contain **other lists** as elements, allowing for multi-dimensional data structures.
   - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
mixed_list = [1, "apple", [10, 20, 30], False]
print(mixed_list[2][1])  # Outputs: 20 (accessing the nested list)
```

Features of a Mixed List

1. **Heterogeneous Data**:
    - A mixed list can hold **elements of different types** in the same list, such as integers, strings, floats, Booleans, or other lists.
    - **Example**:

```
my_list = [1, "hello", 3.14, [2, 3, 4], False]
```

2. **Mutable**:
    - Just like any other list, a mixed list is **mutable**, meaning you can change its contents after creation. You can add, modify, or remove elements.
3. **Ordered**:
    - Mixed lists maintain the **order** of elements. The order in which elements are added to the list is preserved when accessing or iterating through the list.
4. **Dynamic**:
    - Mixed lists are **dynamic**, meaning their size can grow or shrink as elements are added or removed.
5. **Supports Built-in List Methods**:
    - Mixed lists support all the standard list methods, such as:
        - **append()**: Add elements to the end of the list.
        - **remove()**: Remove a specific element.

Copyright © 2025 HyperionDev. All rights reserved.

- **pop()**: Remove an element by its index.
- **sort()**: Sorting mixed lists is only possible when all elements are of the same comparable type.

6. **Slicing and Indexing**:
    - Mixed lists support **slicing** and **negative indexing** to access elements or subsets of the list.
    - **Example**:

```
print(mixed_list[1:3])  # Outputs a slice of the list from index 1 to 2
```

- A **mixed list** in Python is a list that can hold elements of different data types, such as integers, strings, floats, Booleans, and even other lists.
- Mixed lists are **mutable, ordered, and dynamic**, supporting all the standard list operations like **accessing**, **modifying**, **adding**, and **removing** elements.
- They allow for the **flexible storage** of heterogeneous data in a single collection, making them a versatile data structure for various programming tasks.

6.6 List Length Method (KT0606) (IAC0601)

Python lists come with a variety of built-in **methods** that allow you to perform operations on lists, such as adding, removing, and modifying elements. These methods make lists highly versatile and powerful for managing collections of data.

1. Functions of Python List Methods

1. **Adding Elements**:
    - **append(item)**: Adds an element to the **end** of the list.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
my_list = [1, 2, 3]
my_list.append(4)  # [1, 2, 3, 4]
```

- **insert(index, item)**: Inserts an element at a specific **index** in the list.
    - **Example**:

```
my_list.insert(1, "apple")  # [1, "apple", 2, 3]
```

- **extend(iterable)**: Extends the list by adding all elements from another iterable (e.g., list, tuple).
    - **Example**:

```
my_list.extend([4, 5, 6])  # [1, 2, 3, 4, 5, 6]
```

2. **Removing Elements**:
    - **remove(item)**: Removes the **first occurrence** of a value from the list.
        - **Example**:

```
my_list.remove(2)  # Removes the first 2 from the list
```

- **pop(index)**: Removes and **returns** the element at the given index (default is the last element if no index is specified).
    - **Example**:

```
my_list.pop(0)  # Removes and returns the element at index 0
```


Copyright © 2025 HyperionDev. All rights reserved.

- **clear()**: Removes all elements from the list, making it empty.
    - **Example**:

```
my_list.clear()  # []
```

3. **Modifying Lists**:
    - **sort()**: Sorts the list **in place** (in ascending order by default). This method can take an optional parameter to sort in descending order or based on a custom function.
        - **Example**:

```
my_list.sort()  # Sorts the list in ascending order
```

    - **reverse()**: Reverses the **order** of the elements in the list.
        - **Example**:

```
my_list.reverse()  # Reverses the list
```

4. **Finding Information**:
    - **index(item)**: Returns the **index** of the first occurrence of an element in the list.
        - **Example**:

```
my_list.index(3)  # Returns the index of the first occurrence of 3
```

    - **count(item)**: Returns the number of times an element appears in the list.
        - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_list.count(2)  # Returns the count of 2 in the list
```

2. Features of Python List Methods

1. **In-Place Modification**:
    - Many list methods, such as **append()**, **remove()**, and **sort()**, modify the list **in place**, meaning that the changes are applied directly to the original list without creating a new list.
    - **Example**:

```
my_list = [3, 1, 2]
my_list.sort()  # Modifies my_list to [1, 2, 3]
```

2. **Dynamic Resizing**:
    - List methods such as **append()**, **insert()**, and **remove()** allow the list to **grow or shrink** dynamically. Python lists are flexible in size, meaning you don't need to declare the size beforehand.
3. **Handling Multiple Data Types**:
    - List methods work with lists containing different data types. For instance, you can use **append()** to add elements of any type to a list, whether they are integers, strings, or even other lists.
4. **Return Values**:
    - Some list methods like **pop()** and **index()** return values, such as the removed element or the index of an element, while others like **append()** and **sort()** do not return anything (None).
5. **Chaining Methods**:
    - Python lists allow **method chaining**, meaning you can call multiple list methods in a sequence (as long as the method returns the list itself).
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_list.append(4).reverse()  # Works because methods return the list object (if app
```

- **Functions**: Python list methods provide a range of operations such as **adding**, **removing**, **modifying**, and **searching** elements in a list.
- **Features**: List methods support **in-place modification**, **dynamic resizing**, work with **multiple data types**, and can return useful information such as indices or element counts.

Python list methods offer a flexible and powerful way to manage and manipulate lists in your programs.

6.7 List Remove and Del Method (KT0607) (IAC0601)

Both **remove()** and **del** are used to remove elements from a list in Python, but they work in different ways and have distinct features.

1. remove() Method

Function:

- The **remove(item)** method is used to remove the **first occurrence** of a specified value from the list.
- If the specified value is not found in the list, it raises a **ValueError**.

Syntax:

```
list.remove(item)
```

Example:

```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")  # Removes the first occurrence of "banana"
print(fruits)  # Output: ["apple", "cherry", "banana"]
```


Copyright © 2025 HyperionDev. All rights reserved.

Features:

- **Removes by Value**: The remove() method deletes an element by its **value**, not by its index.
- **Removes First Match**: It only removes the **first occurrence** of the specified element. If the element appears multiple times, subsequent occurrences remain.
- **Raises Error if Not Found**: If the item is not found in the list, it raises a ValueError.

2. del Statement

Function:

- The **del** statement is used to remove an element (or slice of elements) from a list based on its **index**.
- The del statement can also be used to delete the entire list.

Syntax:

```
del list[index]       # Deletes a specific element by index
del list[start:end]   # Deletes a slice of elements
del list              # Deletes the entire list
```

Example:

```
fruits = ["apple", "banana", "cherry", "banana"]
del fruits[1]  # Removes the element at index 1 ("banana")
print(fruits)  # Output: ["apple", "cherry", "banana"]

# Removing a slice of elements
del fruits[0:2]  # Removes the first two elements
print(fruits)  # Output: ["banana"]
```

Features:

- **Removes by Index**: The del statement removes elements based on their **index**, not their value.
- **Supports Slicing**: You can use del to remove a **slice** of elements from the list, i.e., a range of elements.

Copyright © 2025 HyperionDev. All rights reserved.

- **Can Delete the Entire List**: You can delete the entire list with del list_name, removing the list from memory.
- **Does Not Return a Value**: Like remove(), the del statement does not return a value.

Key Differences Between remove() and del

Feature	remove()	del
Removes by	Value (first occurrence of the element)	Index (or a range of indices)
Raises Error if Not Found	Yes (ValueError)	No (you must ensure the index exists)
Deletes Slice	No	Yes (supports slicing with start:end)
Deletes Entire List	No	Yes (using del list_name)
Return Value	None	None

- **remove()**: Removes the **first occurrence** of a specified value from the list. It works by value and raises a ValueError if the value is not found.
- **del**: Removes elements from a list by **index** or **slice,** or deletes the entire list. It does not raise an error when removing elements by index if the index is valid.

Both methods are useful for different scenarios, with remove() focusing on values and del focusing on indices or ranges.

6.8 Python Append Method (KT0608) (IAC0601)

The **append()** method in Python is used to **add an element** to the **end** of a list. It is one of the most commonly used list methods for dynamically expanding lists by adding new items. The method modifies the original list in place and does not return a new list or any value (it returns None).

Function of append()

Copyright © 2025 HyperionDev. All rights reserved.

Purpose:

- The append() method **adds a single element** to the end of an existing list.

Syntax:

```
list.append(item)
```

- **item**: The element you want to add to the list. This can be any data type, including integers, strings, floats, Booleans, lists, or even objects.

Example:

```
fruits = ["apple", "banana", "cherry"]
fruits.append("orange")  # Adds "orange" to the end of the list
print(fruits)  # Output: ["apple", "banana", "cherry", "orange"]
```

Features of append()

1. **Adds an Element to the End**:
    - The append() method always adds the element to the **end** of the list, expanding the list by one element.
    - **Example**:

```
numbers = [1, 2, 3]
numbers.append(4)
print(numbers)  # Output: [1, 2, 3, 4]
```

2. **Works with Any Data Type**:
    - The item added to the list can be of any data type, including numbers, strings, lists, and even custom objects.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
mixed_list = [1, "hello"]
mixed_list.append([2, 3])  # Adds a list as an element
print(mixed_list)  # Output: [1, "hello", [2, 3]]
```

3. **Modifies the List In-Place**:
    - The append() method directly modifies the **original list** without creating a copy. It does not return a new list or any other value (returns None).
    - **Example**:

```
my_list = [1, 2]
my_list.append(3)  # Modifies my_list to [1, 2, 3]
print(my_list)  # Output: [1, 2, 3]
```

4. **Single Element Addition**:
    - append() adds **one element** at a time. If you want to add multiple elements, you need to call append() multiple times or use other methods like **extend()**.
    - **Example**:

```
numbers = [1, 2]
numbers.append([3, 4])  # Adds the list [3, 4] as a single element
print(numbers)  # Output: [1, 2, [3, 4]] (nested list)
```

5. **Efficient for Expanding Lists**:
    - The append() method is **efficient** for adding elements to lists, as it only adds the element at the end without having to reassign the entire list.

The **append()** method in Python adds an element to the **end** of a list.

It works with **any data type**, modifies the list **in place**, and adds **only one element** at a time.

append() is useful for dynamically adding items to a list without returning a new list or modifying the entire list.

Copyright © 2025 HyperionDev. All rights reserved.

append() is a simple yet powerful tool for building or expanding lists in Python.

6.9 Python Sort Method (KT0609) (IAC0601)

The Python sort() method is used to sort the elements of a list in place, meaning it modifies the original list. It arranges the elements in ascending order by default but can also be customized with additional parameters to change the behaviour of the sort.

Key Features of sort():

1. **In-place Sorting**: It sorts the list in place, meaning it changes the original list without creating a new one.

```
my_list = [3, 1, 2]
my_list.sort()  # Modifies my_list to [1, 2, 3]
```

2. **Sorting Order**: By default, it sorts in ascending order. To sort in descending order, you can pass reverse=True.

```
my_list = [3, 1, 2]
my_list.sort(reverse=True)  # Now, my_list becomes [3, 2, 1]
```

3. **Custom Key Function**: You can provide a key argument, which allows you to specify a custom function to determine the sorting criteria. For example, you can sort strings by their length:

```
my_list = ['apple', 'banana', 'pear']
my_list.sort(key=len)  # Sorts by length: ['pear', 'apple', 'banana']
```

4. **Stability**: Python's sort() method is stable, meaning it preserves the relative order of elements that compare equal. If two elements have the same key, their original order will be maintained in the sorted list.

Copyright © 2025 HyperionDev. All rights reserved.

Time Complexity:

- The sort() method uses **Timsort**, a hybrid sorting algorithm derived from merge sort and insertion sort.
- Average and worst-case time complexity is **O(n log n)**, where n is the number of elements in the list.

Important Considerations:

- Since sort() works in place, it doesn't return a new list, so its return value is always None.
- If you need to maintain the original list and get a sorted copy instead, you can use the sorted() function, which returns a new sorted list without modifying the original.

The sort() method is a versatile and efficient way to sort lists, offering options for custom sorting behaviour with its key and reverse arguments.

6.10 Python List count() method (KT0610) (IAC0601)

The Python count() method is a built-in function used to count the occurrences of a specific element in a list.

Key Features of count():

1. **Counts Occurrences**: It returns the number of times a specified element appears in the list.

```
my_list = [1, 2, 3, 2, 2, 4]
my_list.count(2)  # Returns 3 because 2 appears 3 times
```

2. **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
list.count(element)
```

- **element**: The item whose occurrences you want to count in the list.

3. **Return Value**: The method returns an integer representing the count of the specified element in the list. If the element is not found, it returns 0.

```
my_list = ['apple', 'banana', 'apple']
my_list.count('apple')  # Returns 2
my_list.count('cherry')  # Returns 0
```

4. **Non-Modifying**: The count() method does not modify the original list. It simply returns the count of the element.

Example:

```
my_list = [10, 20, 10, 30, 10, 40]
count_of_tens = my_list.count(10)  # count_of_tens will be 3
```

Use Case:

The count() method is useful when you need to determine how many times a particular item appears in a list, such as counting votes, duplicates, or specific values in datasets.

The count() method is simple and effective for finding how often a specific element occurs within a list.

6.11 Python List index() method (KT0611) (IAC0601)

The index() method in Python is used to find the position of the **first occurrence** of a specified element in a list. It returns the index (or position) of that element, which tells us where the element is located within the list.

Functions and Features of the index() Method:

1. **Finds the Position of an Element**:

Copyright © 2025 HyperionDev. All rights reserved.

- The main function of the index() method is to return the index (position) of the first time a specified element appears in the list.
- Indexing in Python starts at **0**, meaning the first element has index 0, the second has index 1, and so on.
- **Example**:

```
my_list = ['apple', 'banana', 'cherry']
print(my_list.index('banana'))  # Output: 1, because 'banana' is at index 1
```

2. **Raises an Error if the Element is Not Found**:
   - Unlike the count() method, which returns 0 if the element isn't found, the index() method will raise a **ValueError** if the element is not present in the list.

     This means you need to be sure the element is in the list before using it, or handle the error with a try-except block.
   - **Example**:

```
my_list = [10, 20, 30]
# This will raise an error because 40 is not in the list
# print(my_list.index(40))  # Raises ValueError: 40 is not in list
```

3. **Can Search Within a Specified Range**:
   - You can limit the search to a specific section of the list by providing **start** and **end** index values. This feature helps if you only want to find the position of an element within a particular slice of the list.
   - **Syntax**: list.index(element, start, end)
   - **Example**:

```
my_list = [1, 2, 3, 4, 3, 5]
print(my_list.index(3, 2, 5))  # Output: 2, starts searching from index 2 and stops
```


Copyright © 2025 HyperionDev. All rights reserved.

4. **Returns the Index of the First Occurrence**:
    - If the element appears multiple times in the list, index() will only return the index of the **first** occurrence and stop searching after that.
    - **Example**:

```
my_list = ['apple', 'banana', 'apple', 'cherry']
print(my_list.index('apple'))  # Output: 0, since the first 'apple' is at index 0
```

Step-by-Step Example:

Let's see the index() method in action with a simple list:

```
# Example: Using the index() method
colors = ['red', 'blue', 'green', 'blue', 'yellow']

# Find the index of 'blue'
index_of_blue = colors.index('blue')
print(f"The first 'blue' is at index {index_of_blue}.")  # Output: The first 'blue' is at

# Find the index of 'green'
index_of_green = colors.index('green')
print(f"'green' is at index {index_of_green}.")  # Output: 'green' is at index 2.

# Find 'blue' but start searching from index 2
index_of_blue_after_2 = colors.index('blue', 2)
print(f"The next 'blue' after index 2 is at index {index_of_blue_after_2}.")  # Output: Th
```

Key Takeaways:

- **Simple and Quick**: The index() method helps you find the position of an element in the list.
- **Start Searching from a Specific Index**: You can search for an element starting from a specific index or limit the search to a range of indices.
- **Caution for Errors**: If the element isn't in the list, Python raises a ValueError, so it's important to ensure the element exists or handle the error properly.

Copyright © 2025 HyperionDev. All rights reserved.

6.12 List Comprehension and Reverse (KT0612) (IAC0601)

List comprehension in Python is a concise way to create lists. It allows you to generate new lists by applying an expression to each element in an existing list or another iterable (like a string or a range), all within a single line of code. List comprehensions are known for being more readable and often faster than using traditional loops.

Functions and Features:

1. **Creates New Lists Efficiently**:
    - List comprehension provides a compact way to generate lists without using loops or multiple lines of code. It's especially useful when you want to perform an operation on each element of an existing list or create a filtered version of the list.
    - **Example**:

```
numbers = [1, 2, 3, 4, 5]
squares = [x ** 2 for x in numbers]
print(squares)  # Output: [1, 4, 9, 16, 25]
```

2. **Optional Conditions (Filtering)**:
    - You can add an **if condition** to filter elements based on a condition. Only elements that satisfy the condition will be included in the new list.
    - **Example**:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = [x for x in numbers if x % 2 == 0]
print(even_numbers)  # Output: [2, 4, 6]
```

3. **Nested Loops**:
    - List comprehension supports multiple for loops, allowing you to work with nested data or create combinations of elements.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
pairs = [(x, y) for x in [1, 2, 3] for y in [4, 5, 6]]
print(pairs)  # Output: [(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (
```

4. **Readable and Concise**:
    - One of the key advantages of list comprehension is that it reduces the amount of code, making it more readable and elegant compared to traditional loops.

reverse() Method:

Description:

The reverse() method in Python is a built-in list method that reverses the order of elements in a list **in place**. This means that it changes the original list, rather than creating a new reversed list.

Functions and Features:

1. **Reverses the List In Place**:
    - The reverse() method directly modifies the original list, changing the order of the elements to be in reverse.
    - **Example**:

```
my_list = [1, 2, 3, 4, 5]
my_list.reverse()
print(my_list)  # Output: [5, 4, 3, 2, 1]
```

2. **No Return Value**:
    - The reverse() method does not return a new list. Instead, it modifies the original list in place and returns None.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_list = ['a', 'b', 'c']
reversed_list = my_list.reverse()
print(reversed_list)  # Output: None
print(my_list)  # Output: ['c', 'b', 'a']
```

3. **Useful for In-Place Modification**:
    - The reverse() method is particularly useful when you want to reverse a list without creating a copy, saving memory and time.
4. **Alternative Using Slicing**:
    - While reverse() modifies the list in place, you can also reverse a list using slicing ([::-1]), which creates a new reversed list without changing the original one.
    - **Example**:

```
my_list = [10, 20, 30]
reversed_list = my_list[::-1]
print(reversed_list)  # Output: [30, 20, 10]
```

Key Differences Between List Comprehension and reverse():

- **List Comprehension** is a way to **generate new lists** from existing ones by applying transformations or filters.
- **reverse()** is a method that **modifies the order** of elements in an existing list, but doesn't create a new list.

List comprehension is a powerful tool to create and manipulate lists in a clean and concise manner.

reverse() is a quick way to reverse the order of elements in a list directly.

Both tools help you work efficiently with lists but serve very different purposes!

Copyright © 2025 HyperionDev. All rights reserved.

6.13 Nested lists (KT0613) (IAC0601)

A **nested list** in Python is a list that contains other lists as its elements. In other words, it's a **list of lists**. You can think of it like a 2D (or even higher-dimensional) structure, where each element in the outer list can be another list containing its own set of elements.

Functions and Features of Nested Lists:

1. **Storing Complex Data Structures**:
    - Nested lists allow you to store **complex data structures**. For example, if you want to store a table or matrix, you can use a nested list where each inner list represents a row.
    - **Example**:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Here, matrix is a 2D list, where each row (inner list) contains three numbers.

2. **Accessing Elements in Nested Lists**:
    - You can access elements in a nested list using **multiple indices**. The first index refers to the outer list (row), and the second index refers to the inner list (column).
    - **Example**:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[1][2])  # Output: 6, which is in the second row and third column
```

        - The first index 1 selects the second row [4, 5, 6].
        - The second index 2 selects the third element 6 from that row.

Copyright © 2025 HyperionDev. All rights reserved.

3. **Modifying Elements in Nested Lists**:
    - You can **modify elements** in a nested list by assigning a new value to a specific position, using the same multi-index approach.
    - **Example**:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
matrix[1][1] = 50  # Changes the value at row 2, column 2 to 50
print(matrix)  # Output: [[1, 2, 3], [4, 50, 6], [7, 8, 9]]
```

4. **Iterating Over Nested Lists**:
    - To process all the elements in a nested list, you can use **nested loops**. The outer loop iterates through the outer list (rows), while the inner loop iterates through the inner lists (columns).
    - **Example**:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for row in matrix:
    for item in row:
        print(item, end=' ')  # Output: 1 2 3 4 5 6 7 8 9
```

5. **Nested List Comprehensions**:
    - Python allows you to create nested lists using **nested list comprehensions**. This feature is a concise way to generate complex lists in just one line.
    - **Example**:

```
# Create a 3x3 matrix using nested list comprehension
matrix = [[i * j for j in range(1, 4)] for i in range(1, 4)]
print(matrix)  # Output: [[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```


Copyright © 2025 HyperionDev. All rights reserved.

6. **Flexible Dimensions**:
    - Nested lists are not limited to just 2 dimensions (like tables or matrices). You can nest lists within lists to create higher-dimensional structures, such as 3D lists (lists of lists of lists).

    - **Example**:

```
# 3D list: a list of matrices
cube = [
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
]
print(cube[1][0][1])  # Output: 6, from the second matrix, first row, second element
```

7. **Mixing Data Types**:
    - Nested lists can store lists of mixed data types, allowing for great flexibility. Each inner list doesn't need to have the same length or type of data.

    - **Example**:

```
mixed_list = [
    [1, 2, 3],
    ['apple', 'banana'],
    [True, False, None]
]
print(mixed_list[1][0])  # Output: 'apple'
```

Key Takeaways:

- **Organizes complex data**: Nested lists are ideal for representing things like tables, grids, matrices, and multi-level hierarchical data.
- **Access using multiple indices**: You can easily access elements in nested lists using multiple indices (list[row][column]).
- **Modifiable and flexible**: You can modify elements at any level, create lists with varying dimensions, and use nested loops for processing.

Copyright © 2025 HyperionDev. All rights reserved.

Nested lists in Python are a powerful and flexible way to handle complex data structures in a clean and organized manner!



Formative Assessment Activity [6]

Python List

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT07:

Topic Code	KM-02-KT07:
Topic	Python tuple
Weight	10%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0701)
- Syntax (KT0702)
- Tuple assignment (KT0703)
- Pack, Unpack, Compare, Slicing, Delete, Keys in dictionaries (KT0704)
- Built-in functions with tuples (KT0705)
- Advantages of tuples over lists (KT0706)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0701 Definitions, functions and features of Python tuple are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

7.1 Concept, definition and functions (KT0701) (IAC0701)

A **tuple** is a fundamental data structure in Python that allows you to store a collection of items. Tuples are like lists but have one key difference: **they are immutable**, meaning once a tuple is created, its elements cannot be changed, added, or removed. This immutability makes tuples useful when you want to ensure that the data remains constant throughout your program.

Definition:

A **tuple** is a collection of **ordered elements** that can hold multiple data types (like numbers, strings, etc.). Tuples are defined by enclosing the elements in **parentheses** ().

- **Example**:

```
my_tuple = (1, 2, 3, 'apple', 'banana')
```

You can store any type of data in a tuple (integers, strings, floats, lists, even other tuples). Tuples are often used when you want a sequence of items that should not be modified.

Functions and Features of Tuples:

1. **Immutable**:
    - Once a tuple is created, you cannot change its elements. This immutability makes tuples useful when you need **read-only data**.
    - **Example**:

```
my_tuple = (1, 2, 3)
# Trying to modify a tuple will raise an error:
# my_tuple[0] = 10  # Error: 'tuple' object does not support item assignment
```

2. **Ordered**:
    - Just like lists, tuples are **ordered** collections. This means that the elements in a tuple have a defined order, and you can access them using an **index**.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_tuple = ('a', 'b', 'c')
print(my_tuple[1])  # Output: 'b', because index 1 refers to the second element
```

3. **Supports Multiple Data Types**:
    - Tuples can store elements of different data types (integers, strings, floats, etc.), just like lists.
    - **Example**:

```
my_tuple = (1, 'apple', 3.14)
```

4. **Accessing Tuple Elements**:
    - You can access tuple elements using **indexing** (just like lists) or iterate over the elements using a loop.
    - **Example**:

```
fruits = ('apple', 'banana', 'cherry')
print(fruits[0])  # Output: 'apple'
for fruit in fruits:
    print(fruit)  # Output: apple, banana, cherry
```

5. **Tuple Packing and Unpacking**:
    - You can easily **pack** values into a tuple and then **unpack** them into individual variables.
    - **Example**:

```
# Packing
person = ("John", 25, "New York")

# Unpacking
name, age, city = person
print(name)  # Output: John
print(age)   # Output: 25
print(city)  # Output: New York
```


Copyright © 2025 HyperionDev. All rights reserved.

6. **Tuple Methods**:
    - Although tuples are immutable, Python provides a couple of methods to work with tuples:
        - **count()**: Returns the number of occurrences of a specific element in the tuple.

```
my_tuple = (1, 2, 2, 3, 2)
print(my_tuple.count(2))  # Output: 3
```

        - **index()**: Returns the index of the first occurrence of a specific element.

```
my_tuple = (1, 2, 3, 4)
print(my_tuple.index(3))  # Output: 2
```

7. **Supports Nesting**:
    - Tuples can contain other tuples, making it possible to create **nested tuples**.
    - **Example**:

```
nested_tuple = (1, (2, 3), (4, 5))
print(nested_tuple[1])  # Output: (2, 3)
```

8. **Tuples Are Hashable**:
    - Tuples can be used as **keys in dictionaries** or stored in sets because they are hashable (since their contents cannot be changed). This is not possible with lists.
    - **Example**:

```
my_dict = {(1, 2): 'a', (3, 4): 'b'}
print(my_dict[(1, 2)])  # Output: 'a'
```


Copyright © 2025 HyperionDev. All rights reserved.

- **Tuples** are **immutable**, **ordered**, and **indexable** collections that allow you to store multiple data types.
- They support methods like count() and index(), and you can easily pack/unpack values with tuples.
- Since tuples cannot be modified, they are useful when you need **read-only**, **constant** data, and can even be used as dictionary keys or in sets.

7.2 Syntax (KT0702) (IAC0701)

A **tuple** in Python is a collection of ordered, immutable elements. Tuples allow you to store multiple values in a single variable, and once created, their contents cannot be changed (they are **immutable**).

Basic Syntax:

Tuples are defined using **parentheses** () with elements separated by **commas**.

- **Example**:

```
my_tuple = (1, 2, 3)
```

This creates a tuple my_tuple containing the elements 1, 2, and 3.

Key Features of Tuple Syntax:

1. **Multiple Elements**:
    - You can store multiple values, and they can be of different data types (e.g., numbers, strings).
    - **Example**:

```
my_tuple = (1, "apple", 3.14)
```

2. **Empty Tuple**:

Copyright © 2025 HyperionDev. All rights reserved.

- You can create an empty tuple by using empty parentheses.
- **Example**:

```
empty_tuple = ()
```

3. **Single-Element Tuple**:
   - For a tuple with just one element, you must add a comma after the single item, or Python will not recognize it as a tuple.
   - **Example**:

```
single_element_tuple = (5,)  # Tuple with one element
```

4. **Accessing Elements**:
   - You can access elements in a tuple using **indexing**, where the first element has index 0.
   - **Example**:

```
my_tuple = ('a', 'b', 'c')
print(my_tuple[1])  # Output: 'b'
```

5. **Immutability**:
   - Tuples are **immutable**, meaning you cannot change their contents after they are created. This includes adding, removing, or modifying elements.
   - **Example**:

```
my_tuple = (10, 20, 30)
# This will raise an error: my_tuple[1] = 40
```

- **Tuples** are defined using **parentheses** () with elements separated by **commas**.
- Tuples are **immutable**, meaning once created, they cannot be modified.

Copyright © 2025 HyperionDev. All rights reserved.

- They can store **multiple types of data** and support operations like **indexing** to access elements.

Tuples are ideal when you need a collection of data that shouldn't be changed during the program.

7.3 Tuple assignment (KT0703) (IAC0701)

Tuple assignment in Python is a convenient way to assign multiple values to variables in a single step. This feature is based on the concept of **packing** and **unpacking** tuples. It allows you to assign elements from a tuple (or any iterable) to multiple variables at once, simplifying your code.

Basic Syntax of Tuple Assignment:

In tuple assignment, the values on the **right-hand side** (RHS) are **packed** into a tuple, and the variables on the **left-hand side** (LHS) are **unpacked** to receive those values.

- **Example**:

```
a, b, c = 1, 2, 3
```

Here, the tuple (1, 2, 3) is unpacked, and its values are assigned to the variables a, b, and c in order.

Key Features of Tuple Assignment:

1. **Multiple Assignments in One Step**:
    - Tuple assignment allows you to assign multiple variables at once, reducing code complexity.
    - **Example**:

```
x, y = 10, 20
print(x)  # Output: 10
print(y)  # Output: 20
```

2. **Packing and Unpacking**:

Copyright © 2025 HyperionDev. All rights reserved.

- **Packing** refers to grouping multiple values into a tuple, and **unpacking** refers to assigning those values to individual variables.
- **Example**:

```
# Packing
my_tuple = (5, 10, 15)

# Unpacking
a, b, c = my_tuple
print(a, b, c)  # Output: 5 10 15
```

3. **Swapping Variables**:
   - Tuple assignment makes it easy to **swap** values between variables without needing a temporary variable.
   - **Example**:

```
x, y = 1, 2
x, y = y, x  # Swapping values
print(x, y)  # Output: 2 1
```

4. **Works with Any Iterable**:
   - Tuple assignment works not just with tuples but with any **iterable** like lists or strings.
   - **Example**:

```
a, b, c = [10, 20, 30]  # List
print(a, b, c)  # Output: 10 20 30
```

5. **Unpacking with a Placeholder**:
   - You can unpack only specific values and use _ (underscore) as a placeholder for values you don't need.
   - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
_, y, _ = (100, 200, 300)
print(y)  # Output: 200
```

- **Tuple assignment** allows multiple variables to be assigned values at once using tuple packing and unpacking.
- It simplifies code by handling multiple assignments in a single line and makes tasks like **variable swapping** easier.
- It can be used with any **iterable**, not just tuples.

This feature is great for improving code readability and efficiency when dealing with multiple values.

7.4 Pack, Unpack, Compare, Slicing, Delete, Keys in dictionaries (KT0704) (IAC0701)

Packing refers to combining multiple values into a **tuple** (or other collection like a list) in one step.

- **Example**:

```
my_tuple = 1, 2, 3  # Packing values into a tuple
print(my_tuple)  # Output: (1, 2, 3)
```

Unpack:

Unpacking is the process of assigning the values from a tuple (or another collection) to individual variables.

- **Example**:

```
a, b, c = my_tuple  # Unpacking the tuple into variables
print(a, b, c)  # Output: 1 2 3
```

Unpacking simplifies extracting values from collections into separate variables.

Copyright © 2025 HyperionDev. All rights reserved.

2. Compare in Python

In Python, you can **compare** tuples (and lists) element by element using comparison operators like ==, !=, <, >, etc. Python compares tuples **lexicographically**, which means it compares the first element of each tuple. If they are equal, it compares the second element, and so on.

- **Example**:

```
tuple1 = (1, 2, 3)
tuple2 = (1, 2, 4)

print(tuple1 == tuple2)  # Output: False
print(tuple1 < tuple2)   # Output: True (because 3 < 4)
```

3. Slicing in Python

Slicing is a technique used to access a subset of elements from a tuple, list, or string by specifying a range of indices.

Syntax:

```
slice = collection[start:end]
```

- start: The index to start the slice (inclusive).
- end: The index to end the slice (exclusive).

Example:

```
my_tuple = (0, 1, 2, 3, 4, 5)
print(my_tuple[1:4])  # Output: (1, 2, 3)
```

You can also use **negative indices** to slice from the end of the tuple.

4. Delete in Python

You cannot modify or delete individual elements of a **tuple** because they are **immutable**. However, you can delete the **entire tuple** using the del keyword.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
my_tuple = (10, 20, 30)
del my_tuple  # Deletes the entire tuple
# Trying to print the tuple will raise an error
```

For lists, you can delete specific elements using del or other methods like .remove().

5. Keys in Dictionaries

In **dictionaries**, **keys** are unique identifiers used to store and retrieve values. **Tuples** (and other immutable types like strings and numbers) can be used as dictionary keys, but **lists** and other mutable types cannot.

- **Example**:

```
my_dict = {
    (1, 2): "point A",
    (3, 4): "point B"
}
print(my_dict[(1, 2)])  # Output: point A
```

Dictionaries store key-value pairs, where the key must be an immutable type (like a tuple).

- **Pack** and **Unpack** simplify working with multiple values in collections.
- **Compare** allows you to evaluate tuples (and lists) element by element.
- **Slicing** helps retrieve parts of collections.
- **Delete** can be used to remove entire tuples or specific elements from lists.
- **Keys in dictionaries** must be immutable, and tuples are commonly used as keys in dictionaries for this reason.

These features are essential for managing data structures efficiently in Python.

Copyright © 2025 HyperionDev. All rights reserved.

7.5 Built-in functions with tuples (KT0705) (IAC0701)

Tuples are a built-in data structure in Python, and there are several **built-in functions** that work with tuples to perform operations like counting elements, finding lengths, or transforming data.

1. len() – Find the Length of a Tuple

The **len()** function returns the number of elements in a tuple.

- **Syntax**:

```
len(tuple)
```

- **Example**:

```
my_tuple = (10, 20, 30)
print(len(my_tuple))  # Output: 3
```

2. max() – Find the Maximum Value in a Tuple

The **max()** function returns the largest element in a tuple. All elements must be of the same type (like all numbers or all strings).

- **Syntax**:

```
max(tuple)
```

- **Example**:

```
my_tuple = (10, 50, 20, 30)
print(max(my_tuple))  # Output: 50
```

3. min() – Find the Minimum Value in a Tuple

The **min()** function returns the smallest element in a tuple, similar to max().

Copyright © 2025 HyperionDev. All rights reserved.

- **Syntax**:

```
min(tuple)
```

- **Example**:

```
my_tuple = (10, 50, 20, 30)
print(min(my_tuple))  # Output: 10
```

4. sum() – Sum the Elements in a Tuple

The **sum()** function adds up all the elements in a tuple. This works only with numeric data types.

- **Syntax**:

```
sum(tuple)
```

- **Example**:

```
my_tuple = (10, 20, 30)
print(sum(my_tuple))  # Output: 60
```

5. sorted() – Return a Sorted List of Tuple Elements

The **sorted()** function returns a **new sorted list** of the elements in a tuple. It does **not** modify the original tuple, because tuples are immutable.

- **Syntax**:

```
sorted(tuple)
```

- **Example**:

```
my_tuple = (30, 10, 50, 20)
print(sorted(my_tuple))  # Output: [10, 20, 30, 50]
```


Copyright © 2025 HyperionDev. All rights reserved.

6. any() – Check if Any Element in the Tuple is True

The **any()** function returns True if at least one element in the tuple is True (or truthy). If the tuple is empty or all elements are False, it returns False.

- **Syntax**:

```
any(tuple)
```

- **Example**:

```
my_tuple = (0, False, 5)
print(any(my_tuple))  # Output: True
```

7. all() – Check if All Elements in the Tuple are True

The **all()** function returns True if **all** elements in the tuple are True. If any element is False, it returns False.

- **Syntax**:

```
all(tuple)
```

- **Example**:

```
my_tuple = (1, True, 3)
print(all(my_tuple))  # Output: True
```

8. tuple() – Convert an Iterable into a Tuple

The **tuple()** function converts other iterables (like lists, strings, or sets) into a tuple.

- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
tuple(iterable)
```

- **Example**:

```
my_list = [10, 20, 30]
my_tuple = tuple(my_list)
print(my_tuple)  # Output: (10, 20, 30)
```

9. count() – Count Occurrences of an Element in a Tuple

The **count()** method returns the number of times a specified value occurs in a tuple.

- **Syntax**:

```
tuple.count(element)
```

- **Example**:

```
my_tuple = (1, 2, 2, 3, 2)
print(my_tuple.count(2))  # Output: 3
```

10. index() – Find the Index of an Element

The **index()** method returns the index of the first occurrence of a specified element in the tuple. If the element is not found, it raises a ValueError.

- **Syntax**:

```
tuple.index(element)
```

- **Example**:

```
my_tuple = (10, 20, 30)
print(my_tuple.index(20))  # Output: 1
```


Copyright © 2025 HyperionDev. All rights reserved.

- Tuples work with several **built-in functions** that provide utility for handling and analysing their elements.
- Functions like len(), max(), min(), sum(), and sorted() help you inspect tuple content, while methods like count() and index() give more control over specific elements.
- **tuple()** converts other iterables into tuples, and **any()** and **all()** provide ways to check for truth values in a tuple.

These built-in functions are essential for efficiently working with tuples in Python.

7.6 Advantages of tuples over lists (KT0706) (IAC0701)

While both tuples and lists are used to store collections of items, tuples have certain **advantages** over lists that make them useful in specific situations.

1. Immutability:

- **Tuples are immutable**, meaning their elements cannot be changed, added, or removed after they are created. This immutability offers several benefits:
  - **Data Integrity**: Tuples ensure that the data remains constant throughout the program, which is useful when you want to protect the data from being accidentally modified.
  - **Safe for Multithreading**: Since tuples cannot be changed, they are safe to use in multithreaded environments without the risk of unexpected modifications.

Example:

```
my_tuple = (1, 2, 3)
# This will raise an error: my_tuple[0] = 10
```

2. Performance:

- **Tuples are faster** than lists, particularly for large datasets. Since tuples are immutable, Python can optimize them better for performance, making operations like iteration faster.

Example:

Copyright © 2025 HyperionDev. All rights reserved.

- Accessing elements or iterating over a tuple is generally faster than with a list of the same size.

3. Memory Efficiency:

- **Tuples use less memory** than lists because they are immutable. This makes tuples more efficient in terms of memory usage, especially for large collections of items that don't need to be modified.

Example:

- In situations where memory usage is a concern (e.g., when working with large datasets), tuples are preferable because they take up less space.

4. Used as Dictionary Keys:

- **Tuples can be used as keys in dictionaries**, while lists cannot. This is because dictionary keys must be hashable, and tuples, being immutable, are hashable. Lists, being mutable, are not hashable.

Example:

```
my_dict = { (1, 2): "a", (3, 4): "b" }  # Tuples used as keys
print(my_dict[(1, 2)])  # Output: 'a'
```

5. Return Multiple Values from Functions:

- Tuples are a great way to **return multiple values from a function**. Since they are immutable, they ensure that the returned values won't be accidentally changed after the function call.

Example:

```
def get_coordinates():
    return (10, 20)  # Returning multiple values as a tuple

x, y = get_coordinates()
```

6. Clearer Intent:

Copyright © 2025 HyperionDev. All rights reserved.

- When you use a tuple, it signals to other developers that the collection of items should **not** be modified. This provides a clearer intent in the code, showing that the values are meant to stay constant.

Example:

```
days_of_week = ("Monday", "Tuesday", "Wednesday")  # We know these values won't change
```

7. Functional Programming:

- In **functional programming**, immutability is a core concept. Tuples fit well with this programming paradigm, ensuring that data stays unchanged and is treated as a constant throughout the program.

- **Immutability**: Tuples protect data from being modified.
- **Performance**: Tuples are faster and more efficient in memory usage.
- **Hashable**: Tuples can be used as dictionary keys, unlike lists.
- **Clear Intent**: Tuples signal that data should remain constant.
- **Better for Functional Programming**: Tuples align well with functional programming concepts.

Tuples are preferred when you need a **constant**, **efficient**, and **fixed collection** of data that should not be changed during the execution of the program.



Formative Assessment Activity [7]

Python Tuple

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT08:

Topic Code	KM-02-KT08:
Topic	Dictionary in Python
Weight	10%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0801)
- Syntax (KT0802)
- Properties of dictionary keys (KT0803)
- Python dictionary (Dict) (KT0804)
- Inbuilt methods (KT0805)
- Length of Dictionary (KT0806)
- Dictionary Clear Method (KT0807)
- Update, Cmp, Len, Sort, Copy, Items, str (KT0808)
- Merging dictionaries (KT0809)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0801 Definitions, functions and features of Python dictionaries are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

8.1 Concept, definition and functions (KT0801) (IAC0801)

A **dictionary** in Python is a **mutable**, **unordered** collection of **key-value pairs**. Each key in the dictionary is unique, and it maps to a specific value. Unlike lists or tuples where you access elements by their index, in dictionaries you access values using **keys**. These keys must be **immutable** types (like strings, numbers, or tuples), while values can be of any data type.

- **Syntax**:

```
my_dict = {'key1': 'value1', 'key2': 'value2'}
```

Dictionaries are very efficient for **storing and retrieving** data when the key is known.

Function:

The main function of dictionaries is to allow for **quick and easy lookups** based on keys. They are designed for scenarios where you need to **map** one set of values (keys) to another set (values) and retrieve them efficiently.

- **Example of a lookup**:

```
student = {'name': 'Alice', 'age': 20, 'major': 'Mathematics'}
print(student['name'])  # Output: 'Alice'
```

Dictionaries are great for storing structured data, such as user profiles, product catalogues, or settings.

Features:

1. **Key-Value Pairs**:
    - Dictionaries store data in pairs of **keys** and **values**. Each key is unique, and it allows you to retrieve the associated value.

Copyright © 2025 HyperionDev. All rights reserved.

2. **Keys Must Be Immutable**:
    - Dictionary keys must be immutable types such as strings, numbers, or tuples. Values, on the other hand, can be mutable or immutable.
3. **Fast Data Lookup**:
    - Dictionaries allow **fast retrieval** of values when you know the key, using **constant-time lookups**.
4. **Mutable**:
    - Dictionaries can be modified after creation. You can add, update, or remove key-value pairs dynamically.
    - **Example**:

```
my_dict = {'name': 'Bob'}
my_dict['age'] = 25  # Adding a new key-value pair
```

5. **Unordered (Before Python 3.7)**:
    - Prior to Python 3.7, dictionaries were unordered, meaning the order of items was not guaranteed. Since Python 3.7, dictionaries preserve the **insertion order**.
6. **Common Methods**:
    - **get()**: Safely retrieves a value by key without raising an error if the key is missing.
    - **keys()**: Returns a view of all the keys in the dictionary.
    - **values()**: Returns a view of all the values in the dictionary.
    - **items()**: Returns a view of all the key-value pairs as tuples.
7. **Dynamic Data Handling**:
    - Dictionaries allow you to easily add, remove, and modify key-value pairs, making them very flexible for handling dynamic data.

Copyright © 2025 HyperionDev. All rights reserved.

- **Concept**: A dictionary is a collection of key-value pairs where keys are unique, and values can be any data type.
- **Function**: It provides fast lookups by key, making it efficient for retrieving and managing data.
- **Features**:
    - Keys must be immutable.
    - Dictionaries are mutable and dynamic.
    - They support various useful methods like get(), keys(), and items().
    - From Python 3.7+, they maintain insertion order.

Dictionaries are ideal for scenarios where you need to map and efficiently retrieve data using keys.

8.2 Syntax (KT0802) (IAC0801)

A **dictionary** in Python is a collection of **key-value pairs** where each key is associated with a value. The syntax for creating and manipulating dictionaries is simple and flexible.

Basic Syntax for Creating a Dictionary:

Dictionaries are defined using **curly braces** {}. Inside the curly braces, **keys** and **values** are written in pairs, separated by a colon :. Each key-value pair is separated by a comma ,.

Syntax:

```
my_dict = {
    'key1': 'value1',
    'key2': 'value2',
    'key3': 'value3'
}
```

- **Keys**: Must be unique and immutable (e.g., strings, numbers, or tuples).

Copyright © 2025 HyperionDev. All rights reserved.

- **Values**: Can be any data type (e.g., strings, numbers, lists, or even other dictionaries).

Example:

```
student = {
    'name': 'Alice',
    'age': 22,
    'major': 'Computer Science'
}
```

Accessing Values:

To access a value in a dictionary, you use the **key** inside square brackets [].

Syntax:

```
value = my_dict['key']
```

Example:

```
print(student['name'])  # Output: 'Alice'
```

If the key doesn't exist, it will raise a **KeyError**. To avoid this, you can use the .get() method.

Adding or Modifying Key-Value Pairs:

You can add a new key-value pair or update the value of an existing key by using the assignment operator =.

Syntax:

```
my_dict['new_key'] = new_value
```

Example:

Copyright © 2025 HyperionDev. All rights reserved.

```
student['graduated'] = True  # Adding a new key-value pair
student['age'] = 23  # Modifying an existing value
```

Removing Key-Value Pairs:

To remove a key-value pair, you can use the del statement or the .pop() method.

Syntax:

```
del my_dict['key']  # Removes the key-value pair
value = my_dict.pop('key')  # Removes and returns the value
```

Example:

```
del student['major']  # Removes the 'major' key-value pair
```

Dictionary Methods:

Dictionaries come with several useful methods:

- **get()**: Retrieves a value by key, returns None if the key doesn't exist.

```
print(student.get('name'))  # Output: 'Alice'
print(student.get('grade', 'N/A'))  # Output: 'N/A'
```

- **keys()**: Returns a view of all the keys.

```
print(student.keys())  # Output: dict_keys(['name', 'age', 'graduated'])
```

- **values()**: Returns a view of all the values.

```
print(student.values())  # Output: dict_values(['Alice', 23, True])
```

- **items()**: Returns a view of all key-value pairs as tuples.

```
print(student.items())  # Output: dict_items([('name', 'Alice'), ('age', 23), ('graduat
```


Copyright © 2025 HyperionDev. All rights reserved.

- **Dictionaries** are created using curly braces {} with key-value pairs separated by colons :.
- **Keys** must be unique and immutable, while **values** can be of any data type.
- You can **access**, **add**, **modify**, and **remove** elements using straightforward syntax.
- Useful methods like get(), keys(), values(), and items() help in working with dictionaries efficiently.

Dictionaries are a powerful and flexible way to store and manage data in Python.

8.3 Properties of dictionary keys (KT0803) (IAC0801)

dictionary keys are an essential part of the dictionary data structure, as they are used to **map** values. The keys must follow specific rules and properties to ensure that the dictionary functions properly.

1. Keys Must Be Unique

- **Each key in a dictionary must be unique**. You cannot have duplicate keys. If you assign a value to an existing key, the new value will **overwrite** the previous value associated with that key.
    - **Example**:

```
my_dict = {'name': 'Alice', 'age': 25, 'name': 'Bob'}
print(my_dict)  # Output: {'name': 'Bob', 'age': 25}
```

In this case, the key 'name' appears twice, so the second assignment ('Bob') overwrites the first ('Alice').

2. Keys Must Be Immutable

- **Dictionary keys must be immutable types**, meaning they cannot be modified after creation. This ensures that the keys remain constant throughout the dictionary's lifetime.

Copyright © 2025 HyperionDev. All rights reserved.

- The most common immutable types used as keys are:
    - **Strings**
    - **Numbers (integers or floats)**
    - **Tuples** (if they only contain immutable elements)

Mutable types, such as **lists** or **dictionaries**, cannot be used as keys because their contents can change, which would break the key-value mapping.

- **Example**:

```
my_dict = {(1, 2): 'coordinates', 'name': 'Alice'}
print(my_dict[(1, 2)])  # Output: 'coordinates'
```

In this example, a tuple (1, 2) is used as a key because it is immutable.

3. Keys Are Case-Sensitive

- **Keys are case-sensitive**, which means 'Name' and 'name' would be treated as two different keys, even though the string content looks similar.
    - **Example**:

```
my_dict = {'Name': 'Alice', 'name': 'Bob'}
print(my_dict)  # Output: {'Name': 'Alice', 'name': 'Bob'}
```

Here, both 'Name' and 'name' are considered distinct keys because of their different cases.

4. Keys Can Be of Different Data Types

- **Keys** in a dictionary can be of different data types, as long as they are immutable. You can mix different types of keys (e.g., using both strings and integers in the same dictionary).
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_dict = {'name': 'Alice', 42: 'The Answer'}
print(my_dict)  # Output: {'name': 'Alice', 42: 'The Answer'}
```

In this example, a string ('name') and an integer (42) are both used as keys.

5. Keys Must Be Hashable

- **Keys must be hashable**, meaning they must have a fixed hash value during their lifetime. This hash value is used to map the key to a specific location in the dictionary.

Hashable objects include **immutable** types like strings, numbers, and tuples. **Mutable types** like lists and dictionaries cannot be hashed, and thus, cannot be used as dictionary keys.

  - **Example of a Hashable Key**:

```
my_dict = {('x', 'y'): 'coordinates'}  # A tuple as a key
print(my_dict[('x', 'y')])  # Output: 'coordinates'
```

  - **Non-hashable Key Example (Invalid)**:

```
my_dict = {[1, 2]: 'invalid'}  # This will raise a TypeError because lists are muta
```

- **Keys must be unique**: No duplicates allowed, and reassigning a value to an existing key will overwrite the previous value.
- **Keys must be immutable**: Only immutable types like strings, numbers, and tuples can be used as keys.
- **Keys are case-sensitive**: 'key' and 'Key' are different.
- **Keys can be of different data types**: A dictionary can contain keys of various types, if they are immutable.
- **Keys must be hashable**: The key must have a fixed hash value, making mutable objects like lists unusable as keys.

Copyright © 2025 HyperionDev. All rights reserved.

These properties ensure that dictionaries remain efficient and reliable when storing and retrieving data.

8.4 Python dictionary (Dict) (KT0804) (IAC0801)

A **dictionary** (or **dict**) in Python is a built-in data structure that stores data in the form of **key-value pairs**. Dictionaries are **mutable**, meaning their contents can be changed after creation, and are particularly useful for **fast lookups** by key.

Key Features of Python Dictionaries:

1. **Key-Value Pairs**:
    - A dictionary consists of **keys**, which are unique, and each key maps to a specific **value**. Keys act as labels, and values are the data associated with those labels.
    - **Example**:

```
my_dict = {'name': 'Alice', 'age': 25}
```

Here, 'name' and 'age' are keys, and 'Alice' and 25 are their corresponding values.

2. **Unordered (Before Python 3.7)**:
    - Prior to Python 3.7, dictionaries were **unordered**, meaning there was no guaranteed order for key-value pairs. Since Python 3.7, dictionaries maintain **insertion order** by default, which means the order in which you add items will be preserved.
3. **Mutable**:
    - Dictionaries are mutable, so you can **add**, **modify**, and **remove** key-value pairs dynamically after the dictionary is created.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_dict['city'] = 'New York'  # Adding a new key-value pair
my_dict['age'] = 26  # Modifying an existing value
```

4. **Keys Must Be Unique and Immutable**:
    - **Keys must be unique** in a dictionary. If you try to use the same key twice, the most recent value will overwrite the previous one.
    - **Keys must also be immutable**, meaning they cannot be changed. You can use strings, numbers, or tuples as keys, but **lists** and **dictionaries** cannot be used as keys because they are mutable.

Basic Syntax of a Dictionary:

- Dictionaries are created using **curly braces** {} with key-value pairs separated by a colon :, and each key-value pair is separated by a comma ,.
    - **Syntax**:

```
my_dict = {'key1': 'value1', 'key2': 'value2'}
```

    - **Example**:

```
student = {'name': 'John', 'age': 22, 'major': 'Physics'}
```

Common Operations with Dictionaries:

1. **Accessing Values**:
    - You can access a value in a dictionary using its key inside square brackets [].
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
print(student['name'])  # Output: 'John'
```

2. **Adding/Updating Elements**:
    - You can add a new key-value pair or update the value of an existing key by assigning a value to the key.
    - **Example**:

```
student['graduated'] = True  # Adding a new key-value pair
student['age'] = 23  # Updating an existing key's value
```

3. **Removing Elements**:
    - To remove a key-value pair, use the del statement or the .pop() method.
    - **Example**:

```
del student['major']  # Removes the 'major' key-value pair
```

4. **Using Dictionary Methods**:
    - **get()**: Returns the value for a given key, with an optional default if the key is not found.
    - **keys()**: Returns all the keys in the dictionary.
    - **values()**: Returns all the values in the dictionary.
    - **items()**: Returns all key-value pairs as tuples.

Advantages of Using Dictionaries:

- **Fast Lookups**: You can access values quickly by their keys, making dictionaries efficient for tasks like storing user data, configuration settings, and more.
- **Flexible Data Types**: Keys must be immutable, but values can be any data type, including lists, tuples, or even other dictionaries.

Copyright © 2025 HyperionDev. All rights reserved.

- A **Python dictionary** is a **mutable**, **unordered** (since Python 3.7, ordered by insertion), and **key-value** based data structure.
- **Keys** must be **unique** and **immutable**, while **values** can be any data type.
- **Dictionaries** allow fast lookups, dynamic modifications, and offer many useful methods for working with key-value data.

Dictionaries are highly efficient and versatile for organizing and retrieving data in Python.

8.5 Inbuilt methods (KT0805) (IAC0801)

Python dictionaries come with several **inbuilt methods** that help you work with key-value pairs efficiently. These methods allow you to perform common tasks such as adding, retrieving, and modifying data, as well as more advanced operations like getting all keys or values, or removing items.

1. get() – Safely Retrieve a Value by Key

- **Description**: The get() method returns the value for a given key. If the key does not exist, it returns None (or a default value that you specify) instead of raising an error.
- **Syntax**:

```
dict.get(key, default_value)
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
print(student.get('name'))  # Output: 'Alice'
print(student.get('grade', 'Not Available'))  # Output: 'Not Available'
```

2. keys() – Get All Keys in a Dictionary

Copyright © 2025 HyperionDev. All rights reserved.

- **Description**: The keys() method returns a view object that displays all the keys in the dictionary.
- **Syntax**:

```
dict.keys()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
print(student.keys())  # Output: dict_keys(['name', 'age'])
```

3. values() – Get All Values in a Dictionary

- **Description**: The values() method returns a view object containing all the values in the dictionary.
- **Syntax**:

```
dict.values()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
print(student.values())  # Output: dict_values(['Alice', 20])
```

4. items() – Get All Key-Value Pairs

- **Description**: The items() method returns a view object containing all the key-value pairs in the dictionary as tuples.
- **Syntax**:

```
dict.items()
```

- **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
student = {'name': 'Alice', 'age': 20}
print(student.items())  # Output: dict_items([('name', 'Alice'), ('age', 20)])
```

5. update() – Merge or Update Dictionaries

- **Description**: The update() method updates the dictionary with key-value pairs from another dictionary or an iterable of key-value pairs. If the key already exists, its value is updated; if the key is new, it is added.
- **Syntax**:

```
dict.update(other_dict)
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
student.update({'age': 21, 'major': 'Physics'})  # Updates 'age' and adds 'major'
print(student)  # Output: {'name': 'Alice', 'age': 21, 'major': 'Physics'}
```

6. pop() – Remove a Key-Value Pair and Return Its Value

- **Description**: The pop() method removes a specified key from the dictionary and returns its value. If the key doesn't exist, you can provide a default value to return.
- **Syntax**:

```
dict.pop(key, default_value)
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
age = student.pop('age')  # Removes 'age' and returns its value
print(age)  # Output: 20
print(student)  # Output: {'name': 'Alice'}
```

7. popitem() – Remove and Return the Last Key-Value Pair

Copyright © 2025 HyperionDev. All rights reserved.

- **Description**: The popitem() method removes and returns the **last inserted** key-value pair from the dictionary. This is useful for removing items in insertion order (from Python 3.7+).
- **Syntax**:

```
dict.popitem()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
last_item = student.popitem()  # Removes and returns ('age', 20)
print(last_item)  # Output: ('age', 20)
print(student)  # Output: {'name': 'Alice'}
```

8. clear() – Remove All Key-Value Pairs

- **Description**: The clear() method removes all the elements from the dictionary, leaving it empty.
- **Syntax**:

```
dict.clear()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
student.clear()
print(student)  # Output: {}
```

9. copy() – Create a Shallow Copy of a Dictionary

- **Description**: The copy() method returns a shallow copy of the dictionary. It creates a new dictionary with the same key-value pairs, but does not deeply copy nested objects.
- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
new_dict = dict.copy()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
new_student = student.copy()
print(new_student)  # Output: {'name': 'Alice', 'age': 20}
```

10. fromkeys() – Create a New Dictionary from a Sequence of Keys

- **Description**: The fromkeys() method creates a new dictionary with specified keys and an optional default value.
- **Syntax**:

```
new_dict = dict.fromkeys(sequence_of_keys, default_value)
```

- **Example**:

```
keys = ['name', 'age', 'major']
new_dict = dict.fromkeys(keys, 'unknown')
print(new_dict)  # Output: {'name': 'unknown', 'age': 'unknown', 'major': 'unknown'}
```

- **get()**: Retrieve a value safely by key.
- **keys(), values(), items()**: Get views of the dictionary's keys, values, or key-value pairs.
- **update()**: Add or update multiple key-value pairs.
- **pop(), popitem()**: Remove and return specific key-value pairs.
- **clear()**: Remove all elements.
- **copy()**: Make a shallow copy of the dictionary.
- **fromkeys()**: Create a dictionary from a list of keys.

These inbuilt methods make working with Python dictionaries highly efficient and flexible for managing key-value data.

Copyright © 2025 HyperionDev. All rights reserved.

8.6 Length of Dictionary (KT0806) (IAC0801)

The **length** of a dictionary refers to the **number of key-value pairs** it contains. You can determine the length of a dictionary using the built-in **len()** function.

How to Get the Length of a Dictionary

- The **len()** function, when applied to a dictionary, returns the number of **keys** (i.e., key-value pairs) present in the dictionary.
- **Syntax**:

```
len(dictionary)
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20, 'major': 'Mathematics'}
print(len(student))  # Output: 3
```

In this example, the dictionary student contains three key-value pairs, so the length is 3.

Important Points:

1. **Counts Only Key-Value Pairs**:
    - The **len()** function counts the number of **key-value pairs**, not individual values or keys. Every key-value pair counts as one unit, regardless of the size or type of the values.
2. **Empty Dictionary**:
    - If the dictionary is **empty** (i.e., it contains no key-value pairs), the length will be 0.
    - **Example**:

```
empty_dict = {}
print(len(empty_dict))  # Output: 0
```


Copyright © 2025 HyperionDev. All rights reserved.

- **The length of a dictionary** is the number of key-value pairs it contains.
- Use **len()** to find the number of items in the dictionary.
- An **empty dictionary** has a length of 0.

This is a simple and efficient way to determine the size of your dictionary.

8.7 Dictionary Clear Method (KT0807) (IAC0801)

The **clear()** method in Python is used to **remove all key-value pairs** from a dictionary, leaving it completely **empty**. After using clear(), the dictionary will still exist, but it will have no data inside.

How the clear() Method Works:

- The clear() method **removes all items** from the dictionary, but does not delete the dictionary itself. It simply empties it.
- **Syntax**:

```
dictionary.clear()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20, 'major': 'Physics'}
student.clear()
print(student)  # Output: {}
```

In this example, after calling student.clear(), the dictionary student becomes an empty dictionary {}.

Key Points about clear():

1. **Mutates the Original Dictionary**:

Copyright © 2025 HyperionDev. All rights reserved.

- The clear() method **modifies** the original dictionary, meaning it removes all the items in place. You do not get a new dictionary, but the existing dictionary will be emptied.

2. **No Return Value**:
   - The clear() method **does not return any value**. It modifies the dictionary directly and always returns None.

3. **Dictionary Still Exists**:
   - After calling clear(), the dictionary still exists in memory, but it is empty.
   - **Example**:

```
my_dict = {'a': 1, 'b': 2}
my_dict.clear()
print(my_dict)  # Output: {}
```

4. **Use Case**:
   - The clear() method is useful when you want to **reuse** a dictionary but remove all of its contents. It helps when you need to reset a dictionary to an empty state during a program's execution.

- The **clear()** method **removes all key-value pairs** from a dictionary, leaving it empty.
- It **does not return a value** and modifies the dictionary **in place**.
- After calling clear(), the dictionary still exists but has no data inside.

This method is helpful when you need to reset or empty a dictionary without deleting the variable itself

Copyright © 2025 HyperionDev. All rights reserved.

8.8 Update, Cmp, Len, Sort, Copy, Items, str (KT0808) (IAC0801)

These are the key methods and functions used with Python dictionaries. While not all of these are built-in for dictionaries (like Cmp and Sort), alternatives are available for them as well.

1. update() – Add or Update Key-Value Pairs

- **Description**: The update() method adds new key-value pairs or updates the values of existing keys. If a key already exists, its value is updated; if it's a new key, it is added to the dictionary.
- **Syntax**:

```
dict.update(other_dict)
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
student.update({'age': 21, 'major': 'Physics'})
print(student)  # Output: {'name': 'Alice', 'age': 21, 'major': 'Physics'}
```

2. Cmp (Comparison) – Comparing Dictionaries

Python **does not** have a direct cmp() function for dictionaries, but you can compare dictionaries using equality operators (==, !=). Two dictionaries are considered equal if they have the same keys and values.

- **Example**:

```
dict1 = {'name': 'Alice', 'age': 20}
dict2 = {'name': 'Alice', 'age': 20}
print(dict1 == dict2)  # Output: True
```

If you need to check differences between two dictionaries, you can use a manual comparison.

3. len() – Get the Length of a Dictionary

Copyright © 2025 HyperionDev. All rights reserved.

- **Description**: The len() function returns the **number of key-value pairs** in a dictionary. It counts the keys in the dictionary.
- **Syntax**:

```
len(dict)
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20, 'major': 'Physics'}
print(len(student))  # Output: 3
```

4. Sort – Sorting a Dictionary

Python dictionaries cannot be **sorted directly** because they are unordered collections prior to Python 3.7. However, you can sort the dictionary's keys or values and create a sorted list.

- **Example (Sorting by Keys)**:

```
student = {'name': 'Alice', 'age': 20, 'major': 'Physics'}
sorted_keys = sorted(student)
print(sorted_keys)  # Output: ['age', 'major', 'name']
```

- **Example (Sorting by Values)**:

```
student = {'name': 'Alice', 'age': 20, 'major': 'Physics'}
sorted_by_value = sorted(student.items(), key=lambda item: item[1])
print(sorted_by_value)  # Output: [('age', 20), ('major', 'Physics'), ('name', 'Alice')]
```

5. copy() – Create a Shallow Copy of a Dictionary

- **Description**: The copy() method creates a **shallow copy** of the dictionary. This means it copies the dictionary structure but does not deeply copy any nested objects (e.g., lists or dictionaries inside the dictionary).

Copyright © 2025 HyperionDev. All rights reserved.

- **Syntax**:

```
dict.copy()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
new_student = student.copy()
print(new_student)  # Output: {'name': 'Alice', 'age': 20}
```

6. items() – Get All Key-Value Pairs

- **Description**: The items() method returns a **view object** that contains the dictionary's key-value pairs as tuples.
- **Syntax**:

```
dict.items()
```

- **Example**:

```
student = {'name': 'Alice', 'age': 20}
print(student.items())  # Output: dict_items([('name', 'Alice'), ('age', 20)])
```

7. str() – Convert Dictionary to String

- **Description**: The str() function converts a dictionary into a string representation. This can be useful for printing or logging the dictionary in a readable format.
- **Syntax**:

```
str(dict)
```

- **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
student = {'name': 'Alice', 'age': 20}
print(str(student))  # Output: "{'name': 'Alice', 'age': 20}"
```

- **update()**: Add or modify key-value pairs in the dictionary.
- **Cmp**: Direct dictionary comparison using == or !=.
- **len()**: Get the number of key-value pairs in a dictionary.
- **Sort**: Sort dictionaries by keys or values indirectly using sorted().
- **copy()**: Create a shallow copy of a dictionary.
- **items()**: Get all key-value pairs as tuples.
- **str()**: Convert a dictionary into a string representation.

These methods allow you to manipulate and work with Python dictionaries efficiently.

8.9 Merging dictionaries (KT0809) (IAC0801)

Merging dictionaries is the process of combining the key-value pairs from two or more dictionaries into a **single dictionary**. Python offers several ways to merge dictionaries, depending on your use case and the Python version you're using.

1. Using update() Method

- **Description**: The update() method is used to merge one dictionary into another. It adds key-value pairs from the second dictionary to the first. If a key already exists, its value is **overwritten** by the value from the second dictionary.
- **Syntax**:

```
dict1.update(dict2)
```

- **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
dict1 = {'name': 'Alice', 'age': 20}
dict2 = {'age': 21, 'major': 'Physics'}

dict1.update(dict2)
print(dict1)  # Output: {'name': 'Alice', 'age': 21, 'major': 'Physics'}
```

In this example, the key 'age' exists in both dictionaries, so the value from dict2 (21) overwrites the value from dict1.

2. Using the | Operator (Python 3.9+)

- **Description**: In Python 3.9 and later, you can merge two dictionaries using the **| (pipe)** operator. This creates a **new dictionary** that contains the merged key-value pairs from both dictionaries without modifying the original ones.
- **Syntax**:

```
merged_dict = dict1 | dict2
```

- **Example**:

```
dict1 = {'name': 'Alice', 'age': 20}
dict2 = {'age': 21, 'major': 'Physics'}

merged_dict = dict1 | dict2
print(merged_dict)  # Output: {'name': 'Alice', 'age': 21, 'major': 'Physics'}
```

Here, a new dictionary merged_dict is created by combining the two dictionaries, but dict1 and dict2 remain unchanged.

3. Using Dictionary Unpacking (**)

- **Description**: Another way to merge dictionaries (in Python 3.5+) is by using **dictionary unpacking** (**). This method creates a new dictionary by unpacking the key-value pairs from the dictionaries being merged.

Copyright © 2025 HyperionDev. All rights reserved.

- **Syntax**:

```
merged_dict = {**dict1, **dict2}
```

- **Example**:

```
dict1 = {'name': 'Alice', 'age': 20}
dict2 = {'age': 21, 'major': 'Physics'}

merged_dict = {**dict1, **dict2}
print(merged_dict)  # Output: {'name': 'Alice', 'age': 21, 'major': 'Physics'}
```

The ** operator unpacks the key-value pairs from both dictionaries and merges them into a new dictionary.

4. Using collections.ChainMap

- **Description**: The collections.ChainMap class allows you to group multiple dictionaries together and view them as a single dictionary. However, unlike the other methods, this **does not merge** the dictionaries into one; instead, it creates a **view** over them. Modifications are reflected in the original dictionaries.

- **Syntax**:

```
from collections import ChainMap
merged = ChainMap(dict1, dict2)
```

- **Example**:

```
from collections import ChainMap

dict1 = {'name': 'Alice', 'age': 20}
dict2 = {'age': 21, 'major': 'Physics'}

merged = ChainMap(dict1, dict2)
print(merged)  # Output: ChainMap({'name': 'Alice', 'age': 20}, {'age': 21, 'major': 'Ph
```


Copyright © 2025 HyperionDev. All rights reserved.

The key-value pairs are **not merged** into a single dictionary but are combined in a view. Accessing values will prioritize dict1 first.

Handling Key Conflicts During Merging:

- If the same **key** exists in both dictionaries, the value from the dictionary merged **last** (second dictionary) will overwrite the value from the first dictionary.
    - Example:

```
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
merged_dict = {**dict1, **dict2}
print(merged_dict)  # Output: {'a': 1, 'b': 3, 'c': 4}
```

- **update()**: Merges one dictionary into another, modifying the first dictionary in place.
- **| Operator**: Available in Python 3.9+, merges dictionaries and creates a new one without modifying the originals.
- **Unpacking**: Merges dictionaries by unpacking them into a new dictionary.
- **ChainMap**: Combines multiple dictionaries into a single view without merging them.

These methods provide flexibility in how you merge dictionaries depending on whether you want to modify an existing dictionary or create a new one.



Formative Assessment Activity [8]

Dictionary in Python

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT09:

Topic Code	KM-02-KT09:
Topic	Python sets
Weight	5%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT0901)
- Syntax (KT0902)
- Parameters (KT0903)
- Methods (KT0904)
- Operations performed on sets in Python (KT0905)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC0901 Definitions, functions and features of Python sets are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

9.1 Concept, definition and functions (KT0901) (IAC0901)

A **set** in Python is an **unordered**, **mutable**, and **unindexed** collection of **unique elements**. This means that sets cannot have duplicate values, and the order of elements is not maintained. Sets are often used when you need to store unique items and perform operations like union, intersection, and difference.

- **Syntax**:

```
my_set = {1, 2, 3}
```

You can also create an empty set using the set() constructor, as {} is used to define an empty dictionary in Python.

- **Empty Set**:

```
empty_set = set()
```

Functions of Python Sets:

1. **Storing Unique Elements**:
    - The primary function of a set is to store **unique elements**. Sets automatically remove duplicates when initialized or updated with new values.
    - **Example**:

```
my_set = {1, 2, 2, 3}
print(my_set)  # Output: {1, 2, 3}
```

2. **Set Operations**:
    - Python sets support **mathematical set operations** like **union**, **intersection**, **difference**, and **symmetric difference**.
    - These operations are useful when working with distinct data sets, such as comparing two groups of data or eliminating duplicates.

Copyright © 2025 HyperionDev. All rights reserved.

Key Features of Python Sets:

1. **Unordered and Unindexed**:
    - Sets do not maintain the order of elements, and elements do not have a specific index. You cannot access set elements by an index.
    - **Example**:

```
my_set = {5, 3, 1, 4}
print(my_set)  # Output: {1, 3, 4, 5} (order is not guaranteed)
```

2. **Mutable**:
    - Sets are **mutable**, meaning you can add or remove elements after creation using methods like add(), remove(), or discard().
    - **Example**:

```
my_set = {1, 2, 3}
my_set.add(4)  # Adding an element
my_set.remove(2)  # Removing an element
print(my_set)  # Output: {1, 3, 4}
```

3. **Immutable Subtype: frozenset**:
    - Python also provides an **immutable** version of a set called **frozenset**. Once created, elements cannot be added or removed from a frozenset.
    - **Example**:

```
frozen = frozenset([1, 2, 3])
```


Copyright © 2025 HyperionDev. All rights reserved.

4. **No Duplicates**:
    - Sets automatically eliminate duplicate elements. This makes them ideal for operations where uniqueness is important.
    - **Example**:

```
my_set = {1, 1, 2, 3, 3}
print(my_set)  # Output: {1, 2, 3}
```

5. **Mathematical Set Operations**:
    - **Union (|)**: Combines all elements from two sets (removes duplicates).

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1 | set2)  # Output: {1, 2, 3, 4, 5}
```

    - **Intersection (&)**: Returns elements common to both sets.

```
print(set1 & set2)  # Output: {3}
```

    - **Difference (-)**: Returns elements in the first set but not in the second.

```
print(set1 - set2)  # Output: {1, 2}
```

    - **Symmetric Difference (^)**: Returns elements in either set but not in both.

```
print(set1 ^ set2)  # Output: {1, 2, 4, 5}
```

6. **Membership Testing**:
    - Sets allow you to quickly test if an element is in the set using the **in** keyword.
    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```python
my_set = {1, 2, 3}
print(2 in my_set)  # Output: True
print(4 in my_set)  # Output: False
```

- **Concept**: A set is an **unordered collection** of **unique** elements in Python. Duplicate items are automatically removed, and sets are commonly used for mathematical operations on distinct data sets.
- **Function**: Sets allow efficient storage of unique elements and provide operations for set mathematics like union, intersection, and difference.
- **Features**:
    - **Unordered and unindexed**: No guaranteed order or indexing.
    - **Mutable**: Elements can be added or removed.
    - **No duplicates**: Automatically ensures uniqueness.
    - Supports **set operations** like union, intersection, and difference.
    - **Membership testing**: Quick checks for element existence.

Python sets are a powerful and efficient way to handle unique elements and perform mathematical set operations.

9.2 Syntax (KT0902) (IAC0901)

A **set** in Python is a collection of **unordered**, **unindexed**, and **unique elements**. Sets are defined using curly braces {} or the set() function. They are useful when you want to store non-duplicate elements and perform operations like union, intersection, and difference.

Basic Syntax for Creating a Set:

1. **Using Curly Braces {}**:

Copyright © 2025 HyperionDev. All rights reserved.

- You can create a set by enclosing elements in curly braces, separated by commas.
- **Example**:

```
my_set = {1, 2, 3, 4}
print(my_set)  # Output: {1, 2, 3, 4}
```

2. **Using the set() Function**:
   - You can also create a set using the set() constructor. This is particularly useful when you need to convert other iterables like lists or strings into sets.
   - **Example**:

```
my_set = set([1, 2, 3, 4])
print(my_set)  # Output: {1, 2, 3, 4}
```

3. **Empty Set**:
   - To create an empty set, you must use the set() function. If you use {}, it creates an empty dictionary instead.
   - **Example**:

```
empty_set = set()
print(empty_set)  # Output: set()
```

Adding and Removing Elements:

- **Adding Elements**:
  - You can add elements to a set using the **add()** method.
  - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_set = {1, 2}
my_set.add(3)
print(my_set)  # Output: {1, 2, 3}
```

- **Removing Elements**:
    - You can remove elements from a set using the **remove()** or **discard()** methods. The difference is that remove() raises an error if the element is not found, while discard() does not.
    - **Example (remove)**:

```
my_set = {1, 2, 3}
my_set.remove(2)
print(my_set)  # Output: {1, 3}
```

    - **Example (discard)**:

```
my_set = {1, 2, 3}
my_set.discard(5)  # No error if 5 is not in the set
print(my_set)  # Output: {1, 2, 3}
```

Set Operations Syntax:

- **Union (|)**:
    - Combines elements from two sets, removing duplicates.
    - **Example**:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1 | set2)  # Output: {1, 2, 3, 4, 5}
```

- **Intersection (&)**:
    - Returns the elements common to both sets.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
print(set1 & set2)  # Output: {3}
```

- **Difference (-)**:
  - Returns elements in the first set but not in the second set.
  - **Example**:

```
print(set1 - set2)  # Output: {1, 2}
```

- **Symmetric Difference (^)**:
  - Returns elements in either set but not in both.
  - **Example**:

```
print(set1 ^ set2)  # Output: {1, 2, 4, 5}
```

Checking Membership:

- You can check whether an element is present in a set using the **in** keyword.
- **Example**:

```
my_set = {1, 2, 3}
print(2 in my_set)  # Output: True
print(5 in my_set)  # Output: False
```

- **Creating a Set**: Use curly braces {} or the set() function.
- **Adding Elements**: Use the add() method.
- **Removing Elements**: Use remove() or discard().
- **Set Operations**: Perform union (|), intersection (&), difference (-), and symmetric difference (^) on sets.

Copyright © 2025 HyperionDev. All rights reserved.

- **Membership Testing**: Use the in keyword to check if an element exists in a set.

Python sets are easy to use and ideal for situations where you need **unique, unordered collections.**

9.3 Parameters (KT0903) (IAC0901)

Sets are a built-in data structure that can hold an **unordered** collection of **unique elements**. When working with sets, the concept of "parameters" comes into play when you create a set or perform operations on a set.

1. Parameters for Creating a Set

When creating a set using the set() function, the primary parameter is an **iterable**. An iterable is any Python object that can return its elements one at a time (e.g., a list, tuple, string, etc.).

- **Syntax**:

```
set(iterable)
```

- **Parameter**:
    - **iterable**: Any iterable data type like a list, tuple, string, or another set. This is optional, and if omitted, an empty set is created.
- **Examples**:
    - **Creating a set from a list**:

```
my_set = set([1, 2, 3, 4])
print(my_set)  # Output: {1, 2, 3, 4}
```

    - **Creating a set from a string**:

Copyright © 2025 HyperionDev. All rights reserved.

```
my_set = set("hello")
print(my_set)  # Output: {'h', 'e', 'l', 'o'}  # Note: Only unique characters are
```

- **Creating an empty set**:

```
empty_set = set()
print(empty_set)  # Output: set()
```

2. Parameters for Set Methods

Many set methods accept parameters to manipulate or perform operations on sets. Let's explore some commonly used set methods and their parameters:

a) add() Method:

- **Description**: Adds an element to the set.
- **Syntax**:

```
set.add(element)
```

- **Parameter**:
  - **element**: The element to add to the set. This can be of any immutable type (e.g., numbers, strings, tuples).
- **Example**:

```
my_set = {1, 2, 3}
my_set.add(4)
print(my_set)  # Output: {1, 2, 3, 4}
```

b) remove() and discard() Methods:

- **Description**: Remove a specific element from the set.
  - remove() raises an error if the element is not found.
  - discard() does not raise an error if the element is not found.

Copyright © 2025 HyperionDev. All rights reserved.

- **Syntax**:

```
set.remove(element)
set.discard(element)
```

- **Parameter**:
    - **element**: The element to remove from the set.
- **Example**:

```
my_set = {1, 2, 3}
my_set.remove(2)  # Removes 2
my_set.discard(4)  # No error, even though 4 is not in the set
```

c) union() Method:

- **Description**: Returns a new set containing all elements from both sets (removes duplicates).
- **Syntax**:

```
set1.union(set2)
```

- **Parameter**:
    - **set2**: The second set whose elements are combined with the first set.
- **Example**:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
result = set1.union(set2)
print(result)  # Output: {1, 2, 3, 4, 5}
```

d) intersection() Method:

- **Description**: Returns a new set containing only the elements that are common to both sets.
- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
set1.intersection(set2)
```

- **Parameter**:
    - **set2**: The set to compare with and find the common elements.
- **Example**:

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
result = set1.intersection(set2)
print(result)  # Output: {2, 3}
```

e) difference() Method:

- **Description**: Returns a new set containing elements that are in the first set but not in the second.
- **Syntax**:

```
set1.difference(set2)
```

- **Parameter**:
    - **set2**: The set whose elements will be excluded from the result.
- **Example**:

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
result = set1.difference(set2)
print(result)  # Output: {1}
```

f) symmetric_difference() Method:

- **Description**: Returns a new set containing elements that are in either set, but not in both.
- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
set1.symmetric_difference(set2)
```

- **Parameter**:
    - **set2**: The set to compare with for symmetric difference.
- **Example**:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
result = set1.symmetric_difference(set2)
print(result)  # Output: {1, 2, 4, 5}
```

Set Parameters:

- **Creating a Set**: The set() function accepts an **iterable** parameter like a list, string, or another set.
- **Common Methods**:
    - **add()**: Adds an element to the set.
    - **remove()/discard()**: Removes a specific element.
    - **union(), intersection(), difference(), symmetric_difference()**: Perform set operations by taking another set as a parameter.

These parameters help you perform various operations with sets, making them a versatile tool for managing unique collections in Python.

9.4 Methods (KT0904) (IAC0901)

Python **sets** are collections of **unordered**, **unique** elements, and they come with several built-in methods that help in performing common set operations, such as adding elements, removing elements, and performing set theory operations like union, intersection, and difference.

Copyright © 2025 HyperionDev. All rights reserved.

1. add() – Add an Element

- **Description**: Adds an element to the set.
- **Example**:

```
my_set = {1, 2, 3}
my_set.add(4)  # Adds 4 to the set
```

2. remove() and discard() – Remove an Element

- **remove()**: Removes a specific element. Raises a **KeyError** if the element is not found.
- **discard()**: Removes an element, but **does not raise an error** if the element is missing.
- **Example**:

```
my_set.remove(2)  # Removes 2
my_set.discard(5)  # No error if 5 is not in the set
```

3. pop() – Remove and Return an Arbitrary Element

- **Description**: Removes and returns a random element from the set. Raises a **KeyError** if the set is empty.
- **Example**:

```
my_set.pop()  # Removes and returns an arbitrary element
```

4. clear() – Remove All Elements

- **Description**: Removes all elements from the set, making it empty.
- **Example**:

```
my_set.clear()  # Clears all elements from the set
```


Copyright © 2025 HyperionDev. All rights reserved.

5. union() – Combine Sets (Union)

- **Description**: Returns a new set that contains all elements from two or more sets (duplicates are removed).
- **Example**:

```
set1.union(set2)  # Combines elements from both sets
```

6. intersection() – Common Elements (Intersection)

- **Description**: Returns a new set containing elements that are **common** to both sets.
- **Example**:

```
set1.intersection(set2)  # Elements common to both sets
```

7. difference() – Elements Only in One Set (Difference)

- **Description**: Returns a new set containing elements that are in the first set but **not in the second**.
- **Example**:

```
set1.difference(set2)  # Elements in set1 but not in set2
```

8. symmetric_difference() – Elements in Either Set but Not Both

- **Description**: Returns a new set containing elements that are in either set but **not in both**.
- **Example**:

```
set1.symmetric_difference(set2)  # Elements in either set1 or set2, but not both
```

9. issubset() – Check Subset

Copyright © 2025 HyperionDev. All rights reserved.

- **Description**: Returns **True** if all elements of one set are present in another set.
- **Example**:

```
set1.issubset(set2)  # Checks if set1 is a subset of set2
```

10. issuperset() – Check Superset

- **Description**: Returns **True** if all elements of another set are present in the first set.
- **Example**:

```
set1.issuperset(set2)  # Checks if set1 is a superset of set2
```

Key Methods:

- **add()**: Add an element to the set.
- **remove()/discard()**: Remove an element, with remove() raising an error if the element is not found.
- **pop()**: Remove and return an arbitrary element.
- **clear()**: Remove all elements from the set.
- **union(), intersection(), difference(), symmetric_difference()**: Perform mathematical set operations.
- **issubset()/issuperset()**: Check if a set is a subset or superset of another set.

These methods allow you to efficiently manage sets and perform mathematical operations in Python. Sets are particularly useful for ensuring uniqueness and performing fast membership checks.

Copyright © 2025 HyperionDev. All rights reserved.

9.5 Operations performed on sets in Python (KT0905) (IAC0901)

Python **sets** are collections of **unordered, unique elements**. They provide a variety of operations to manipulate and compare sets, especially for tasks involving **mathematical set theory**. These operations allow you to find common elements, combine sets, and compare relationships between them.

1. Union (| or union()) – Combine Two Sets

- **Description**: The **union** operation returns a new set containing all the elements from both sets. **Duplicates are removed**.
- **Syntax**:

```
set1 | set2
set1.union(set2)
```

- **Example**:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
result = set1 | set2  # Output: {1, 2, 3, 4, 5}
```

2. Intersection (& or intersection()) – Common Elements

- **Description**: The **intersection** operation returns a new set containing only the elements that are **common** to both sets.
- **Syntax**:

```
set1 & set2
set1.intersection(set2)
```

- **Example**:

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
result = set1 & set2  # Output: {2, 3}
```

3. Difference (- or difference()) – Elements in One Set but Not the Other

Copyright © 2025 HyperionDev. All rights reserved.

- **Description**: The **difference** operation returns a new set containing elements that are in the first set but **not in the second**.
- **Syntax**:

```
set1 - set2
set1.difference(set2)
```

- **Example**:

```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
result = set1 - set2  # Output: {1}
```

4. Symmetric Difference (^ or symmetric_difference()) – Elements in Either Set but Not Both

- **Description**: The **symmetric difference** operation returns a new set containing elements that are in **either** of the two sets, but **not in both**.
- **Syntax**:

```
set1 ^ set2
set1.symmetric_difference(set2)
```

- **Example**:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
result = set1 ^ set2  # Output: {1, 2, 4, 5}
```

5. Subset (<= or issubset()) – Check if One Set is a Subset of Another

- **Description**: The **subset** operation checks if all elements of one set are contained in another set. Returns **True** if the first set is a subset.
- **Syntax**:

```
set1 <= set2
set1.issubset(set2)
```

- **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
set1 = {1, 2}
set2 = {1, 2, 3}
result = set1 <= set2  # Output: True
```

6. Superset (>= or issuperset()) – Check if One Set is a Superset of Another

- **Description**: The **superset** operation checks if all elements of another set are contained in the first set. Returns **True** if the first set is a superset.
- **Syntax**:

```
set1 >= set2
set1.issuperset(set2)
```

- **Example**:

```
set1 = {1, 2, 3}
set2 = {1, 2}
result = set1 >= set2  # Output: True
```

7. Disjoint (isdisjoint()) – Check if Two Sets Have No Common Elements

- **Description**: The **disjoint** operation checks if two sets have no elements in common. Returns **True** if the sets are disjoint.
- **Syntax**:

```
set1.isdisjoint(set2)
```

- **Example**:

```
set1 = {1, 2, 3}
set2 = {4, 5, 6}
result = set1.isdisjoint(set2)  # Output: True
```

8. Membership Test (in, not in) – Check if an Element is in a Set

- **Description**: The **membership test** checks if an element is present in a set using in or not in.

Copyright © 2025 HyperionDev. All rights reserved.

- **Syntax**:

```
element in set
element not in set
```

- **Example**:

```
my_set = {1, 2, 3}
print(2 in my_set)  # Output: True
print(4 not in my_set)  # Output: True
```

Key Set Operations:

- **Union (|)**: Combines all elements from both sets.
- **Intersection (&)**: Finds common elements between two sets.
- **Difference (-)**: Finds elements in one set but not the other.
- **Symmetric Difference (^)**: Finds elements in either set but not both.
- **Subset (<=)**: Checks if one set is a subset of another.
- **Superset (>=)**: Checks if one set is a superset of another.
- **Disjoint (isdisjoint())**: Checks if two sets have no common elements.
- **Membership (in, not in)**: Checks if an element is present in the set.

These operations are useful for tasks involving **comparison** and **manipulation** of sets, especially in situations where you need to handle unique elements or perform mathematical set operations.



Formative Assessment Activity [9]

Python Sets

Complete the formative activity in your **KM2 Learner Workbook**,

Copyright © 2025 HyperionDev. All rights reserved.

Knowledge Topic KM-02-KT10:

Topic Code	KM-02-KT10:
Topic	Arrays in Python
Weight	10%

This knowledge topic will cover the following topic elements:

- Concept, definition and functions (KT1001)
- Syntax to create an array (KT1002)
- Python module (KT1003)
- Methods (KT1004)
- Array representation (KT1005)
- Array operations (KT1006)

After working through this knowledge topic, your competence will be assessed according to the following criterion/criteria:

- IAC1001 Definitions, functions and features of Python arrays are understood and explained

Copyright © 2025 HyperionDev. All rights reserved.

10.1 Concept, definition and functions (KT1001) (IAC1001)

An **array** in Python is a **collection of elements** that are **ordered** and typically **homogeneous**, meaning they are usually of the same data type. Arrays allow you to store and manipulate multiple values efficiently, especially when you need to work with large amounts of numerical data or when performance is critical.

In Python, arrays are mostly used via the **array module** or the more powerful **NumPy library** (for scientific computing).

Definition of Python Arrays:

A Python **array** is a **list-like data structure** that can hold elements of the same data type. Unlike Python **lists**, which can hold elements of mixed data types, arrays are typically used to store only one type of data (e.g., all integers or all floats).

- **Array Module**: The built-in array module allows you to create arrays in Python with elements of the same type.
    - **Example**:

```
import array
arr = array.array('i', [1, 2, 3, 4])
print(arr)  # Output: array('i', [1, 2, 3, 4])
```

Here, 'i' represents an array of integers, and the elements are [1, 2, 3, 4].

Functions and Features of Python Arrays:

1. **Efficient Storage**:
    - Arrays are memory-efficient because they store elements of the same type, which reduces overhead. This is particularly useful for handling large datasets where performance matters.
2. **Type Code**:

Copyright © 2025 HyperionDev. All rights reserved.

- Arrays in Python use a **type code** to specify the data type of the elements. For example, 'i' is used for integers, and 'f' is used for floats. All elements in an array must conform to the specified type.
- **Example**:

```
import array
arr = array.array('f', [1.0, 2.5, 3.2])
```

3. **Accessing Elements by Index**:
   - You can access elements in an array by using an **index** (just like lists). Indexing starts at 0.
   - **Example**:

```
arr = array.array('i', [10, 20, 30])
print(arr[1])  # Output: 20
```

4. **Modifying Elements**:
   - You can **update** or **replace** elements in an array by specifying the index.
   - **Example**:

```
arr[0] = 100
print(arr)  # Output: array('i', [100, 20, 30])
```

5. **Common Array Methods**:
   - Arrays come with several built-in methods to manipulate their contents:
     - **append()**: Add an element to the end of the array.

```
arr.append(40)
print(arr)  # Output: array('i', [100, 20, 30, 40])
```


Copyright © 2025 HyperionDev. All rights reserved.

- **pop()**: Remove and return an element from a specific index.

```
arr.pop(2)
print(arr)  # Output: array('i', [100, 20, 40])
```

- **extend()**: Add multiple elements from another iterable.

```
arr.extend([50, 60])
print(arr)  # Output: array('i', [100, 20, 40, 50, 60])
```

6. **Slicing**:
    - Just like lists, arrays support **slicing**, which allows you to retrieve a subset of elements from the array.
    - **Example**:

```
arr = array.array('i', [1, 2, 3, 4, 5])
sliced_arr = arr[1:4]
print(sliced_arr)  # Output: array('i', [2, 3, 4])
```

7. **Iterating Over Arrays**:
    - You can **iterate** over the elements in an array using loops.
    - **Example**:

```
for element in arr:
    print(element)
```

8. **Advantages of Using Arrays**:
    - Arrays are more **efficient** than lists when working with large datasets of the same type.

Copyright © 2025 HyperionDev. All rights reserved.

- They provide **better performance** in terms of memory and processing speed, especially for numerical operations.
- Arrays can be easily **integrated** with libraries like **NumPy** for advanced mathematical operations and scientific computing.

Key Differences Between Arrays and Lists:

- **Arrays** are used when you need to store large volumes of **same-type data**, while **lists** can store elements of **mixed types**.
- **Arrays** provide better **performance** for numerical and scientific computing tasks, while **lists** offer more flexibility for general-purpose programming.

- **Concept**: An array is a collection of **ordered, homogeneous elements** that is optimized for performance and memory efficiency.
- **Definition**: Arrays in Python are created using the array module or through libraries like NumPy, and they require all elements to be of the same data type.
- **Functions**: Arrays support operations like **indexing**, **slicing**, **adding**, **removing**, and **iterating** over elements, making them ideal for handling large datasets efficiently.

Arrays are particularly useful when working with numerical data, especially in scientific computing, where performance and memory optimization are crucial.

10.2 Syntax to create an array (KT1002) (IAC1001)

In Python, arrays are typically created using the **array** module. Arrays store multiple elements of the **same data type**, such as integers or floats, and provide efficient memory usage, making them useful for large datasets or numerical computations.

1. Import the array Module

To work with arrays in Python, you first need to import the **array** module.

- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
import array
```

2. Syntax to Create an Array

- **Syntax**:

```
array_name = array.array(typecode, [elements])
```

- **Explanation**:
    - **array**: The array module.
    - **typecode**: A character that defines the type of elements the array will hold. For example:
        - 'i' for integers.
        - 'f' for floating-point numbers.
    - **[elements]**: The list of elements to initialize the array.

3. Example of Creating an Array

Creating an Integer Array:

- **Syntax**:

```
arr = array.array('i', [1, 2, 3, 4, 5])
```

    - 'i': Indicates that the array will store integers.
    - [1, 2, 3, 4, 5]: The initial elements of the array.

Creating a Float Array:

- **Syntax**:

```
arr = array.array('f', [1.1, 2.2, 3.3])
```

    - 'f': Indicates that the array will store floating-point numbers.

Copyright © 2025 HyperionDev. All rights reserved.

- [1.1, 2.2, 3.3]: The initial elements of the array.

4. Creating an Empty Array

You can also create an empty array, which can later be populated with elements.

- **Syntax**:

```
empty_arr = array.array('i')  # Empty integer array
```

Common Typecodes:

- 'i': Signed integers (e.g., -1, 2, 3).
- 'f': Floating-point numbers (e.g., 1.0, 2.5, 3.3).
- 'd': Double precision floats (larger floating-point numbers).
- 'b': Signed integers (1 byte).

- **To create an array in Python**:
  - Import the array module.
  - Use the array() function with the appropriate **typecode** and **elements**.
- **Example**:

```
import array
arr = array.array('i', [1, 2, 3, 4, 5])  # Integer array
```

Arrays are ideal for efficiently handling large datasets where all elements are of the same type.

10.3 Python module (KT1003) (IAC1001)

The **array module** provides support for creating and working with **arrays**, which are collections of **ordered**, **homogeneous** elements (all elements are of the same type). Arrays are more memory-efficient than lists, making them useful when dealing with

Copyright © 2025 HyperionDev. All rights reserved.

large datasets, especially for numerical data. The array module is part of Python's standard library, so it doesn't require any additional installation.

Key Features of the array Module:

1. **Homogeneous Data**:
    - Arrays created with the array module can only hold elements of the same **data type**. This makes them different from Python **lists**, which can store mixed data types.
2. **Typecode**:
    - Arrays in Python are defined by a **typecode**, a character that indicates the type of data the array will hold (e.g., integers, floats).
        - 'i' for signed integers.
        - 'f' for floating-point numbers.
    - This ensures that all elements in the array conform to the specified data type.
3. **Memory-Efficient**:
    - Arrays are more **memory-efficient** than lists because they store data in a more compact form. They are particularly useful when you need to handle large collections of numbers.
4. **Array Module Operations**:
    - Arrays support a variety of operations like lists, including indexing, slicing, adding elements, removing elements, and iteration.

How to Use the array Module

1. Import the array Module:

- To create arrays, you must first **import** the array module.
- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
import array
```

2. Create an Array:

- Use the **array()** function to create an array, specifying the typecode and the initial elements.
- **Syntax**:

```
arr = array.array(typecode, [elements])
```

- **Example** (integer array):

```
arr = array.array('i', [1, 2, 3, 4])
print(arr)  # Output: array('i', [1, 2, 3, 4])
```

Common Typecodes in the array Module:

- **'i'**: Signed integers.
- **'f'**: Floating-point numbers.
- **'d'**: Double-precision floating-point numbers.
- **'b'**: Signed integers (1 byte).

Each typecode ensures that all elements in the array adhere to the specified data type.

Basic Operations with Arrays:

1. **Accessing Elements by Index**:
    - Just like lists, you can access array elements using their index.
    - **Example**:

```
print(arr[1])  # Output: 2 (accessing the second element)
```

2. **Modifying Elements**:

Copyright © 2025 HyperionDev. All rights reserved.

- You can modify elements in the array using their index.
- **Example**:

```
arr[0] = 10
print(arr)  # Output: array('i', [10, 2, 3, 4])
```

3. **Adding Elements**:
   - Use **append()** to add a single element or **extend()** to add multiple elements to the array.
   - **Example**:

```
arr.append(5)  # Adds 5 to the end of the array
arr.extend([6, 7])  # Adds multiple elements
```

4. **Removing Elements**:
   - Use **pop()** or **remove()** to delete elements from the array.
   - **Example**:

```
arr.pop(2)  # Removes the element at index 2
arr.remove(10)  # Removes the element with the value 10
```

5. **Slicing**:
   - You can slice arrays just like lists to extract subsets of elements.
   - **Example**:

```
print(arr[1:3])  # Output: array('i', [2, 3])
```

When to Use the array Module:

- **Numerical Data**: Arrays are ideal when working with large sets of numerical data (e.g., scientific computations, simulations) where all elements are of the same type.

Copyright © 2025 HyperionDev. All rights reserved.

- **Performance**: Arrays offer better performance and memory efficiency compared to lists when dealing with homogeneous data types.
- The **array module** provides the ability to create arrays, which are collections of **homogeneous elements**.
- Arrays are defined by a **typecode** that ensures all elements are of the same data type (e.g., 'i' for integers, 'f' for floats).
- **Arrays** are more **memory-efficient** than lists and are particularly useful for numerical computations and large datasets.
- Arrays support operations like **indexing**, **slicing**, **adding**, and **removing** elements.

This makes arrays a great choice when you need to work with **large collections of numbers** in an efficient manner.

10.4 Methods (KT1004) (IAC1001)

arrays are collections of **homogeneous elements** (i.e., elements of the same type). They are created using the **array** module and come with several useful **methods** that allow you to manipulate the array by adding, removing, or modifying elements.

1. append() – Add an Element to the End of the Array

- **Description**: Adds a single element to the end of the array.
- **Syntax**:

```
array_name.append(element)
```

- **Example**:

```
import array
arr = array.array('i', [1, 2, 3])
arr.append(4)
print(arr)  # Output: array('i', [1, 2, 3, 4])
```


Copyright © 2025 HyperionDev. All rights reserved.

2. extend() – Add Multiple Elements to the Array

- **Description**: Adds multiple elements from an iterable (e.g., a list or another array) to the array.
- **Syntax**:

```
array_name.extend(iterable)
```

- **Example**:

```
arr.extend([5, 6])
print(arr)  # Output: array('i', [1, 2, 3, 4, 5, 6])
```

3. insert() – Insert an Element at a Specific Position

- **Description**: Inserts an element at a specified index in the array.
- **Syntax**:

```
array_name.insert(index, element)
```

- **Example**:

```
arr.insert(1, 10)
print(arr)  # Output: array('i', [1, 10, 2, 3, 4, 5, 6])
```

4. pop() – Remove and Return an Element from a Specific Index

- **Description**: Removes and returns the element at the specified index. If no index is provided, it removes the last element.
- **Syntax**:

```
array_name.pop(index)
```

- **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
arr.pop(2)
print(arr)  # Output: array('i', [1, 10, 3, 4, 5, 6])
```

5. remove() – Remove the First Occurrence of an Element

- **Description**: Removes the first occurrence of the specified element from the array.
- **Syntax**:

```
array_name.remove(element)
```

- **Example**:

```
arr.remove(10)
print(arr)  # Output: array('i', [1, 3, 4, 5, 6])
```

6. index() – Find the Index of the First Occurrence of an Element

- **Description**: Returns the index of the first occurrence of the specified element in the array.
- **Syntax**:

```
array_name.index(element)
```

- **Example**:

```
idx = arr.index(4)
print(idx)  # Output: 2 (index of element 4)
```

7. reverse() – Reverse the Order of the Elements in the Array

- **Description**: Reverses the elements in the array in place.
- **Syntax**:

Copyright © 2025 HyperionDev. All rights reserved.

```
array_name.reverse()
```

- **Example**:

```
arr.reverse()
print(arr)  # Output: array('i', [6, 5, 4, 3, 1])
```

8. buffer_info() – Get Information About the Array's Memory

- **Description**: Returns a tuple containing the memory address of the array and the number of elements in the array.
- **Syntax**:

```
array_name.buffer_info()
```

- **Example**:

```
info = arr.buffer_info()
print(info)  # Output: (memory_address, number_of_elements)
```

9. count() – Count the Number of Occurrences of an Element

- **Description**: Returns the number of occurrences of the specified element in the array.
- **Syntax**:

```
array_name.count(element)
```

- **Example**:

```
count = arr.count(3)
print(count)  # Output: 1 (number of times 3 appears in the array)
```


Copyright © 2025 HyperionDev. All rights reserved.

10. tolist() – Convert the Array to a List

- **Description**: Converts the array into a Python list, which can be useful when you need to work with list-specific methods.
- **Syntax**:

```
array_name.tolist()
```

- **Example**:

```
lst = arr.tolist()
print(lst)  # Output: [6, 5, 4, 3, 1]
```

Array Methods:

- **append()**: Adds a single element to the end of the array.
- **extend()**: Adds multiple elements from an iterable to the array.
- **insert()**: Inserts an element at a specific position.
- **pop()**: Removes and returns an element from a specific index.
- **remove()**: Removes the first occurrence of a specific element.
- **index()**: Returns the index of the first occurrence of an element.
- **reverse()**: Reverses the order of elements in the array.
- **buffer_info()**: Returns the memory address and size of the array.
- **count()**: Counts the occurrences of an element in the array.
- **tolist()**: Converts the array to a list.

These methods provide a variety of operations that help you manipulate arrays efficiently, making them a powerful tool for working with homogeneous data in Python.

Copyright © 2025 HyperionDev. All rights reserved.

10.5 Array representation (KT1005) (IAC1001)

Arrays are represented using the **array** module or the more advanced **NumPy** library (for numerical computing). The array module allows you to create and manipulate **homogeneous arrays**, which means that all elements in an array must be of the same data type.

1. Importing the Array Module

Before working with arrays, you must import the **array** module, which provides support for creating arrays in Python.

- **Syntax**:

```
import array
```

2. Basic Array Representation Syntax

To create an array, use the **array()** function, specifying the **typecode** and the initial **elements**. The typecode indicates the data type of the elements in the array.

- **Syntax**:

```
array_name = array.array(typecode, [elements])
```

- **Typecodes**:
    - 'i': Signed integer.
    - 'f': Floating-point number.
    - 'd': Double-precision floating-point number.
- **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
import array
arr = array.array('i', [1, 2, 3, 4])
print(arr)  # Output: array('i', [1, 2, 3, 4])
```

3. Typecode Representation

The **typecode** is a single character that defines the data type of the elements in the array. Some common typecodes include:

- 'i': **Signed integers**.
- 'f': **Floating-point** numbers.
- 'd': **Double precision floats** (more precision than 'f').
- 'b': **Signed integers** (1 byte).

The typecode ensures that all elements in the array conform to the specified data type.

- **Example** (float array):

```
arr = array.array('f', [1.0, 2.5, 3.2])
print(arr)  # Output: array('f', [1.0, 2.5, 3.2])
```

4. Memory-Efficient Representation

Arrays in Python are **memory-efficient** compared to lists because they store data in a more compact form, especially when dealing with large datasets of a single type. Each element is stored in contiguous memory locations, which allows for faster access and manipulation.

- **Example**:

```
arr = array.array('i', [1, 2, 3, 4])
print(arr.buffer_info())  # Output: (memory_address, number_of_elements)
```

5. Accessing Elements in an Array

You can access individual elements in an array by **indexing**, just like with Python lists. Indexing starts at **0**, meaning the first element has index 0.

Copyright © 2025 HyperionDev. All rights reserved.

- **Example**:

```
arr = array.array('i', [1, 2, 3, 4])
print(arr[0])  # Output: 1 (first element)
```

6. Slicing in Arrays

Arrays also support **slicing**, which allows you to retrieve a portion of the array. The syntax for slicing is like lists.

- **Example**:

```
arr = array.array('i', [1, 2, 3, 4, 5])
sliced_arr = arr[1:4]  # Get elements from index 1 to 3
print(sliced_arr)  # Output: array('i', [2, 3, 4])
```

7. Array Methods

Arrays support a variety of **methods** for manipulating the array's contents, including append(), remove(), pop(), and extend().

- **Example**:

```
arr.append(6)  # Add element 6 to the end
arr.remove(2)  # Remove the first occurrence of element 2
```

- **Array Representation** in Python is done using the **array** module, which allows you to create **homogeneous arrays**.
- Arrays are defined by their **typecode**, which specifies the data type of their elements (e.g., 'i' for integers, 'f' for floats).
- **Memory-efficient**: Arrays are stored more compactly in memory compared to lists, making them faster for numerical operations.
- **Array methods**: Arrays provide methods such as **append()**, **pop()**, **remove()**, and **extend()** to manipulate the array.

Copyright © 2025 HyperionDev. All rights reserved.

- Arrays support **indexing** and **slicing** to access or modify elements.

Python arrays are useful when working with **large, homogeneous datasets**, providing efficient memory use and performance, especially for numerical operations.

10.6 Array operations (KT1006) (IAC1001)

Arrays are used to store collections of **homogeneous elements** (elements of the same data type) and come with several operations that allow you to manipulate and work with these elements efficiently. These operations are supported by the **array** module in Python.

1. Creating an Array

- **Description**: Arrays are created using the **array()** function, specifying a **typecode** to define the data type of elements.
- **Syntax**:

```
import array
arr = array.array(typecode, [elements])
```

- **Example**:

```
arr = array.array('i', [1, 2, 3, 4])  # Integer array
```

2. Accessing Elements by Index

- **Description**: You can access individual elements in an array by their index, where indexing starts from 0 (like lists).

- **Syntax**:

```
arr[index]
```

- **Example**:

```
print(arr[2])  # Output: 3 (element at index 2)
```


Copyright © 2025 HyperionDev. All rights reserved.

3. Modifying Elements

- **Description**: You can modify the value of an element at a specific index.
- **Syntax**:

```
arr[index] = new_value
```

- **Example**:

```
arr[0] = 10
print(arr)  # Output: array('i', [10, 2, 3, 4])
```

4. Adding Elements

- **append()** – Add an Element to the End:
    - **Description**: Adds a single element to the end of the array.
    - **Syntax**:

```
arr.append(element)
```

    - **Example**:

```
arr.append(5)
print(arr)  # Output: array('i', [10, 2, 3, 4, 5])
```

- **extend()** – Add Multiple Elements:
    - **Description**: Adds multiple elements to the array from an iterable (like a list or another array).
    - **Syntax**:

```
arr.extend([elements])
```

    - **Example**:

```
arr.extend([6, 7])
print(arr)  # Output: array('i', [10, 2, 3, 4, 5, 6, 7])
```


Copyright © 2025 HyperionDev. All rights reserved.

5. Removing Elements

- **remove()** – Remove a Specific Element:
  - **Description**: Removes the first occurrence of the specified element.
  - **Syntax**:

```
arr.remove(element)
```

  - **Example**:

```
arr.remove(10)
print(arr)  # Output: array('i', [2, 3, 4, 5, 6, 7])
```

- **pop()** – Remove and Return an Element by Index:
  - **Description**: Removes and returns the element at the specified index. If no index is provided, it removes the last element.
  - **Syntax**:

```
arr.pop(index)
```

  - **Example**:

```
arr.pop(2)
print(arr)  # Output: array('i', [2, 3, 5, 6, 7])
```

6. Slicing

- **Description**: You can retrieve a **subset** of elements from the array using slicing.
- **Syntax**:

```
arr[start:end]
```

- **Example**:

```
sliced_arr = arr[1:4]
print(sliced_arr)  # Output: array('i', [3, 5, 6])
```

7. Searching for an Element

Copyright © 2025 HyperionDev. All rights reserved.

- **index()** – Find the Index of an Element:
    - **Description**: Returns the index of the first occurrence of the specified element in the array.
    - **Syntax**:

```
arr.index(element)
```

    - **Example**:

```
idx = arr.index(5)
print(idx)  # Output: 2 (index of element 5)
```

8. Reversing an Array

- **Description**: You can reverse the order of elements in an array using the **reverse()** method.
- **Syntax**:

```
arr.reverse()
```

- **Example**:

```
arr.reverse()
print(arr)  # Output: array('i', [7, 6, 5, 3, 2])
```

9. Counting Elements

- **count()** – Count Occurrences of an Element:
    - **Description**: Counts how many times a specific element appears in the array.
    - **Syntax**:

```
arr.count(element)
```

    - **Example**:

Copyright © 2025 HyperionDev. All rights reserved.

```
count = arr.count(5)
print(count)  # Output: 1 (5 appears once in the array)
```

10. Convert Array to a List

- **tolist()** – Convert an Array to a List:
    - **Description**: Converts the array into a Python list.
    - **Syntax**:

```
arr.tolist()
```

    - **Example**:

```
lst = arr.tolist()
print(lst)  # Output: [7, 6, 5, 3, 2]
```

Array Operations:

- **Creating**: Use array() to create arrays with a specified typecode and initial elements.
- **Accessing**: Retrieve elements using indexing.
- **Modifying**: Change elements at a specific index.
- **Adding**: Use append() to add a single element, and extend() to add multiple elements.
- **Removing**: Use remove() to delete specific elements, and pop() to remove elements by index.
- **Slicing**: Retrieve a subset of elements from the array.
- **Searching**: Use index() to find the position of an element.
- **Reversing**: Use reverse() to reverse the order of elements.
- **Counting**: Use count() to find the occurrences of an element.
- **Convert to List**: Use tolist() to convert the array to a list.

These operations allow you to efficiently manage and manipulate arrays for a wide range of applications in Python, particularly when dealing with **numerical data** or **large datasets.**

Copyright © 2025 HyperionDev. All rights reserved.



Formative Assessment Activity [10]

Arrays in Python

Complete the formative activity in your **KM2 Learner Workbook**,



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.

References

Beazley, D., & Jones, B. K. (2013). Python cookbook (3rd ed.). O'Reilly Media.

Lutz, M. (2010). Programming Python (4th ed.). O'Reilly Media.

Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

Matthes, E. (2019). Python crash course (2nd ed.). No Starch Press.

McKinney, W. (2017). Python for data analysis (2nd ed.). O'Reilly Media.

Martelli, A. (2017). Python in a nutshell (3rd ed.). O'Reilly Media.

Ramalho, L. (2015). Fluent Python. O'Reilly Media.

Copyright © 2025 HyperionDev. All rights reserved.